HLD

SYSTEM DESIGN INTERVIEW · SDE2 / SSE / SDE3

HLD Design Problems

> 18 problems. Answer ni గుర్తుపెట్టుకోవడం కాదు – **thinking process** ని పట్టుకోవడం. Problem చూడగానే మీ బుర్రలో ఏ order లో ఏం జరగాలో, అదే ఈ book.

ప్రతి problem కి: interviewer అడిగే విధానం → మీరు అడగాల్సిన clarifying questions → requirements → capacity estimation → API & data model → architecture diagram → deep dive → scale & failure → interview lo loud గా చెప్పాల్సిన పూర్తి English script → follow-up questions.

తెలుగు + ENGLISH

YASWANTH · INTERVIEW PREP

ఈ BOOK ఎలా వాడాలి

00

మొదట ఇది చదవండి

Read time **10 min** · ఇది skip చేస్తే మిగతాదంతా వృథా

ఈ book ఎందుకు ఇలా ఉంది

మీ దగ్గర ఇప్పటికే `HLD_Telugu.md` ఉంది – అది **concepts** నేర్పుతుంది (caching అంటే ఏమిటి, sharding అంటే ఏమిటి, CAP theorem అంటే ఏమిటి). కానీ interview lo మీకు concept అడగరు. **"Design WhatsApp"** అని ఒక వాక్యం విసిరి, 45 నిమిషాలు మీ ఆలోచనని చూస్తారు.

అక్కడ fail అయ్యేది knowledge కాదు – **order**. చాలా మంది వెంటనే boxes గీయడం మొదలుపెడతారు. Interviewer కి కావలసింది అది కాదు.

> **గుర్తుంచుకోండి:** System design interview lo మీరు evaluate అవుతున్నది "correct answer" మీద కాదు. **"ఈ మనిషితో కలిసి పని చేయగలనా?"** అన్న ప్రశ్న మీద. అంటే – clarifying questions అడుగుతారా, assumptions బయటికి చెప్తారా, trade-off ని అంగీకరిస్తారా, తప్పు చేస్తే correct చేసుకుంటారా.

అందుకే ఇక్కడ ప్రతి problem కి **అదే fixed order** follow చేశాను. మీరు ఈ 18 problems ఇలా చదివితే, 19వ కొత్త problem కూడా మీకు కొత్తగా అనిపించదు – because మీకు process వచ్చేసింది.

ప్రతి problem లో ఉండే sections

1 · The Ask
Interviewer అసలు ఏం అడుగుతున్నాడు, ఈ problem ఏ skill ని test చేస్తుంది.

2 · Clarify
మొదటి 5 నిమిషాల్లో అడగాల్సిన ప్రశ్నలు. వీటికే మార్కులు.

3 · Requirements
Functional + Non-functional, scope lo ఏముంది ఏం లేదు.

4 · Estimation
Numbers. QPS, storage, bandwidth – మొత్తం maths చూపిస్తూ.

5 · Design
API → Data model → Architecture diagram. ఈ order తప్పకూడదు.

6 · Deep Dive
అసలు problem ఇక్కడే. Options compare చేసి ఒకటి ఎంచుకోవడం.

7 · Scale & Failure
Bottleneck ఎక్కడ, ఏది down అయితే ఏమవుతుంది.

8 · Interview Script
పూర్తి English lo – నోటితో ఏం చెప్పాలో అచ్చంగా అదే.

9 · Gotchas
Follow-up ప్రశ్నలు + candidates చేసే common తప్పులు.

ఏ problem ఏం నేర్పుతుంది

ఒకే problem రెండుసార్లు అడగరు. కానీ **techniques మాత్రం repeat అవుతాయి.** అందుకే ఈ 18 ని technique ప్రకారం ఎంచుకున్నాను – ఒక్కొక్కటి ఒక్కో కొత్త ఆయుధం.

#	Problem	ఇది నేర్పే core technique
01	URL Shortener	Key generation, base62, read-heavy caching, 301 vs 302
02	Distributed Rate Limiter	Token bucket vs sliding window, Redis atomicity, sync lag
03	News Feed	**Fan-out on write vs read** – ఈ ఒక్క trade-off system design యొక్క గుండె
04	Chat System (WhatsApp)	WebSockets, presence, message ordering, delivery receipts
05	Notification System	Fan-out, dedupe, retry with backoff, DLQ, user preferences
06	Video Streaming (YouTube)	Upload → transcode pipeline, CDN, adaptive bitrate
07	Ride Sharing (Uber)	Geospatial indexing (geohash/H3), matching, trip state machine
08	Typeahead / Autocomplete	Trie, top-k, offline index build + online serving split
09	Payment System & Ledger	**Idempotency**, double-entry, saga, exactly-once illusion
10	Distributed Job Scheduler	Time-based sharding, leases, at-least-once execution
11	Web Crawler	Frontier, politeness, URL dedupe (bloom filter), traps
12	Google Docs	**OT vs CRDT** – real-time collaboration యొక్క రెండు దారులు
13	Message Queue (Kafka)	Append-only log, partitions, consumer groups, ordering
14	Key-Value Store (Dynamo)	Consistent hashing, quorum (R+W>N), vector clocks, LSM
15	Dropbox / File Sync	Chunking, content-addressed dedupe, delta sync, conflicts
16	Ticket Booking at scale	Inventory holds, overselling, hot-event thundering herd
17	Ad Click Aggregation	Stream processing, windowing, exactly-once, late events
18	Metrics & Monitoring	Time-series storage, cardinality explosion, downsampling

Universal Framework – ఏ problem కైనా ఇదే 7 అడుగులు

Timer లేకుండా interview lo వెళ్ళకండి. 45 నిమిషాల interview ని ఇలా విడగొట్టండి:

45-MINUTE HLD INTERVIEW · TIME BUDGET

MOST CANDIDATES BURN 30 MIN IN HERE

Clarify	Requirements	Estimation	API + Data	Architecture	Deep Dive	Scale + Wrap
5 min	3 min	4 min	6 min	8 min	12 min	7 min

...and never reach the deep dive, which is where seniority is judged

Deep dive కి చేరకపోతే మీరు "senior" గా కనిపించరు. అందుకే మొదటి భాగం fast గా, disciplined గా ముగించాలి.

#	అడుగు	ఇక్కడ interviewer ఏం చూస్తాడు
1	**Clarify**	Ambiguity ని గుర్తిస్తున్నారా, లేక assume చేసి పరిగెత్తుతున్నారా
2	**Requirements**	Scope ని కుదించగలరా (ఇది maturity signal)
3	**Estimation**	Numbers తో comfort. Order of magnitude చాలు
4	**API design**	Client ఏం అడుగుతుందో స్పష్టత
5	**Data model**	Access pattern → DB choice, ఈ direction lo ఆలోచిస్తున్నారా
6	**Architecture**	Components + అవి ఎందుకు ఉన్నాయో justify చేయగలరా
7	**Deep dive**	అసలు seniority test. ఒక్క component ని internals దాకా తవ్వగలరా

GOLDEN RULE

ఎప్పుడూ boxes ముందు గీయకండి. Requirements రాకముందే architecture గీస్తే, interviewer దృష్టిలో మీరు ఒక pattern ని memorize చేసి వాంతి చేస్తున్నారు. Requirements → estimation → అప్పుడే మొదటి box.

SDE2 vs SSE/SDE3 – level ని ఏది decide చేస్తుంది

ఇద్దరూ ఒకే architecture గీయొచ్చు. తేడా ఎక్కడ వస్తుందంటే:

	SDE2 జవాబు	SSE / SDE3 జవాబు
Component	"ఇక్కడ Kafka పెడతాను"	"Kafka పెడతాను ఎందుకంటే producer ని consumer నుంచి decouple చేయాలి, replay కావాలి, ordering per-key చాలు. SQS అయితే replay రాదు"
Failure	ప్రస్తావించడు	"ఈ service down అయితే requests queue lo పేరుకుంటాయి, 5 నిమిషాల తర్వాత load shed చేసి 503 ఇస్తాను"
Numbers	"చాలా data"	"రోజుకి 2 TB, 30 రోజులు hot storage lo, తర్వాత S3 Glacier"
Consistency	"DB వాడతాను"	"ఇక్కడ eventual consistency చాలు, కానీ payment path lo మాత్రం strong కావాలి – అందుకే ఆ ఒక్క భాగం వేరే store"
Trade-off	ఒకే option చెప్తాడు	రెండు options చెప్పి, **ఎందుకు ఒకటి ఎంచుకున్నాడో** చెప్తాడు

	SDE2 జవాబు	SSE / SDE3 జవాబు
Scope	అన్నీ design చేయబోతాడు	"ఇది ఇప్పుడు అవసరం లేదు, v2 lo చూద్దాం" అని కోసేస్తాడు

COMMON MISTAKES — ఈ 7 చేస్తే ROUND అక్కడే అయిపోతుంది

1. Clarifying questions అడగకుండా వెంటనే design మొదలుపెట్టడం.
2. Trade-off లేకుండా "I'll use Kafka" అనడం — ఎందుకు అన్నది లేకపోవడం.
3. ప్రతిదానికీ microservices. Interviewer scale అడగకముందే మీరే complexity తెచ్చుకోవడం.
4. Numbers ఇవ్వకపోవడం. "It will be a lot of data" ≠ "roughly 2 TB per day".
5. Failure గురించి ఒక్క మాట కూడా చెప్పకపోవడం. Happy path మాత్రమే గీయడం.
6. Interviewer hint ఇచ్చినప్పుడు దాన్ని పట్టించుకోకపోవడం. Hint అంటే "నువ్వు తప్పు దారిలో ఉన్నావు" అని అర్థం.
7. మౌనంగా ఆలోచించడం. బోర్డ్ మీద ఏమీ లేకుండా 2 నిమిషాలు ఆలోచిస్తే interviewer కి మీ బుర్ర కనిపించదు.

ప్రతి problem lo తిరిగి తిరిగి వచ్చే 6 tools

ఈ ఆరు దాదాపు అన్ని problems lo వస్తాయి. వీటిని ఒకసారి సరిగ్గా అర్థం చేసుకుంటే, ప్రతి problem lo మళ్ళీ ఆలోచించాల్సిన అవసరం లేదు.

Tool	ఒక్క వాక్యంలో	ఎక్కడ వస్తుంది
Consistent hashing	Node add/remove చేసినప్పుడు keys అన్నీ కాకుండా 1/N భాగం మాత్రమే కదులుతాయి	Cache cluster, KV store, sharding — problems 01, 13, 14
Snowflake IDs	64-bit = timestamp + machine id + sequence. Sortable, collision-free, coordination లేకుండా	Chat, feed, ledger — 03, 04, 09
Idempotency key	Client ఇచ్చే unique key ని store చేసి, retry వచ్చినప్పుడు మళ్ళీ execute చేయకపోవడం	Payment, notification, booking — 05, 09, 16
Quorum (R + W > N)	ఎన్ని replicas చదవాలి/రాయాలో tune చేసి consistency vs latency ని adjust చేయడం	KV store, multi-region — 14
CDC / Outbox	DB write + event publish ని atomic చేయడం (dual-write problem కి పరిష్కారం)	Feed, notification, analytics — 03, 05, 17
Bloom filter	"ఇది ఖచ్చితంగా లేదు" అని O(1) lo చెప్పే memory-పొదుపు structure	Crawler dedupe, LSM read path — 11, 14

PROBLEM 01 · WARM-UP

H1

Design a URL Shortener (TinyURL / bit.ly)

Difficulty **Easy-Medium** · Frequency **చాలా ఎక్కువ** · నేర్పే concepts: hashing, base62, KGS, cache, 301 vs 302, sharding

1. The Ask – interviewer అసలు ఏం అడుగుతున్నాడు

> "Design a service that takes a long URL like `https://example.com/very/long/path?utm=xyz` and returns a short one like `https://sho.rt/aB3xK9p`. Hitting the short one should redirect to the original."

ఇది **మొదటి problem** గా అందరూ అడుగుతారు – ఎందుకంటే ఇది మోసపూరితంగా సులభం. Feature ఒక్కటే: string in, string out. కానీ interviewer చూసేది feature ని కాదు.

ఇది ఎందుకు అడుగుతారు

Read-heavy system ని 100:1 ratio తో handle చేయగలరా – cache, replica, CDN ఆలోచన వస్తుందా.

అసలు core

Unique short key ని distributed servers మీద collision లేకుండా ఎలా generate చేస్తారు. అదే ఈ problem యొక్క గుండె.

Hidden test

301 vs 302 redirect. దీన్ని సరిగ్గా చెప్పే candidate 10 మందిలో ఒకరు.

2. Clarifying Questions – మొదటి 5 నిమిషాలు

ఇక్కడ **నోరు మూసుకుని design చేయకూడదు**. ఈ ప్రశ్నలు అడగండి, జవాబులు board మీద రాయండి.

మీరు అడగాల్సినది	ఎందుకు ఇది ముఖ్యం	సాధారణంగా వచ్చే జవాబు
రోజుకి ఎన్ని URLs create అవుతాయి?	మొత్తం estimation, storage ఇక్కడి నుంచే మొదలు	~500M per month
Read : Write ratio ఎంత?	Cache కావాలా వద్దా అన్నది ఇది decide చేస్తుంది	100 : 1
Short link ఎంత కాలం బతకాలి?	Expiry ఉంటే cleanup job + storage తగ్గుతుంది	Default 5 years
User తనకు నచ్చిన custom alias ఇవ్వొచ్చా?	ఇది ఒక పూర్తిగా వేరే write path	అవును, optional
Analytics (click count, geo) కావాలా?	301 vs 302 decision దీని మీద ఆధారపడి ఉంటుంది	అవును, basic

మీరు అడగాల్సినది	ఎందుకు ఇది ముఖ్యం	సాధారణంగా వచ్చే జవాబు
Link update / delete చేయగలరా?	Immutable అయితే caching చాలా సులభం	ఇప్పుడు వద్దు
Latency target ఏమిటి?	Redirect ఎంత fast ఉండాలో number ఇస్తుంది	p99 < 100ms

ఇక్కడ ఒక చిన్న TRICK

ప్రశ్న అడిగాక **మీ assumption కూడా చెప్పండి** — "Read:write 100:1 అనుకుంటున్నాను, అంటే ఇది read-heavy system, కాబట్టి cache ఖచ్చితంగా వస్తుంది." ఇలా చెప్తే interviewer కి మీ ఆలోచన కనిపిస్తుంది. మౌనంగా ఆలోచించడం interview lo సున్నా మార్కులు.

3. Requirements

Functional

- Long URL → short URL create చేయాలి
- Short URL hit అయితే original కి redirect
- Custom alias (optional)
- Expiry time (optional, default 5y)
- Basic click analytics

Non-Functional

- **High availability** — redirect down అయితే అన్ని links చచ్చినట్టే
- Redirect latency p99 < 100ms
- Short key predictable గా ఉండకూడదు (security)
- Horizontally scalable

OUT OF SCOPE — ఇలా బయటికి చెప్పండి

User accounts & social features, link preview generation, malware scanning, QR codes, payments. ఇవి scope lo లేవని **మీరే ముందు చెప్పడం** maturity signal — interviewer అడిగే వరకు ఆగకండి.

4. Capacity Estimation — numbers బోర్డ్ మీద రాయండి

Estimation lo perfection అవసరం లేదు. **Order of magnitude** చాలు. కానీ maths మాత్రం బయటికి కనిపించాలి.

Step	లెక్క	ఫలితం
Writes / month	ఇచ్చినది	500 M
Write QPS	500M ÷ (30 × 86,400)	**≈ 200 / sec**
Read QPS	200 × 100	**≈ 20,000 / sec**
Peak read QPS	avg × 3 (peak factor)	≈ 60,000 / sec
ఒక record size	shortKey 7B + longURL 500B + metadata ~100B	≈ 600 bytes

Step	లెక్క	ఫలితం
Storage / month	500M × 600 B	≈ 300 GB
Storage 5 years	300 GB × 60 నెలలు	**≈ 18 TB**
Read bandwidth	20,000 × 600 B	≈ 12 MB / sec
Cache size (80/20 rule)	రోజు reads 1.7B × 20% × 600B	≈ 200 GB → shard చేసిన Redis cluster

KEY SPACE – ఇది ఖచ్చితంగా చూపించాలి

Base62 = `a–z A–Z 0–9` = 62 characters.

6 chars → 62^6 ≈ **56.8 billion** · 7 chars → 62^7 ≈ **3.5 trillion**
మనకి 5 ఏళ్ళకి కావలసినది 500M × 60 = **30 billion**. 6 chars సరిపోతుంది కానీ 53% వాడేస్తాం – collision rate పెరుగుతుంది. అందుకే **7 characters** ఎంచుకుంటున్నాను. ఇలా number తో justify చేస్తే అది senior answer.

5. API Design

Architecture కంటే ముందు API. ఎందుకంటే API నే మీ system యొక్క contract.

```
POST /api/v1/urls
Content-Type: application/json
Authorization: Bearer <token>

{
  "longUrl":     "https://example.com/very/long/path?utm=xyz",
  "customAlias": "my-link",        // optional
  "expiresAt":   "2031-01-01T00:00:00Z"   // optional
}

201 Created
{ "shortUrl": "https://sho.rt/aB3xK9p", "expiresAt": "2031-01-01T00:00:00Z" }

409 Conflict     → customAlias ఇప్పటికే వాడుకలో ఉంది
429 Too Many     → rate limit దాటింది
```

```
GET /aB3xK9p

302 Found
Location: https://example.com/very/long/path?utm=xyz
Cache-Control: private, max-age=300
```

RATE LIMITING ని ఇక్కడే చెప్పండి

Create API కి **తప్పనిసరిగా** rate limit కావాలి – లేకపోతే ఒక్క script మీ మొత్తం key space ని మింగేస్తుంది. Per user/API-key: గంటకి 100 links. దీన్ని మీరే చెప్తే, interviewer "abuse గురించి ఆలోచించాడు" అని note చేసుకుంటాడు.

6. Data Model – DB choice ని access pattern నుంచి derive చేయండి

మన access pattern ఒక్కటే: `shortKey` **ఇచ్చి** `longUrl` **తీసుకోవడం.** Joins లేవు, range queries లేవు, complex transactions లేవు. ఇది స్వచ్ఛమైన **key-value lookup**.

Table	Column	Type	Note
`urls`	`short_key`	varchar(7)	**Primary key / partition key**
	`long_url`	varchar(2048)	
	`user_id`	bigint	nullable (anonymous links)
	`created_at`	timestamp	
	`expires_at`	timestamp	null = never

Option	సరిపోతుందా?	ఎందుకు
PostgreSQL / MySQL	పర్లేదు	ACID, custom alias uniqueness సులభం. కానీ 18 TB కి manual sharding కావాలి
Cassandra / DynamoDB	✅ ఇదే మంచిది	Partition key మీద O(1) lookup, auto-sharding, 20K QPS ని తేలిగ్గా తట్టుకుంటుంది
MongoDB	పర్లేదు	పని చేస్తుంది కానీ ఇక్కడ document model వల్ల ఎలాంటి లాభమూ లేదు
Redis (only)	❌ కాదు	Primary store గా వద్దు – 18 TB RAM ఖరీదు దారుణం, durability risk

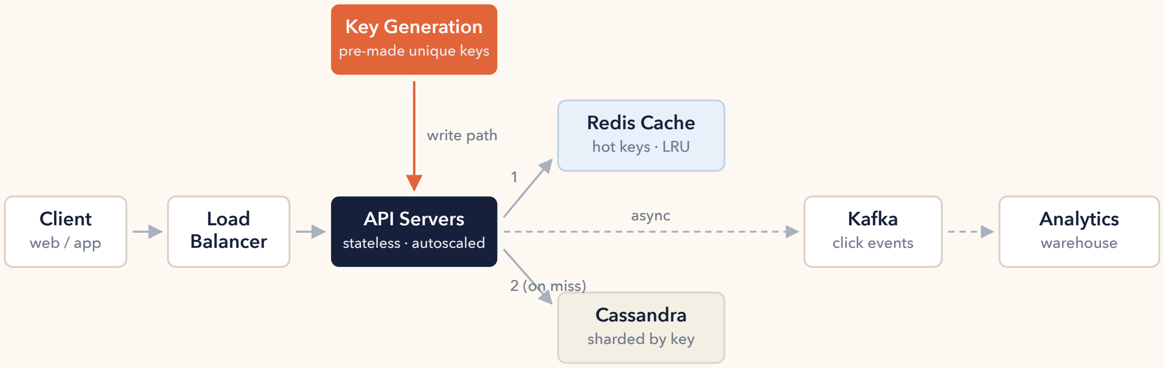
నా choice: Cassandra (లేదా DynamoDB), partition key = `short_key`. కారణం – access pattern సరిగ్గా key-value, write throughput ఎక్కువ, cross-partition transaction అవసరం లేదు.

CUSTOM ALIAS కి ఒక CATCH ఉంది

Cassandra lo సాధారణ insert last-write-wins – అంటే ఇద్దరు ఒకే alias అడిగితే ఒకరు రెండోవాడి link ని silent గా తుడిచేస్తారు. దీనికి **lightweight transaction** (`INSERT ... IF NOT EXISTS`) వాడాలి. ఇది Paxos వాడుతుంది కాబట్టి నెమ్మది – కానీ custom alias అరుదు కాబట్టి పర్లేదు. ఈ ఒక్క వాక్యం చెప్తే మీరు distributed systems తెలిసినవారని అర్థమవుతుంది.

7. High-Level Architecture

URL SHORTENER · HIGH LEVEL ARCHITECTURE



Read path: Client → LB → API → Cache (hit అయితే అక్కడితో అయిపోయింది) → miss అయితే Cassandra. Analytics ఎప్పుడూ **async** – click count రాయడం కోసం redirect ని ఆపకూడదు.

ప్రతి box ఎందుకు ఉందో ఒక్క వాక్యంలో చెప్పగలగాలి:

Component	ఎందుకు ఉంది	ఇది తీసేస్తే ఏమవుతుంది
Load Balancer	Traffic ని stateless API servers మీద పంచడం + health check	ఒక server చస్తే ఆ traffic అంతా చస్తుంది
API Servers (stateless)	Session state లేదు కాబట్టి ఎన్నైనా జోడించొచ్చు	Sticky sessions వల్ల scaling కష్టం
Key Generation Service	Distributed servers మధ్య collision లేకుండా unique key	Write path lo ప్రతిసారీ "ఈ key ఖాళీగా ఉందా" అని DB చదవాలి
Redis Cache	20K read QPS lo 90%+ ని DB కి చేరనివ్వకుండా ఆపుతుంది	DB మీద 20K QPS – ఖరీదు, latency రెండూ పెరుగుతాయి
Kafka	Analytics write ని redirect path నుంచి వేరు చేస్తుంది	Analytics DB నెమ్మదిస్తే user redirect నెమ్మదిస్తుంది

8. Deep Dive – Short key ఎలా పుట్టాలి?

ఇదే ఈ problem యొక్క అసలు గుండె. ఇక్కడ 12 నిమిషాలు గడపండి. మిగతా అంతా ఏ system design కైనా common – ఇది మాత్రం ఈ problem కే ప్రత్యేకం.

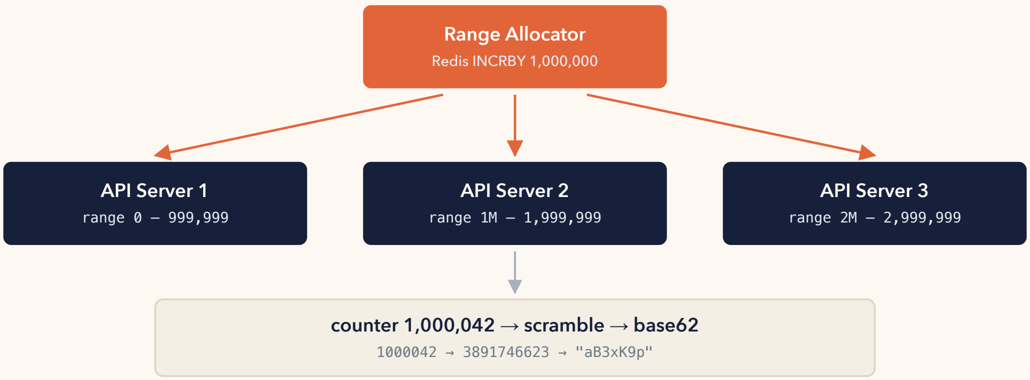
సమస్య ఇది: 200 writes/sec, 50 API servers, ఎవరూ ఒకరితో ఒకరు మాట్లాడుకోకుండా – **అందరూ unique 7-character key ఇవ్వాలి.**

Approach	ఎలా పని చేస్తుంది	ప్లస్	మైనస్
A. Hash(longUrl)	MD5/SHA-256 తీసి మొదటి 43 bits → base62 → 7 chars	ఒకే URL కి ఎప్పుడూ ఒకే key (dedupe free)	**Collision తప్పదు.** ప్రతి write కి "ఈ key ఉందా?" అని DB read. అదే URL ని ఇద్దరు users వేరే వేరే expiry తో shorten చేయలేరు
B. Random 7 chars	Secure random → base62	Predictable కాదు, code చాలా simple	Key space నిండుతున్న కొద్దీ collision ఎక్కువ → ప్రతి write కి 1+ DB read. 20 TB DB మీద అది ఖరీదు
C. Global auto-increment	ఒక counter → base62 encode	Collision **సున్నా**, key చిన్నది	Single point of failure + **keys sequential** → `aB3xK9p` తర్వాత ఏమిందో ఊహించి అందరి links ని scrape చేయొచ్చు
D. Counter ranges (KGS)	Redis/ZooKeeper ఒక్క server కి 1M numbers range ఇస్తుంది. Server ఆ range ని local గా వాడుతుంది	Collision సున్నా, DB read సున్నా, network call అరుదు (1M కి ఒకసారి)	Server crash అయితే ఆ range వృథా (కానీ 3.5 trillion lo 1M వృథా అంటే లెక్కలోకి రాదు)

నా CHOICE · APPROACH D + ఒక చిన్న TWIST

Counter ranges తీసుకుంటాను, కానీ counter ని నేరుగా base62 చేయను – ముందు **reversible bijective transform** (ఉదా. Feistel network / multiply-by-large-coprime mod 62^7) వేసి తర్వాత encode చేస్తాను. దీంతో collision సున్నా అలాగే ఉంటుంది, కానీ keys sequential గా కనిపించవు. Approach C యొక్క ఏకైక పెద్ద లోపం ఇలా పోతుంది.

KEY GENERATION SERVICE · RANGE ALLOCATION

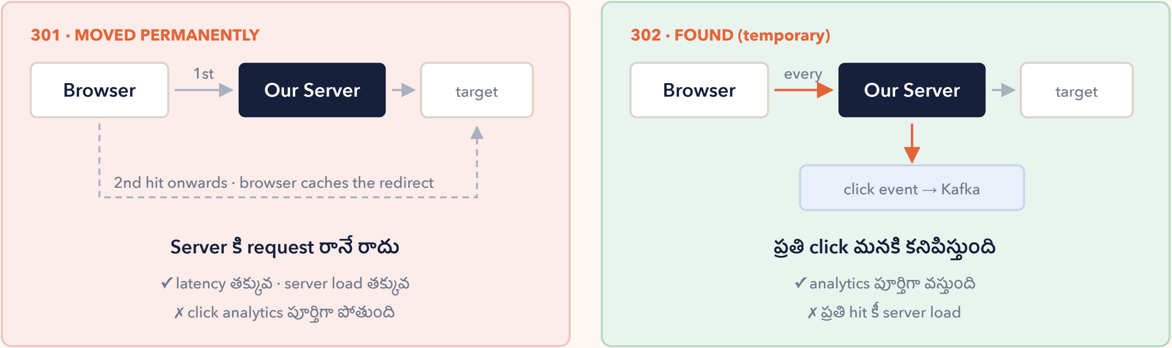


ఒక్క server తన range ని memory lo వాడుకుంటుంది. అంటే **ప్రతి write కి network call లేదు, DB read లేదు** – 1 million writes కి ఒకే ఒక్క Redis call. ఇదే ఈ approach యొక్క అందం.

9. Redirect – 301 నా 302 నా? (దీన్ని 90% మంది తప్పుతారు)

ఇది చిన్న detail లాగా కనిపిస్తుంది. కానీ ఇది **ఒక product decision** – మరియు interviewer ఖచ్చితంగా అడుగుతాడు.

301 PERMANENT VS 302 FOUND · ఏం మారుతుంది



చెప్పాల్సిన జవాబు

"Analytics మనకి functional requirement కాబట్టి నేను **302** ఎంచుకుంటాను – 301 ఇస్తే browser cache చేసేసి రెండో click నుంచి మన server కి request రాదు, అప్పుడు click count ఎప్పటికీ 1 దగ్గరే ఆగిపోతుంది. Server load పెరిగే ఖర్చుని Redis cache తో సర్దుకుంటాను. ఒకవేళ analytics అవసరం లేని plan అయితే 301 ఇస్తాను – అప్పుడు అది ఖరీదు తగ్గించే optimization."

10. Scale, Bottlenecks & Failure

ఎక్కడ నొప్పి	లక్షణం	పరిష్కారం
DB read	Cache miss ఎక్కువై Cassandra latency పెరగడం	Read replicas + cache TTL పెంచడం. Data immutable కాబట్టి TTL ధైర్యంగా పెద్దది పెట్టొచ్చు
Hot link	ఒక viral link కి secondకి 50K hits	ఆ ఒక్క Redis shard వేడెక్కుతుంది → API server memory lo local LRU (L1 cache) పెట్టండి. లేదా CDN edge lo redirect
Cache cold start	Redis restart అయితే మొత్తం load DB మీద పడి DB కూలుతుంది	Warm-up job + request coalescing (ఒకే key కి 1000 concurrent miss వస్తే DB కి ఒకే ఒక్క query)
Key space అయిపోవడం	5 ఏళ్ళ తర్వాత	7 → 8 chars కి పెంచడం. కొత్త keys పొడవుగా వస్తాయి, పాతవి పని చేస్తూనే ఉంటాయి
Expired links	18 TB lo చాలా భాగం చచ్చిన data	**Lazy deletion** – read time lo expired అయితే 404 ఇచ్చి delete queue కి పంపడం. ప్లస్ రాత్రి cleanup job
Region outage	ఒక AWS region పోవడం	Multi-region active-active. Data immutable కాబట్టి async replication తో conflict ఏమీ ఉండదు – ఇది ఈ problem యొక్క పెద్ద అదృష్టం

ఇక్కడ ఒక బలమైన POINT చెప్పండి

"ఈ system యొక్క data **immutable** – ఒకసారి short key create అయ్యాక అది ఎప్పుడూ మారదు. అందుకే caching, replication, multi-region అన్నీ trivial అవుతాయి. Consistency సమస్యే లేదు – eventual consistency సరిపోతుంది, ఎందుకంటే key ఇంకా replicate కాకముందు ఎవరూ దాన్ని hit చేయలేరు." – ఈ observation చెప్పే candidate చాలా తక్కువ.

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me start by confirming scope. I'll assume 500 million new links a month, a read-to-write ratio of about 100 to 1, links live for five years by default, custom aliases are supported, and we need basic click analytics. Is that reasonable?"

"That gives me roughly **200 writes per second and 20,000 reads per second**, so this is overwhelmingly a read-heavy system – caching is going to matter far more than write throughput. At 600 bytes a record, five years of data is about **18 terabytes**, which is too big for a single node, so I'll need sharding."

"On key length: base62 with 7 characters gives 3.5 trillion combinations against the 30 billion we need, so we stay under 1% utilisation and collisions stay negligible. Six characters would work numerically but we'd be at 53% utilisation, and I'd rather not design for that."

"The interesting problem here is generating unique keys across many stateless servers without coordinating on every write. Hashing the URL means a database check on every write to detect collisions. Random keys have the same problem, and it gets worse as the key space fills. A single global counter removes collisions entirely but is a single point of failure, and the keys come out sequential, which means anyone can enumerate every link in the system."

"So I'll use **counter ranges**. Redis hands each API server a block of a million integers, the server consumes that block from local memory, and only asks for a new block once it's exhausted. That's one network call per million writes, zero database reads, and zero collisions. To fix the enumeration problem, I run the counter through a reversible scramble before base62-encoding it, so the output looks random but is still guaranteed unique."

"For storage I'd pick Cassandra or DynamoDB, partitioned on the short key – our access pattern is a pure key-value lookup with no joins and no range scans. Custom aliases are the one exception: those need **insert-if-not-exists** so two users can't silently overwrite each other."

"For the redirect I'll return **302, not 301**. A 301 gets cached by the browser, so from the second click onwards the request never reaches us and our analytics stop at one. Since analytics is a stated requirement, 302 is the right call, and I'll absorb the extra load with Redis."

"One property that makes everything easier: this data is immutable. A short key never changes after creation, so I can cache aggressively, replicate asynchronously across regions, and live comfortably with eventual consistency."

"If we have time, the two things I'd dig into next are hot-link handling – a single viral link overloading one cache shard – and how we expire and reclaim dead links."

11. Follow-up ప్రశ్నలు – వీటికి సిద్ధంగా ఉండండి

Interviewer అడిగేది	మీ జవాబు (సారాంశం)
"ఒకే URL ని ఇద్దరు shorten చేస్తే?"	Default గా రెండు వేరే keys – ఎందుకంటే ఒక్కొక్కరికి వేరే expiry, వేరే analytics. Dedupe కావాలంటే `(user_id, long_url)` మీద ఒక secondary index, కానీ అది ఒక extra write
"Analytics ని ఎలా design చేస్తారు?"	Redirect path lo Kafka కి fire-and-forget event. Consumer అది aggregate చేసి time-series store lo రాస్తుంది. Exact count అవసరం లేదు కాబట్టి HyperLogLog తో unique visitors ని approximate చేయొచ్చు
"Malicious URL లు?"	Create time lo Google Safe Browsing API async check. అనుమానం ఉంటే redirect ముందు interstitial warning page
"Redis పూర్తిగా పోతే?"	System బతికే ఉంటుంది, కానీ latency పెరుగుతుంది. అందుకే API server lo చిన్న local LRU (L1) ఉంచడం – Redis చనిపోయినా hot links పని చేస్తాయి
"Custom alias ని ఎలా validate చేస్తారు?"	Length + charset check, reserved words blocklist (`api`, `admin`, `login`), profanity filter, తర్వాత conditional insert
"1 billion writes/day కి scale చేయాలంటే?"	Range size పెంచడం (1M → 10M), KGS ని multi-region చేయడం, key ని 8 chars కి పెంచడం. Architecture మారదు – అదే ఈ design యొక్క బలం

ముగింపులో ఇలా చెప్పండి

"To summarise: the system is read-heavy, the data is immutable, and the only genuinely hard part is unique key generation – which I solved with pre-allocated counter ranges plus a reversible scramble. Everything else is caching and sharding." – **ఒక వాక్యంలో summary** ఇవ్వడం interview ని బలంగా ముగిస్తుంది.

PROBLEM 02 · ALGORITHMS + DISTRIBUTED STATE

02

Design a Distributed Rate Limiter

Difficulty **Medium** · Frequency **చాలా ఎక్కువ** · నేర్పే concepts: token bucket, sliding window, Redis atomicity, fail-open

1. The Ask

> "Design a rate limiter that allows at most N requests per user per minute, across a fleet of API servers."

ఇది మోసపూరితంగా చిన్న problem లా కనిపిస్తుంది – "counter పెంచు, limit దాటితే 429 ఇవ్వు". కానీ ఇక్కడ **రెండు నిజమైన కష్టాలు** ఉన్నాయి:

కష్టం 1 · Algorithm
"నిమిషానికి 100" అంటే అర్థం ఏమిటి? Clock నిమిషమా, rolling నిమిషమా? ఈ ఒక్క ప్రశ్న 5 వేర్వేరు algorithms కి దారితీస్తుంది.

కష్టం 2 · Distributed state
50 API servers ఉన్నాయి. Counter ఎక్కడ ఉంటుంది? అందరూ Redis కి వెళ్తే latency + SPOF. Local గా ఉంచితే limit 50 రెట్లు తప్పు.

Hidden test
Rate limiter down అయితే ఏం చేస్తారు? Fail-open నా fail-closed నా? దీనికి జవాబు ఇచ్చే candidate చాలా అరుదు.

2. Clarifying Questions

మీరు అడగాల్సినది	ఎందుకు ముఖ్యం	సాధారణ జవాబు
ఎవరిని limit చేస్తున్నాం – user, IP, API key?	Key design ఇక్కడి నుంచే	API key (+ IP anonymous కి)
Burst allow చేయాలా, లేక smooth గా ఉండాలా?	Token bucket vs leaky bucket ని ఇది decide చేస్తుంది	కొంత burst ఫర్వాలేదు
Limit దాటితే – reject నా queue నా?	Queue అంటే latency, reject అంటే error	Reject with 429
ప్రతి endpoint కి వేరే limit ఉందా?	Rule engine కావాలా అన్నది	అవును, tier-based
Approximate సరిపోతుందా, exact కావాలా?	ఇదే అతి ముఖ్యమైన ప్రశ్న – exact అంటే ఖరీదు	~1% error ఫర్వాలేదు
Rate limiter down అయితే?	Fail-open vs fail-closed	Fail-open (availability ముఖ్యం)

"APPROXIMATE సరిపోతుందా?" – ఈ ప్రశ్న అడిగితే మీరు వేరే LEVEL

Exact rate limiting అంటే ప్రతి request కీ distributed consensus. అది చాలా ఖరీదు. నిజ ప్రపంచంలో Stripe, GitHub, AWS – అందరూ **approximate** limiters వాడతారు, ఎందుకంటే 100 బదులు 103 requests వెళ్ళినా ఆకాశం విరిగిపడదు. ఈ మాట interview lo చెప్తే over-engineering నుంచి బయటపడతారు.

3. Requirements & Estimation

Functional

- Per-key limit enforce చేయాలి
- 429 + `Retry-After` header
- Tier ప్రకారం వేరే rules
- Rules ని runtime lo మార్చగలగాలి

Non-Functional

- Added latency **< 5 ms p99** – ఇది hot path
- Rate limiter చస్తే API చావకూడదు
- Memory: 1M active keys × ~50 B ≈ **50 MB** – Redis కి పలుచన

లెక్క	విలువ
API traffic	100,000 QPS
Active keys (users/API keys)	1 M
ప్రతి key కి state	counter + timestamp ≈ 50 bytes
మొత్తం memory	≈ 50 MB – ఒక్క Redis node lo సునాయాసంగా పడుతుంది
Redis కి extra round trip	~0.5 ms same-AZ – ఇది budget lo సరిపోతుంది

4. Deep Dive – 5 Algorithms

ఈ problem యొక్క గుండె ఇదే. ఇక్కడ 12 నిమిషాలు గడపండి.

ALGORITHM 1 & 2 · FIXED WINDOW కి ఉన్న BOUNDARY PROBLEM

FIXED WINDOW COUNTER · limit = 100 / minute

SLIDING WINDOW LOG · exact కానీ ఖరీదు

ప్రతి request timestamp ని sorted set lo store చేసి, ప్రతిసారీ 1 నిమిషం కంటే పాతవాటిని తీసేయడం.
✓ ఖచ్చితమైనది · ✗ ఒక్కో key కి 100 timestamps – 1M keys అంటే memory పేలిపోతుంది

ALGORITHM 4 · TOKEN BUCKET — ఇదే INDUSTRY DEFAULT

రెండే numbers store చేస్తే చాలు: `tokens` మరియు `lastRefillTime`. Request వచ్చినప్పుడు గడిచిన కాలానికి సరిపడా tokens కలిపి, తర్వాత ఒకటి తీసేయడం — O(1) time, O(1) memory.

Algorithm	Memory / key	ఖచ్చితత్వం	Burst	ఎప్పుడు వాడాలి
Fixed window counter	1 number	❌ boundary lo 2×	అనుకోకుండా	సులభం కావాలి, ఖచ్చితత్వం అవసరం లేదు
Sliding window log	N timestamps	✅ perfect	లేదు	Limit చాలా చిన్నది (ఉదా. గంటకి 5)
Sliding window counter	2 numbers	~99%	లేదు	**General purpose best** — memory తక్కువ, boundary bug లేదు
Token bucket	2 numbers	✅	✅ controlled	**API rate limiting కి default.** Stripe, AWS ఇదే వాడతాయి
Leaky bucket	queue	✅	❌ smooth output	Downstream ని constant rate lo protect చేయాలంటే

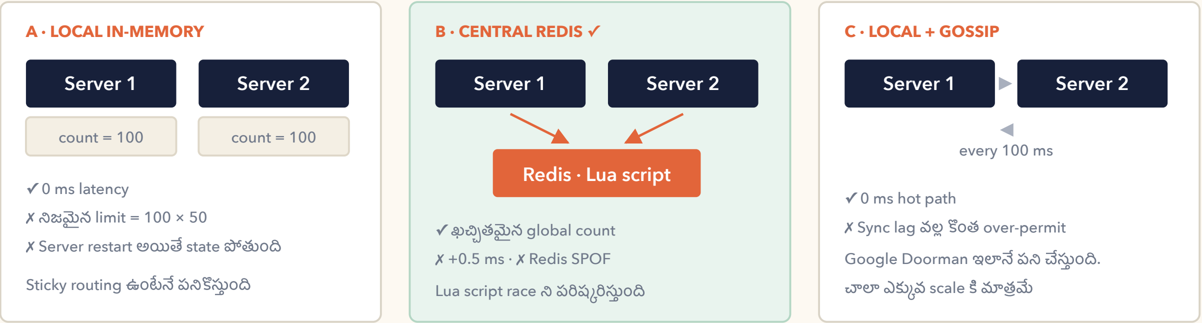
నా CHOICE

Token bucket. కారణం — (1) ఒక్కో key కి రెండే numbers, memory పలుచన; (2) O(1) computation; (3) burst ని *controlled* గా allow చేస్తుంది, ఇది నిజమైన API వాడకానికి సరిపోతుంది (ఒక client 10 requests ఒక్కసారి పంపి తర్వాత ఆగడం సహజం); (4) రెండు knobs — `capacity` (burst ఎంత) మరియు `refillRate` (sustained rate ఎంత) — వీటిని విడిగా tune చేయొచ్చు. Fixed window ని **boundary problem వల్ల తిరస్కరిస్తున్నాను** అని స్పష్టంగా చెప్పండి.

5. Deep Dive — Counter ఎక్కడ ఉంచాలి?

50 API servers, ఒక్కో దానికీ 2000 QPS. Counter state ఎక్కడ?

THREE PLACEMENTS · LATENCY VS ACCURACY



నేను B ఎంచుకుంటాను – 0.5 ms latency budget lo ఇమిడిపోతుంది, code సులభం. QPS 1M దాటినప్పుడు మాత్రమే C గురించి ఆలోచిస్తాను, ఎందుకంటే అప్పుడు Redis round-trip ఖరీదు అవుతుంది.

Redis lo race condition – ఇది తప్పకుండా ప్రస్తావించండి

```
-- ❌ తప్పు: GET → లెక్క → SET. మధ్యలో వేరే server కూడా అదే చేస్తుంది.
-- ✓ సరైనది: మొత్తం logic ని Redis లోపలే ఒకే atomic step గా నడపడం.

-- token_bucket.lua  (KEYS[1] = "rl:user:42", ARGV = capacity, rate, now, cost)
local bucket   = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local capacity = tonumber(ARGV[1])
local rate     = tonumber(ARGV[2])   -- సెకనుకి tokens
local now      = tonumber(ARGV[3])
local cost     = tonumber(ARGV[4])

local tokens = tonumber(bucket[1]) or capacity
local ts     = tonumber(bucket[2]) or now

tokens = math.min(capacity, tokens + (now - ts) * rate)   -- గడిచిన కాలానికి refill

if tokens < cost then
  return {0, tokens}                                       -- 429
end

tokens = tokens - cost
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', KEYS[1], math.ceil(capacity / rate) * 2)  -- idle keys ని
వదిలించుకోవడం
return {1, tokens}
```

ఈ CODE LO మూడు SENIOR DETAILS ఉన్నాయి

1. Lua script – Redis single-threaded కాబట్టి script మొత్తం atomic. `WATCH/MULTI` కంటే ఇది simple, వేగం కూడా ఎక్కువ.
2. Lazy refill – background timer తో tokens నింపడం లేదు. Request వచ్చినప్పుడే "ఇంతసేపట్లో ఎన్ని రావాలో" లెక్కిస్తున్నాం. 1M keys కి timers పెట్టడం అసాధ్యం, ఇది O(1).
3. EXPIRE – ఇది లేకపోతే inactive keys Redis lo శాశ్వతంగా పేరుకుని memory తినేస్తాయి. చాలా మంది దీన్ని మర్చిపోతారు.

6. API Response – ఇది కూడా design lo భాగం

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1735689600
```

Client కి **ఎప్పుడు మళ్ళీ try చేయాలో** చెప్పకపోతే, అది వెంటనే retry చేసి మీ system ని ఇంకా ఎక్కువ కొడుతుంది. `Retry-After` ఇవ్వడం ఒక్క header కాదు – అది **backpressure mechanism**.

7. Scale & Failure

సమస్య	ఏమవుతుంది	పరిష్కారం
Redis down	ప్రతి request కి error వస్తుంది	**Fail-open** – limiter చస్తే traffic ని allow చెయ్యి. Rate limiter ఒక optimisation, అది core business కాదు. (కానీ login/payment లాంటి చోట fail-closed)
Hot key	ఒక పెద్ద customer ఒక్క Redis shard ని వేడెక్కిస్తాడు	Key ని `user:42:{shard}` గా విడగొట్టి, limit ని shards మధ్య పంచడం
Redis latency spike	API p99 పెరుగుతుంది	Timeout 5 ms పెట్టి, timeout అయితే allow చేయడం (fail-open)
Rules deployment	కొత్త limit ఇవ్వడానికి deploy కావాలా?	Rules ని config store (etcd/S3) lo పెట్టి servers 30 సెకన్లకోసారి poll చేయడం
Multi-region	Region-1 lo 100, region-2 lo 100 → మొత్తం 200	Region కి limit ని విభజించడం (ఒక్కో దానికి 50), లేదా global limit ని ఒకే region lo ఉంచడం. Cross-region sync విలువ చేయదు

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"First, some scoping. Are we limiting per user, per IP, or per API key? Do we want to allow bursts or smooth traffic out? And the question that actually shapes the design – does the limit need to be exact, or is roughly right good enough?"

"I'll assume approximate is fine. Exact distributed rate limiting requires consensus on every request, and nobody does that in production – Stripe, GitHub and AWS all run approximate limiters, because letting 103 requests through instead of 100 costs nothing."

"On algorithms, the naive answer is a fixed window counter, and I want to reject it explicitly: it has a **boundary problem**. A client can send 100 requests at 10:00:59 and another 100 at 10:01:01 – two hundred requests in two seconds against a limit of a hundred per minute."

"A sliding window log fixes that exactly, but it stores every request timestamp. At a million keys that's far too much memory for what we get."

"I'd go with a **token bucket**. It stores two numbers per key – the token count and the last refill timestamp – and refills lazily on read rather than with a background timer, which is what makes it O(1) in both time and memory. It also gives me two independent knobs: capacity controls how big a burst I tolerate, and refill rate controls the sustained throughput."

"The harder half of this problem is where the counter lives. With fifty stateless API servers, keeping counters locally means the effective limit is fifty times what we intended. So the state goes in Redis."

"But a naive Redis implementation has a race: read the count, compute, write it back – two servers interleave and both think they're under the limit. I'd put the whole read-compute-write into a **Lua script**, which Redis executes atomically because it's single-threaded. I'd also set an EXPIRE on the key, otherwise idle users accumulate in memory forever."

"On failure: if Redis is unavailable, I **fail open**. The rate limiter protects us from abuse, but it isn't the product – refusing all traffic because the limiter is down turns a partial outage into a total one. I'd make that configurable per rule though, because for something like a login endpoint or a payment API I'd rather fail closed."

"Finally, on the response: a 429 should always carry **Retry-After**. Without it, clients retry immediately and make the overload worse. That header is part of the backpressure design, not just documentation."

8. Follow-ups

అడిగేది	జవాబు
"Client-side rate limiting కూడా పెట్టాలా?"	పెట్టొచ్చు, కానీ దాన్ని ఎప్పుడూ నమ్మకూడదు – client మీ control lo లేదు. అది optimisation మాత్రమే, enforcement కాదు
"ఒక్కో endpoint కి వేరే limit?"	Key lo endpoint ని కలపడం: `rl:{apiKey}:{endpoint}`. Rules ని ordered list గా evaluate చేసి మొదటి match తీసుకోవడం

అడిగేది	జవాబు
"Gateway lo నా service lo?"	**Gateway lo** – అలా అయితే ప్రతి service తనకి తానుగా implement చేయనవసరం లేదు, పైగా malicious traffic backend కి చేరకముందే ఆగుతుంది
"Sliding window counter ఎలా పని చేస్తుంది?"	ప్రస్తుత window count + మునుపటి window count × (overlap ratio). రెండే numbers, boundary bug లేదు, ~99% ఖచ్చితం
"Distributed lock వాడొచ్చా?"	వద్దు. ప్రతి request కీ lock అంటే throughput చచ్చిపోతుంది. Redis Lua single-threaded atomicity ఇక్కడ lock కంటే చాలా చౌక
"Abuse detection తో ఎలా కలపాలి?"	429 events ని stream lo పంపి, ఒక detector నిరంతరం limit కొడుతున్న keys ని గుర్తించి ఎక్కువ కాలం block చేయడం (tarpit)

PROBLEM 03 · THE MOST IMPORTANT TRADE-OFF

03

Design a News Feed (Twitter / Instagram)

Difficulty **Hard** · Frequency **అత్యధికం** · నేర్పే concepts: fan-out write vs read, hybrid, celebrity problem, feed cache

1. The Ask

> "Design the home timeline for Twitter. A user opens the app and sees recent posts from everyone they follow, newest first."

ఇది ఎందుకు అన్నిటికంటే ముఖ్యమైన PROBLEM

ఈ problem lo ఉన్న **fan-out on write vs fan-out on read** అనే trade-off – system design మొత్తంలో అతి ముఖ్యమైన pattern. ఇదే idea Instagram feed, YouTube subscriptions, LinkedIn timeline, Slack channels, notification systems – అన్నిచోట్లా తిరిగి వస్తుంది. ఇది ఒక్కసారి సరిగ్గా అర్థమైతే, 5 వేరే problems కి జవాబు వచ్చినట్టే.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	సాధారణ జవాబు
Feed chronological నా ranked నా?	Ranked అంటే ML scoring pipeline మొత్తం అదనంగా వస్తుంది	ముందు chronological, తర్వాత ranking
ఒక user సగటున ఎంత మందిని follow చేస్తాడు?	Fan-out ఖర్చు ఇక్కడి నుంచే	~200
గరిష్ఠంగా ఎంత మంది followers?	ఇదే celebrity problem ని బయటపెట్టే ప్రశ్న	100 M+
Post lo ఏముంటుంది – text, image, video?	Media storage + CDN అవసరమా	Text + images
Feed ఎంత తాజాగా ఉండాలి?	Seconds అంటే push, నిమిషాలు అంటే pull కూడా చాలు	కొన్ని సెకన్లు
Edit / delete ఉందా?	ఉంటే fan-out చేసిన copies అన్నీ సరిచేయాలి	Delete ఉంది, edit లేదు

"గరిష్ఠంగా ఎంత మంది FOLLOWERS?" — ఈ ఒక్క ప్రశ్న INTERVIEW ని మారుస్తుంది

సగటు 200. కానీ గరిష్ఠం 100 million. ఈ **అసమానతే** ఈ problem ని కష్టతరం చేస్తుంది. ఇది మీరే గుర్తించి బయటికి చెప్తే, interviewer వెంటనే మీరు దీన్ని ఇంతకుముందు ఆలోచించారని అర్థం చేసుకుంటాడు.

3. Estimation

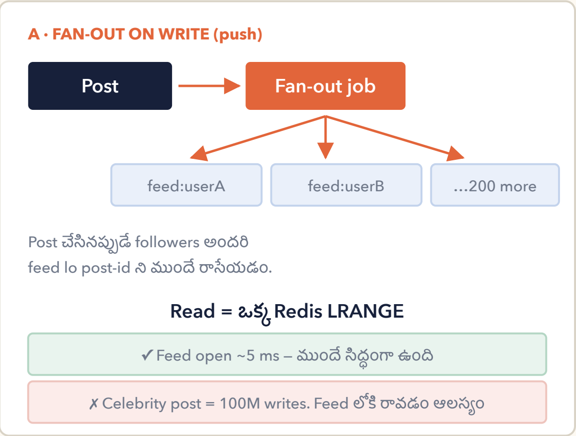
Step	లెక్క	ఫలితం
DAU	ఇచ్చినది	300 M
Posts / day	300M × 0.3 (30% మంది రాస్తారు)	90 M
Write QPS	90M ÷ 86,400	**≈ 1,000 / sec** (peak ≈ 5,000)
Feed opens / day	300M × 10	3 B
Read QPS	3B ÷ 86,400	**≈ 35,000 / sec** (peak ≈ 150,000)
Read : Write	35,000 : 1,000	**35 : 1 — read-heavy**
Fan-out writes / sec	1,000 posts × 200 followers	**200,000 feed inserts / sec**
Post size	text 300 B + metadata	≈ 1 KB
Post storage / year	90M × 365 × 1 KB	≈ 33 TB
Feed cache	300M users × 500 post-ids × 8 B	≈ 1.2 TB (active users కే ఉంచితే చాలా తక్కువ)

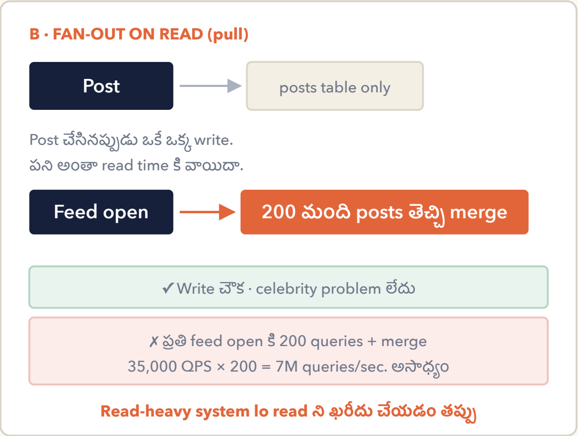
ఈ ఒక్క NUMBER ని బోర్డ్ మీద పెద్దగా రాయండి

1,000 posts/sec → 200,000 feed writes/sec. ఒక్క post 200 రెట్లు అమ్ప్లిఫై అవుతోంది. అదే fan-out. మరి 100 million followers ఉన్న celebrity post చేస్తే? **ఒక్క write → 100 million writes.** ఇక్కడే మొత్తం design విరిగిపోతుంది.

4. Deep Dive – Fan-out on Write vs Read

THE CENTRAL TRADE-OFF · పని ఎప్పుడు చేయాలి



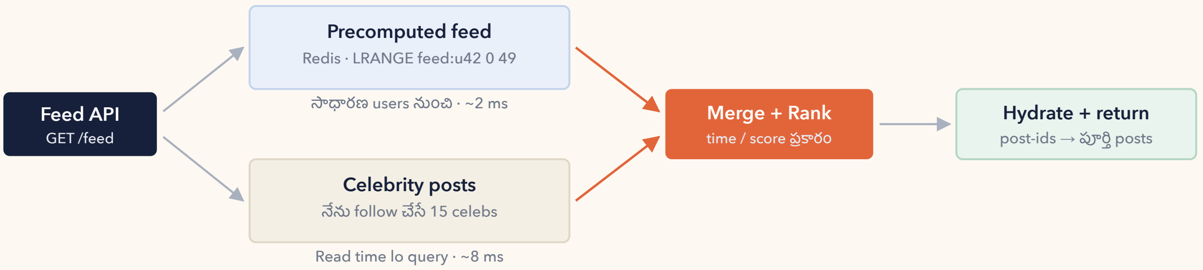


	Fan-out on write (push)	Fan-out on read (pull)
పని ఎప్పుడు	Post చేసినప్పుడు	Feed చూసినప్పుడు
Read latency	**~5 ms** (ముందే సిద్ధం)	~500 ms (200 queries + merge + sort)
Write ఖర్చు	followers సంఖ్యకి సమానం	O(1)
Storage	ఎక్కువ (ప్రతి user కీ copy)	తక్కువ
Celebrity	💀 విరిగిపోతుంది	✅ ఇబ్బంది లేదు
Inactive users	వృథా పని (వాళ్ళు ఎప్పటికీ చూడరు)	వృథా లేదు

నా CHOICE · HYBRID – ఇదే నిజమైన జవాబు

"రెండూ కాదు – **రెండూనూ**. సాధారణ users (99.9%) కి **push** చేస్తాను, ఎందుకంటే వాళ్ళకి 200 followers మాత్రమే, fan-out చౌక. Celebrities కి (ఒక threshold, ఉదా. 100K followers దాటినవాళ్ళు) **push చేయను** – వాళ్ళ posts ని read time lo merge చేస్తాను. అంటే feed open అయినప్పుడు: ముందే సిద్ధంగా ఉన్న cached feed + నేను follow చేసే celebrities యొక్క తాజా posts (అది ఒక 10-20 మంది మాత్రమే) – ఈ రెండింటినీ merge చేసి ఇస్తాను. దీంతో fan-out పేలుడూ లేదు, read explosion లేదు."

HYBRID FEED · ఒక FEED OPEN అయినప్పుడు జరిగేది

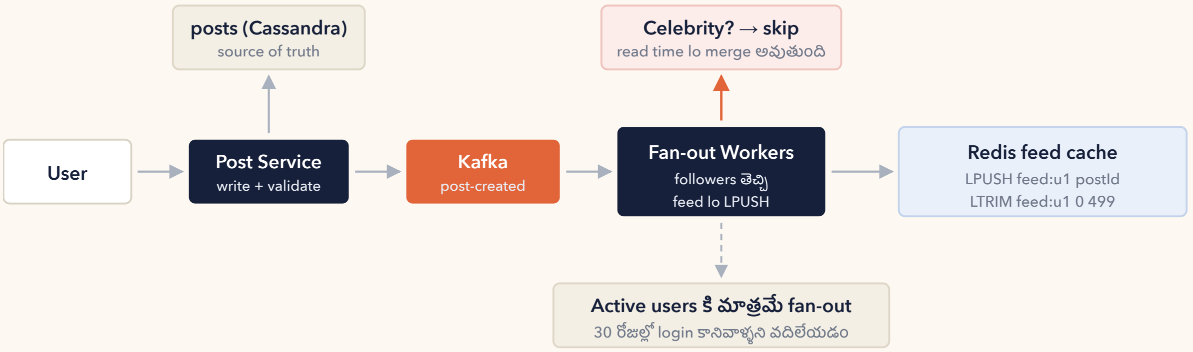


Feed cache lo post-ids మాత్రమే ఉంటాయి, పూర్తి posts కాదు. ఒక post 1 KB, ఒక id 8 bytes – 300M users × 500 posts lo ఈ తేడా 150 TB vs 1.2 TB. Hydration ని చివర్లో ఒక్క batch call తో చేస్తాం.

5. Data Model & Architecture

Store	ఏముంటుంది	ఎందుకు ఈ store
`posts` – Cassandra	post_id (Snowflake), user_id, text, media_url, created_at	Write-heavy, append-only, sharded by post_id
`follows` – MySQL/Graph	follower_id → followee_id (రెండు దిక్కులా index)	"నా followers ఎవరు" + "నేను ఎవరిని follow చేస్తున్నాను" రెండూ కావాలి
`feed:{userId}` – Redis List	ఇటీవలి 500 post-ids	O(1) LPUSH, LTRIM తో పరిమాణం అదుపు
`celebrity_set` – Redis Set	Followers > 100K ఉన్న user-ids	Fan-out ముందు ఒక్క lookup

NEWS FEED · WRITE PATH (FAN-OUT)



Post write మరియు fan-out ని **Kafka తో విడదీయడం** ముఖ్యం. లేకపోతే celebrity post చేసినప్పుడు అతని API call 100M writes పూర్తయ్యేదాకా వేలాడుతుంది. ఇప్పుడు post ఒక్క write తో success అవుతుంది, fan-out background lo జరుగుతుంది.

6. Scale, Bottlenecks & Failure

సమస్య	పరిష్కారం
Celebrity fan-out	Threshold దాటినవాళ్ళకి push చేయకపోవడం (పైన చెప్పిన hybrid)
Inactive users కి వృథా fan-out	30 రోజుల్లో active అయినవాళ్ళకే push. మిగతావాళ్ళు తిరిగి వచ్చినప్పుడు feed ని on-demand rebuild చేయడం. ఇది fan-out పనిని 60-70% తగ్గిస్తుంది
Feed cache పరిమాణం	`LTRIM 0 499` – ఎవరూ 500 posts కంటే వెనక్కి scroll చేయరు. అంతకంటే వెనక్కి వెళ్తే DB నుంచి
Redis node down	Feed ఒక cache మాత్రమే, source of truth కాదు. Cache miss అయితే posts నుంచి rebuild (నెమ్మది కానీ correct)
Fan-out worker వెనకబడటం	Kafka lag monitor చేయడం. Celebrity posts కి వేరే priority topic – సాధారణ posts వెనక వాటిని ఆపకుండా
Post delete	Feed cache నుంచి తీసేయడం ఖరీదు (100M lists). బదులుగా **read time lo filter** – hydration lo deleted posts ని వదిలేయడం. ఇది "tombstone" approach
Duplicate posts feed lo	Fan-out at-least-once కాబట్టి retry లో duplicate రావొచ్చు. Redis Sorted Set (score = post_id) వాడితే dedupe ఉచితం

RANKING కి JUMP ఎప్పుడు చేయాలి

Interviewer "chronological కాకుండా ranked feed ఎలా?" అని అడిగితే: "Feed cache lo ఇప్పటికే candidate post-ids ఉంటాయి. Read time lo ఒక ranking service ఆ 500 candidates ని తీసుకుని – recency, engagement rate, author affinity, media type లాంటి features తో score చేసి top 50 ఇస్తుంది. Architecture మారదు; **merge step కి ఒక scoring step చేరుతుంది** అంతే." ఈ జవాబు మీ design flexible అని నిరూపిస్తుంది.

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me confirm scope first: chronological or ranked, roughly how many people does a user follow, and – the one that matters most – what's the **maximum** follower count? If the answer is a hundred million, that single number changes the entire design."

"I'll assume 300 million daily users, 90 million posts a day, and each user opening the feed about ten times. That's roughly a thousand writes a second against thirty-five thousand reads a second, so this is heavily read-dominated – which tells me I should be willing to pay on the write path to make reads cheap."

"But here's the amplification: a thousand posts a second times two hundred followers is **two hundred thousand feed writes a second**. That's the number worth writing on the board."

"There are two ways to build this. Fan-out on write means when you post, I immediately push the post ID into the precomputed feed list of every follower. Reading is then a single Redis range query – about five milliseconds. Fan-out on read means I store the post once and, when you open the feed, I go fetch posts from all two hundred people you follow and merge them. Writing is cheap, but reading becomes two hundred queries – at thirty-five thousand reads a second that's seven million queries a second. In a read-heavy system, making reads expensive is the wrong direction."

"So push looks right – except for celebrities. Someone with a hundred million followers turns one write into a hundred million writes, and their post takes minutes to finish propagating."

"My answer is a **hybrid**. Normal users get pushed, because two hundred writes is nothing. Accounts above a threshold – say a hundred thousand followers – are excluded from fan-out entirely. When you open your feed I read your precomputed list, then separately fetch recent posts from the handful of celebrities you follow, and merge the two. You typically follow fifteen or twenty of those, so it's a small, bounded read."

"Two implementation details I'd call out. First, the feed cache stores **post IDs, not posts** – eight bytes instead of a kilobyte, which is the difference between a terabyte and a hundred and fifty terabytes. I hydrate the full posts in one batch call at the end. Second, fan-out runs off Kafka rather than inline, so the poster's request returns after one write instead of waiting for the fan-out to finish."

"For efficiency I'd only fan out to users active in the last thirty days. Most accounts are dormant and will never read that feed – skipping them cuts fan-out work by more than half. When a dormant user returns, I rebuild their feed on demand."

"On deletes: removing a post from millions of cached feeds is prohibitively expensive, so I'd filter at read time during hydration instead – a tombstone approach. Slightly wasteful, dramatically cheaper."

"If you'd like, the two areas I'd go deeper on are how ranking slots into this, and how I'd keep the fan-out workers from falling behind during a traffic spike."

7. Follow-ups

అడిగేది	జవాబు
"Celebrity threshold ఎంత?"	Fixed number కంటే **cost-based** మేలు: fan-out ఖర్చు (followers × write cost) vs read-merge ఖర్చు – ఏది తక్కువో అది. ఆచరణలో 100K తో మొదలుపెట్టి metrics చూసి tune చేయడం
"Pagination ఎలా?"	Offset వాడొద్దు – కొత్త posts వస్తే items repeat/skip అవుతాయి. **Cursor** వాడాలి: చివరి post_id. Snowflake ids time-sortable కాబట్టి ఇది సహజంగా పని చేస్తుంది
"కొత్తగా follow చేస్తే?"	ఆ user యొక్క ఇటీవలి posts ని feed lo backfill చేయడం (async). లేదా ఏమీ చేయకుండా వదిలేయడం – తర్వాతి posts నుంచే కనిపిస్తాయి
"Feed lo ads ఎలా?"	Merge step తర్వాత ఒక injection step – ప్రతి 5వ position lo ad slot. Feed pipeline lo ఇంకో stage మాత్రమే
"Twitter vs Instagram తేడా?"	Twitter చాలా ఎక్కువ chronological + higher post rate. Instagram ranking-heavy + media-heavy కాబట్టి CDN, image variants ఎక్కువ ముఖ్యం
"Fan-out worker crash అయితే posts పోతాయా?"	లేదు – Kafka offset commit ని fan-out పూర్తయ్యాకే చేయడం. At-least-once అవుతుంది, duplicates ని Sorted Set dedupe చూసుకుంటుంది

PROBLEM 04 · STATEFUL CONNECTIONS

04

Design a Chat System (WhatsApp / Messenger)

Difficulty **Hard** · Frequency **అత్యధికం** · నేర్పే concepts: WebSockets, presence, ordering, delivery receipts, offline queue

1. The Ask

> "Design WhatsApp. One-to-one messaging, group chats, online status, and the blue ticks."

ఇంతకుముందు problems అన్నీ **request-response** (client అడుగుతుంది, server జవాబిస్తుంది). ఇది వేరే జాతి – ఇక్కడ **server కి క్లయింట్‌ని pushచేయాలి**. అదే ఈ problem యొక్క కొత్తదనం.

కొత్త కష్టం 1
Server నుంచి client కి push. HTTP request-response తో ఇది కాదు – persistent connection కావాలి.

కొత్త కష్టం 2
Servers ఇప్పుడు **stateful**. "User 42 ఏ server తో connect అయ్యాడు?" అని తెలియాలి.

కొత్త కష్టం 3
Receiver offline ఉంటే? Message ని ఎక్కడ దాచాలి, ఎప్పుడు ఇవ్వాలి, ఎప్పుడు తీసేయాలి.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
1:1 మాత్రమేనా, groups కూడానా?	Group అంటే fan-out మళ్ళీ వస్తుంది	రెండూ
Group గరిష్ఠ పరిమాణం?	256 vs 100,000 – పూర్తిగా వేరే designs	256 (WhatsApp limit)
Message history ఎంతకాలం?	Server lo permanent నా, delivery తర్వాత delete నా	WhatsApp: delivery తర్వాత delete. మనం 30 రోజులు ఉంచుదాం
End-to-end encryption ఉందా?	ఉంటే server content చూడలేదు → search, moderation సాధ్యం కాదు	ఉంది (కానీ scope బయట)
Media (photos, video)?	వేరే upload path + CDN	అవును
Online status / last seen?	Presence system – దీనికి ప్రత్యేక fan-out సమస్య ఉంది	అవును
Multi-device?	ఒకే user కి బహుళ connections	అవును

3. Estimation

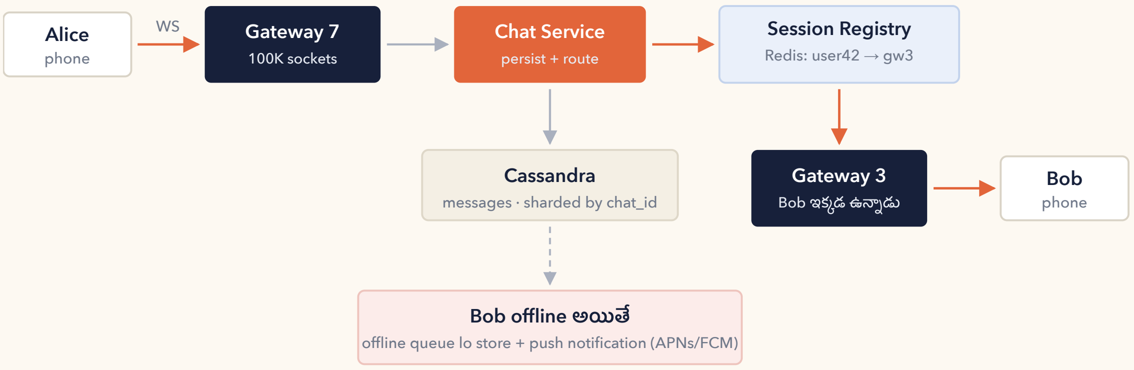
Step	లెక్క	ఫలితం
DAU	ఇచ్చినది	500 M
Messages / user / day	ఇచ్చినది	40
Messages / day	500M × 40	20 B
Avg QPS	20B ÷ 86,400	**≈ 230,000 / sec**
Peak QPS	avg × 3	≈ 700,000 / sec
Message size	text ~100 B + metadata ~100 B	≈ 200 B
Storage / day	20B × 200 B	**≈ 4 TB / day**
30 రోజుల storage	4 TB × 30	≈ 120 TB
Concurrent connections	500M DAU lo ~20% ఒకేసారి	**≈ 100 M open sockets**
ఒక server ఎన్ని sockets?	Tuned Linux box ≈ 100K	**≈ 1,000 gateway servers**

ఈ NUMBER ని బోర్డ్ మీద రాయండి

100 million concurrent WebSocket connections. ఇదే ఈ problem lo అసలు constraint – QPS కాదు. Stateless API servers అయితే load balancer ఏ server కైనా పంపొచ్చు. కానీ ఇక్కడ user 42 కి message పంపాలంటే, **అతను ఏ ఒక్క server తో connect అయ్యాడో సరిగ్గా ఆ server కి** వెళ్ళాలి. ఇది routing problem – ఇంతకుముందు problems lo ఇది లేదు.

4. Deep Dive – Connection ఎలా ఉంచాలి?

Option	ఎలా	Verdict
Polling	ప్రతి 2 సెకన్లకి "కొత్త message ఉందా?"	❌ 500M × ప్రతి 2s = అపారమైన వృథా traffic, latency కూడా పెద్దది
Long polling	Request ని server పట్టుకుని, message వచ్చినప్పుడు జవాబివ్వడం	⚠️ పని చేస్తుంది, కానీ ప్రతి message తర్వాత reconnect. Client → server పంపడానికి వేరే request
SSE	Server-sent events – ఒక దిక్కు stream	⚠️ Server→client మాత్రమే. Chat కి రెండు దిక్కులా కావాలి
WebSocket	ఒకసారి handshake, తర్వాత full-duplex	✅ **ఇదే సరైనది.** ఒకే connection lo రెండు దిక్కులా, తక్కువ overhead



SESSION REGISTRY లేకపోతే – Bob కి message పంపడానికి 1,000 gateways అన్నిటిని అడగాలి

Session Registry ఈ design యొక్క గుండె. "ఏ user ఏ gateway మీద ఉన్నాడు" అనే map. Gateway crash అయితే ఆ entries TTL తో తమంతట తామే పోవాలి – అందుకే heartbeat తో refresh అయ్యే short TTL (~30s) పెట్టాలి.

5. Deep Dive – Message Ordering (ఇది తప్పకుండా అడుగుతారు)

సమస్య: Alice రెండు messages వరుసగా పంపింది. అవి రెండు వేరే servers గుండా వెళ్ళి, Bob కి తప్పు వరుసలో చేరాయి.

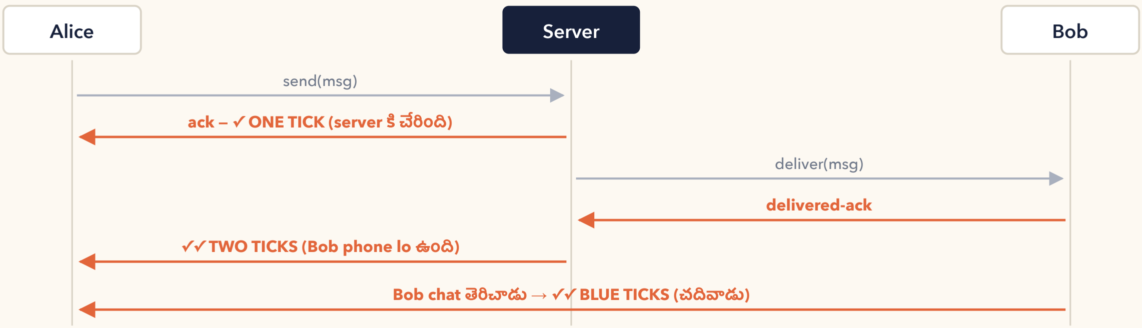
Approach	సమస్య
Client timestamp	Client clocks తప్పుగా ఉంటాయి. User తన phone time మార్చొచ్చు
Server timestamp	Servers మధ్య clock drift (NTP అయినా milliseconds తేడా). రెండు messages ఒకే ms lo వస్తే tie
Global sequence number	పని చేస్తుంది కానీ ఒక global counter అంటే bottleneck + SPOF
Per-conversation sequence ✅	ఒక్కో chat కి ఒక counter. `chat_id` shard lo atomic increment. Conversations మధ్య order అవసరం లేదు – **ఒక chat లోపల మాత్రమే order ముఖ్యం**

ఇక్కడ చెప్పాల్సిన INSIGHT

"Global ordering అవసరం లేదు – అది over-engineering. Alice-Bob chat lo messages order lo ఉండాలి, కానీ ఆ chat కీ Charlie-Dave chat కీ మధ్య order ఎవరూ చూడరు. అందుకే **per-conversation sequence number** చాలు. ఇది సహజంగా shard అవుతుంది – `chat_id` ఏ shard lo ఉందో అక్కడే counter, cross-shard coordination అవసరం లేదు." – ఈ "అవసరమైన దానికంటే బలహీనమైన guarantee ఎంచుకోవడం" అనేదే senior thinking.

6. Deep Dive – మూడు Ticks (delivery receipts)

SENT · DELIVERED · READ – మూడు వేర్వేరు ACKNOWLEDGEMENTS



మూడూ వేర్వేరు events, వేర్వేరు సమయాల్లో వస్తాయి. Blue tick కి **వేరే ack** కావాలి – message phone కి చేరడం వేరు, user చూడడం వేరు. చాలా మంది candidates ఈ తేడాని మిస్ అవుతారు.

7. Deep Dive – Presence (online status) యొక్క దాచిన బాంబు

సమస్య: Alice కి 500 contacts ఉన్నారు. ఆమె online అయితే 500 మందికీ చెప్పాలా? 100M users × ప్రతి connect/disconnect = presence fan-out పేలుతుంది.

Approach	ఖర్చు	Verdict
Online అయినప్పుడు contacts అందరికీ push	100M × 500 = 50 B events	❌ Message traffic కంటే presence traffic ఎక్కువవుతుంది
Pull on demand ✅	Chat list తెరిచినప్పుడు కనిపించే 20 మంది status ని మాత్రమే అడగడం	✅ 500 కాదు, 20. 25× తక్కువ
Heartbeat + TTL	Client ప్రతి 30s కి `SET presence:u42 online EX 45`. Key లేకపోతే offline	✅ Server crash అయినా status తనంతట తానే సరిచేసుకుంటుంది

ఇలా చెప్పండి

"Presence ని **eventually consistent, pull-based** గా చేస్తాను. Last-seen 30 సెకన్లు పాతది అయినా ఎవరూ గమనించరు – ఇది chat message కాదు. ఈ ఒక్క నిర్ణయం presence traffic ని 25 రెట్లు తగ్గిస్తుంది. Presence కోసం message system ని ఖరీదు చేయడం విలువ లేని పని."

8. Group Chat – News Feed మళ్ళీ ఇక్కడ

Group lo 256 మంది. Message వచ్చినప్పుడు 256 copies రాయాలా, లేక ఒక్కటి రాసి అందరూ చదవాలా?

	Fan-out on write	Fan-out on read
256 members	256 inserts – పెద్ద ఖర్చు కాదు	1 insert, 256 reads

	Fan-out on write	Fan-out on read
Verdict	✅ **256 చిన్న సంఖ్య** కాబట్టి write fan-out సరిపోతుంది. ఇక్కడ celebrity problem లేదు (group size capped)	Group size unlimited అయితే మాత్రమే

ఇక్కడ NEWS FEED తో CONNECT చేయండి

"News feed lo fan-out on write విరిగింది ఎందుకంటే followers unbounded (100M). ఇక్కడ group size **256 కి capped** కాబట్టి అదే approach బాగా పని చేస్తుంది. **Bound ఉందా లేదా అన్నదే** ఆ నిర్ణయాన్ని మారుస్తుంది – problem పేరు కాదు." ఇలా రెండు problems ని కలిపి మాట్లాడితే interviewer కి మీరు pattern ని అర్థం చేసుకున్నారని తెలుస్తుంది.

9. Scale & Failure

సమస్య	పరిష్కారం
Gateway crash	ఆ 100K users disconnect అవుతారు, client auto-reconnect చేసి వేరే gateway కి వెళ్తాడు. Session registry entries TTL తో పోతాయి. Reconnect అయ్యాక client `last_seq` పంపి missed messages తెచ్చుకుంటాడు
Message duplicate	Client-generated `message_id` (UUID). Server అదే id మళ్ళీ వస్తే ignore చేస్తుంది – idempotent write
Offline user	Message ని offline queue lo (Cassandra) store చేసి, push notification పంపడం. User తిరిగి వచ్చాక queue drain
Load balancer stickiness	WebSocket కాబట్టి connection ఒకే server కి అంటుకుని ఉండాలి. L4 LB + connection-aware routing
Thundering herd on restart	Deploy చేసినప్పుడు 100K clients ఒకేసారి reconnect. **Jittered exponential backoff** client side lo తప్పనిసరి
Hot chat (పెద్ద group)	`chat_id` shard వేడెక్కుతుంది. Group messages ని `(chat_id, time_bucket)` తో sub-shard చేయడం
Media	Message lo media వెళ్ళదు – client ముందు S3 కి upload చేసి, message lo URL మాత్రమే పంపుతుంది

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"A few things I want to pin down: one-to-one only or groups too, what's the maximum group size, how long do we retain history on the server, and is it end-to-end encrypted? I'll assume both chat types, groups capped at 256, thirty days of retention, and E2E encryption which I'll treat as out of scope for the architecture."

"Five hundred million daily users at forty messages each is twenty billion messages a day – about two hundred and thirty thousand a second. But the number that actually constrains this design isn't QPS, it's **a hundred million concurrent connections**. At roughly a hundred thousand sockets per tuned server, that's about a thousand gateway servers."

"That changes the shape of the system. Everything so far has been stateless – the load balancer could send a request anywhere. Here, to deliver a message to Bob I have to reach the **one specific gateway** holding Bob's socket. So I need a session registry: a Redis map from user ID to gateway ID, refreshed by heartbeat with a short TTL so that entries for a crashed gateway expire on their own."

"For transport I'd use WebSockets. Polling wastes enormous traffic at this scale, and server-sent events are one-directional – chat needs both ways over one connection."

"On ordering: I'd avoid a global sequence number, because that's a single bottleneck for something we don't actually need. Ordering only matters **within** a conversation – nobody can observe the relative order of two unrelated chats. So I use a per-conversation sequence number, which shards naturally by chat ID with no cross-shard coordination. Choosing the weakest guarantee that's still correct is usually the right move."

"The ticks are three separate acknowledgements, and it's worth being precise: one tick is the server acknowledging receipt, two ticks is the recipient's device acknowledging delivery, and blue ticks require a **third** ack sent when the user actually opens the chat. Delivered and read are genuinely different events."

"Presence has a hidden scaling trap. If I push status changes to all of a user's contacts, someone with five hundred contacts generates five hundred events every time they connect – presence traffic ends up exceeding message traffic. Instead I make presence **pull-based**: when you open your chat list, I fetch status for the twenty conversations actually on screen. Presence is eventually consistent by nature; a thirty-second-stale last-seen is invisible to users."

"Group chat is the fan-out question again. In the news feed, fan-out on write broke because follower count is unbounded. Here group size is capped at 256, so writing 256 copies is perfectly fine. The bound is what changes the answer, not the problem name."

"For failure: if a gateway dies, its clients reconnect elsewhere and send their last sequence number to pull what they missed. I'd insist on jittered exponential backoff on reconnect, otherwise a deploy causes a hundred thousand clients to stampede back simultaneously."

10. Follow-ups

అడిగేది	జవాబు
"Multi-device ఎలా?"	Session registry ని `user → [device1:gw3, device2:gw9]` గా చేయడం. Message అన్ని devices కీ fan-out. ప్రతి device కి తన సొంత `last_seq`
"E2E encryption ఉంటే server ఏం చేస్తుంది?"	Server ఒక **dumb router** అవుతుంది – ciphertext ని store చేసి forward చేయడం మాత్రమే. Key exchange (Signal protocol) clients మధ్య. Server-side search, moderation సాధ్యం కాదు
"Message search?"	E2E ఉంటే client-side index మాత్రమే. లేకపోతే Elasticsearch, chat_id ప్రకారం sharded
"Typing indicator?"	Presence లాంటిదే కానీ ఇంకా ephemeral – persist చేయవద్దు, నేరుగా gateway గుండా push, 5 సెకన్ల TTL
"Cassandra ఎందుకు, MySQL కాదు?"	Access pattern: `chat_id` + time range తో messages చదవడం. అది సరిగ్గా Cassandra యొక్క partition-key + clustering-key model. పైగా 4 TB/day write throughput
"Message delete for everyone?"	Tombstone record పంపడం. Clients దాన్ని చూసి local గా తీసేస్తాయి. Server copy ని కూడా delete. కానీ receiver already చూసి ఉంటే మనం ఏమీ చేయలేం – దాన్ని honest గా చెప్పండి

PROBLEM 05 · RELIABILITY ENGINEERING

05

Design a Notification System

Difficulty **Medium-Hard** · Frequency **ఎక్కువ** · నేర్పే concepts: idempotency, retry + backoff, DLQ, circuit breaker, priority queues

1. The Ask

> "Design a system that sends push notifications, SMS and email – a billion a day – reliably."

ఈ problem lo **architecture సులభం** (queue + workers). కష్టం అంతా **reliability** lo ఉంది. అందుకే ఇది ఒక reliability engineering interview – system design interview ముసుగులో.

ఇక్కడ మీరు గెలిచేది ఇక్కడ

Basic pipeline ని అందరూ గీస్తారు. మీరు వేరుగా కనిపించేది ఈ నాలుగు ప్రశ్నలకి జవాబు ఇచ్చినప్పుడు: (1) **Third party (APNs) down అయితే** ఏమవుతుంది? (2) Same notification **రెండుసార్లు** వెళ్ళకుండా ఎలా ఆపుతారు? (3) User కి రోజుకి 500 notifications వెళ్ళకుండా ఎలా ఆపుతారు? (4) OTP మరియు marketing email **ఒకే queue lo** ఉంటే ఏమవుతుంది?

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఏఏ channels?	ఒక్కో దానికి వేరే provider, వేరే latency, వేరే ఖరీదు	Push, SMS, Email, in-app
Delivery guarantee – at-least-once నా exactly-once?	Exactly-once distributed systems lo సాధ్యం కాదు; at-least-once + dedupe అనే నిజం చెప్పాలి	At-least-once + idempotency
Priority ఉందా?	OTP 5 సెకన్లలో వెళ్ళాలి, newsletter గంట ఆలస్యమైనా ఫర్వాలేదు	అవును, 3 tiers
User preferences / opt-out?	Legal requirement (GDPR, TRAI). ప్రతి send ముందు check	తప్పనిసరి
Scheduling ఉందా?	"రేపు ఉదయం 9కి పంపు" అంటే scheduler కూడా కావాలి	అవును
Templates ఎవరు manage చేస్తారు?	Content ని code నుంచి వేరు చేయాలి	Template service

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Analytics – delivered/opened/clicked?	Feedback loop, provider webhooks	అవును

3. Estimation

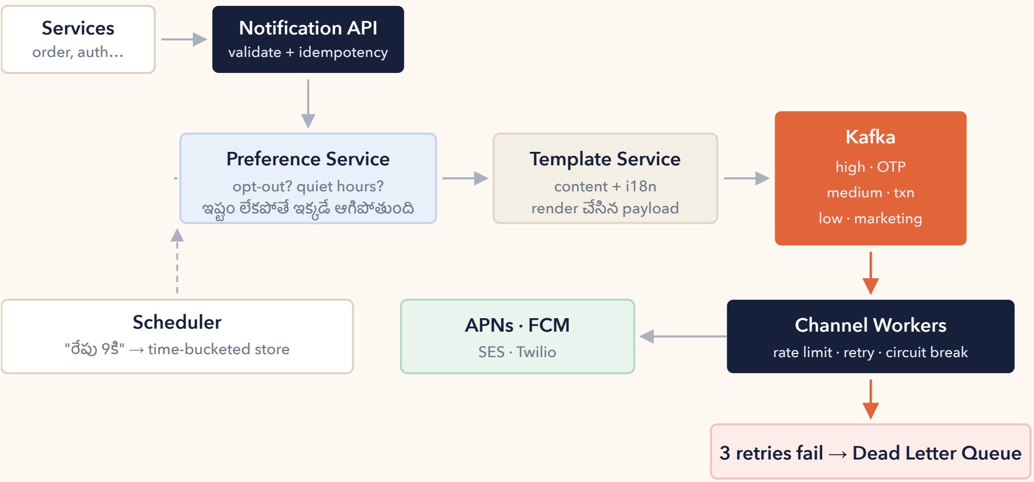
లెక్క	విలువ
Notifications / day	1 B
Avg QPS	1B ÷ 86,400 ≈ **12,000 / sec**
Peak (ఉదయం 9, పండగలు)	≈ 50,000 / sec
Channel split	Push 70% · Email 25% · SMS 5%
Payload size	≈ 1 KB
Kafka throughput	12,000 × 1 KB ≈ 12 MB/sec – ఇది చాలా తక్కువ
Dedupe key store	1B keys × 24h TTL × 50 B ≈ **50 GB Redis**

ఇక్కడ ఒక మంచి OBSERVATION

"12,000 QPS అనేది scale పరంగా చాలా తక్కువ – ఒక్క Kafka cluster తో సునాయాసంగా అవుతుంది. అంటే **ఈ problem lo కష్టం throughput కాదు**. కష్టం అంతా *correctness* lo – duplicates, retries, third-party outages, user fatigue. అందుకే నా design ఎక్కువ భాగం failure handling మీద ఉంటుంది." – ఇలా చెప్పి మీరే problem యొక్క నిజమైన స్వభావాన్ని పేరు పెట్టడం బలమైన opening.

4. Architecture

NOTIFICATION PIPELINE · PRODUCER నుంచి PROVIDER దాకా



Preference check ని **pipeline మొదట్లోనే** పెట్టడం ముఖ్యం – opt-out చేసిన user కోసం template render చేసి, queue lo పెట్టి, worker దాకా తీసుకెళ్ళి అప్పుడు వదిలేయడం వృథా. ముందే ఆపితే 100% ఆ పని ఆదా.

5. Deep Dive – Duplicate ఎలా ఆపాలి

నిజం ఏమిటంటే: Distributed system lo **exactly-once delivery సాధ్యం కాదు**. Worker notification పంపి, ack రాయకముందే crash అయితే – retry lo మళ్ళీ పంపుతుంది. ఇది గణితపరంగా తప్పించలేనిది.

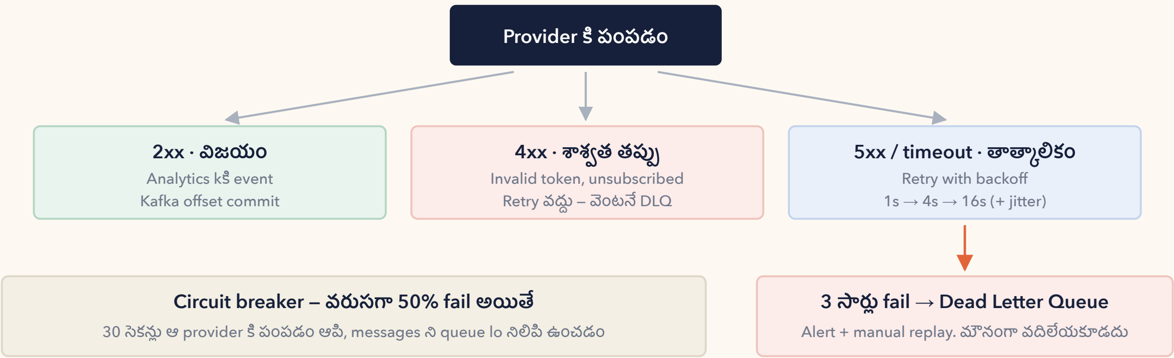
సరైన జవాబు

"Exactly-once **delivery** సాధ్యం కాదు. కానీ exactly-once **effect** సాధ్యం – at-least-once delivery + receiver side idempotency. Producer ప్రతి notification కీ ఒక `idempotency_key` ఇస్తుంది (ఉదా. `order-4471-shipped`). API దాన్ని Redis lo `SET key NX EX 86400` తో రాస్తుంది. అది false ఇస్తే ఇది duplicate – silently drop." ఇలా చెప్పడం "exactly-once చేస్తాను" అనే అమాయకమైన జవాబు కంటే చాలా బలం.

```
// API layer – ఇదే మొత్తం system యొక్క duplicate gate
async function accept(req) {
  const key = req.idempotencyKey ?? `${req.event}:${req.userId}:${req.entityId}`;
  const fresh = await redis.set(`notif:${key}`, '1', { NX: true, EX: 86400 });
  if (!fresh) return { status: 200, deduped: true };   // ఇప్పటికే పంపాం
  await kafka.produce(topicFor(req.priority), req);
  return { status: 202 };
}
```

6. Deep Dive – Retry, Backoff, DLQ

FAILURE HANDLING · ఏ ERROR ని ఎలా చూడాలి



నియమం	ఎందుకు
4xx ని retry చేయొద్దు	Invalid device token 100 సార్లు retry చేసినా pass కాదు. Retry చేయడం అంటే provider ని అనవసరంగా కొట్టడం
Jitter తప్పనిసరి	Jitter లేని exponential backoff అంటే – provider recover అయిన క్షణంలో అన్ని workers ఒకేసారి కొట్టి మళ్ళీ కూల్చేస్తాయి (thundering herd)
DLQ ని monitor చేయాలి	DLQ ఉండటం సరిపోదు. దాని మీద alert లేకపోతే messages మౌనంగా చస్తాయి – అది silent data loss
Circuit breaker	APNs down అయినప్పుడు ప్రతి message కీ 30 సెకన్లు timeout అంటే workers అన్నీ ఇరుక్కుపోతాయి. Breaker fast-fail చేసి queue ని కాపాడుతుంది

7. Deep Dive – Notification Fatigue

Technical గా ప్రతిదీ సరిగ్గా ఉన్నా, user కి రోజుకి 200 notifications వెళ్తే **అతను app ని uninstall చేస్తాడు.** అది system failure కంటే పెద్ద failure.

Control	ఎలా
Per-user rate limit	"గంటకి గరిష్ఠంగా 5, రోజుకి 20" – token bucket (Problem 02 అక్కడ నేర్చుకున్నదే ఇక్కడ వాడుతున్నాం)
Quiet hours	User timezone lo రాత్రి 10 - ఉదయం 8 మధ్య low priority ని ఆపి, ఉదయానికి వాయిదా
Digest / batching	10 likes = 10 notifications కాదు, "10 people liked your post" ఒకటే
Priority bypass	OTP, security alert – వీటికి fatigue limits వర్తించవు

8. Priority – ఒకే queue యొక్క ప్రమాదం

ఒకే QUEUE LO అన్నీ పెడితే ఏమవుతుంది

Marketing team 50 million newsletters ని queue lo కుమ్మరిస్తుంది. అదే క్షణంలో ఒక user login చేసి **OTP** అడుగుతాడు. అతని OTP ఆ 50 million వెనక వరుసలో నిలబడుతుంది – 40 నిమిషాల తర్వాత వస్తుంది. అప్పటికి అతను login attempt వదిలేశాడు.

పరిష్కారం: Priority ప్రకారం **వేరే topics + వేరే worker pools**. High priority workers ఎప్పుడూ ఖాళీగా ఉండాలి – వాటిని low priority పని చేయనివ్వకూడదు. ఇది resource వృథా అనిపిస్తుంది, కానీ అదే latency guarantee కి ఖరీదు.

9. Scale & Failure

సమస్య	పరిష్కారం
APNs పూర్తిగా down	Circuit breaker open → messages Kafka lo నిలిచి ఉంటాయి (అవి పోవు). Provider తిరిగి వచ్చాక drain. కానీ **stale notifications కి TTL** పెట్టాలి – 6 గంటల పాత "your order is arriving" పంపడం వల్ల లాభం లేదు
Provider fallback	SMS కి రెండు providers (Twilio + MSG91). Primary fail అయితే secondary. Push కి fallback లేదు – APNs ఒక్కటే iOS కి దారి
Kafka lag పెరగడం	Consumer workers ని autoscale. Lag metric మీద alert. Low priority ని shed చేయడం (marketing ని వదిలేసి transactional ని కాపాడటం)
Preference service down	**Fail-closed** – preferences తెలియకపోతే పంపొద్దు. Opt-out చేసిన వాడికి పంపడం legal problem. (Rate limiter lo fail-open, ఇక్కడ fail-closed – ఈ తేడాని బయటికి చెప్పండి)
Analytics feedback	Provider webhooks (delivered, bounced, opened) → Kafka → warehouse. Hard bounce వస్తే ఆ token/email ని invalidate చేయడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Before designing: which channels, is there a priority notion, do we honour user preferences and quiet hours, and what delivery guarantee do we promise? I'll assume push, SMS and email, three priority tiers, mandatory preference checks, and at-least-once delivery."

"A billion a day is about twelve thousand a second, peaking near fifty thousand. That's genuinely small – a single Kafka cluster handles it comfortably. So I want to say up front that **throughput isn't the hard part of this problem**. The hard parts are duplicates, retries, third-party outages, and not burning the user out. My design will spend most of its time there."

"The pipeline is: an API that validates and de-duplicates, then a preference check, then template rendering, then Kafka, then per-channel workers that talk to APNs, FCM, SES or Twilio. I put the preference check **early** deliberately – rendering a template and queuing a message for someone who opted out is pure waste."

"On duplicates, I want to be precise: exactly-once **delivery** is impossible. A worker can send successfully and crash before committing its offset, and the retry will send again. What is achievable is exactly-once **effect** – at-least-once delivery plus idempotency. Every notification carries an idempotency key like 'order-4471-shipped', and the API does a Redis SET NX with a day's TTL. If the key already exists, we drop it silently."

"For retries I'd classify errors rather than blindly retrying. A 4xx – invalid device token, unsubscribed – is permanent; retrying it just hammers the provider, so it goes straight to the dead letter queue. A 5xx or timeout is transient, so exponential backoff with **jitter**. The jitter matters: without it, every worker retries at the same instant the provider recovers and knocks it over again."

"I'd wrap each provider in a circuit breaker. If APNs starts failing, every message costs a thirty-second timeout and all my workers block on dead connections. The breaker fast-fails instead and lets messages accumulate in Kafka, which is exactly where I want them – durable and replayable."

"Two product-level things that are really engineering decisions. First, priority needs **separate topics and separate worker pools**. If marketing dumps fifty million newsletters into a shared queue, a user's login OTP queues behind them and arrives forty minutes late. Dedicated high-priority workers sitting mostly idle look wasteful, but that idleness **is** the latency guarantee."

"Second, notification fatigue. Per-user rate limits, quiet hours in the user's own timezone, and digesting – ten likes become one notification, not ten. A technically perfect system that gets the app uninstalled has still failed."

"One asymmetry worth naming: I'd fail **open** on a rate limiter, but **closed** on the preference service. If I can't tell whether a user opted out, I don't send – that's a legal exposure, not a latency one."

10. Follow-ups

అడిగేది	జవాబు
"Scheduled notifications ఎలా?"	Time-bucketed store – `schedule:{yyyymmddhhmm}` sorted set. ఒక poller ప్రతి నిమిషం ఆ bucket ని తీసి queue lo పెడుతుంది. (Problem 10 lo దీన్ని లోతుగా చూస్తాం)
"50 million users కి ఒకేసారి పంపాలంటే (broadcast)?"	ఒక్కో notification API call కాదు – ఒక **campaign job**. User list ని chunks గా విడగొట్టి workers కి పంచడం, progress ని track చేయడం, resumable గా ఉంచడం
"Notification cost ఎలా తగ్గించాలి?"	SMS అన్నిటికంటే ఖరీదు (₹0.15/msg). Push ఉచితం. కాబట్టి push ముందు try చేసి, device unreachable అయితేనే SMS – **channel fallback ladder**
"Deduplication window ఎంత?"	24 గంటలు సాధారణం. కానీ business event ని బట్టి – "order shipped" జీవితకాలంలో ఒకసారే, "cart reminder" వారానికోసారి కావొచ్చు
"In-app notifications?"	ఇవి push కాదు – user కోసం ఒక inbox. DB lo store చేసి, unread count ని Redis lo. Chat problem lo నేర్చుకున్న WebSocket ఇక్కడ real-time badge కి పనికొస్తుంది
"GDPR / data deletion?"	User delete అడిగితే notification history, device tokens, analytics events అన్నీ తీసేయాలి. అందుకే ప్రతిచోటా `user_id` ని foreign key గా ఉంచి deletion job రాయడం సులభం చేయాలి

06

Design Video Streaming (YouTube / Netflix)

Difficulty **Hard** · Frequency **ఎక్కువ** · నేర్పే concepts: transcoding DAG, HLS/DASH, adaptive bitrate, CDN economics

1. The Ask

> "Design YouTube. Users upload videos, and other users watch them on any device, on any network."

ఈ problem lo **రెండు పూర్తిగా వేరే systems** ఉన్నాయి. వాటిని విడదీసి చెప్పడమే మీ మొదటి పని:

Write path · Upload

తక్కువ traffic (సెకనుకి కొన్ని వందలు), కానీ ఒక్కో పని **భారీ** – ఒక video ని transcode చేయడానికి నిమిషాల CPU. ఇది batch processing problem.

Read path · Playback

అపారమైన traffic, కానీ ఒక్కో పని **చౌక** – ఒక file ని పంపడం. ఇది bandwidth + CDN problem.

Interviewer చూసేది

ఈ రెండింటినీ మీరు విడిగా చూస్తున్నారా. ఒకే architecture తో రెండింటినీ కప్పేయాలని ప్రయత్నిస్తే అది తప్పు దారి.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
VOD నా live streaming నా?	పూర్తిగా వేరే designs – live lo transcode real-time గా జరగాలి	VOD (live ని follow-up lo)
ఏఏ resolutions?	Transcoding ఖర్చు rendition సంఖ్యకి proportional	240p – 4K, 6 renditions
Video పొడవు గరిష్ఠం?	10 నిమిషాలా, 10 గంటలా – chunking strategy మారుతుంది	సగటు 10 నిమిషాలు
DRM కావాలా?	Netflix కి తప్పనిసరి, YouTube కి కాదు	ఇప్పుడు వద్దు
Recommendations scope lo ఉన్నాయా?	ఉంటే అది ఒక పూర్తి ML system	బయట
Comments, likes?	సాధారణ CRUD – video problem కి సంబంధం లేదు	బయట

3. Estimation – ఇక్కడ bandwidth నే రాజు

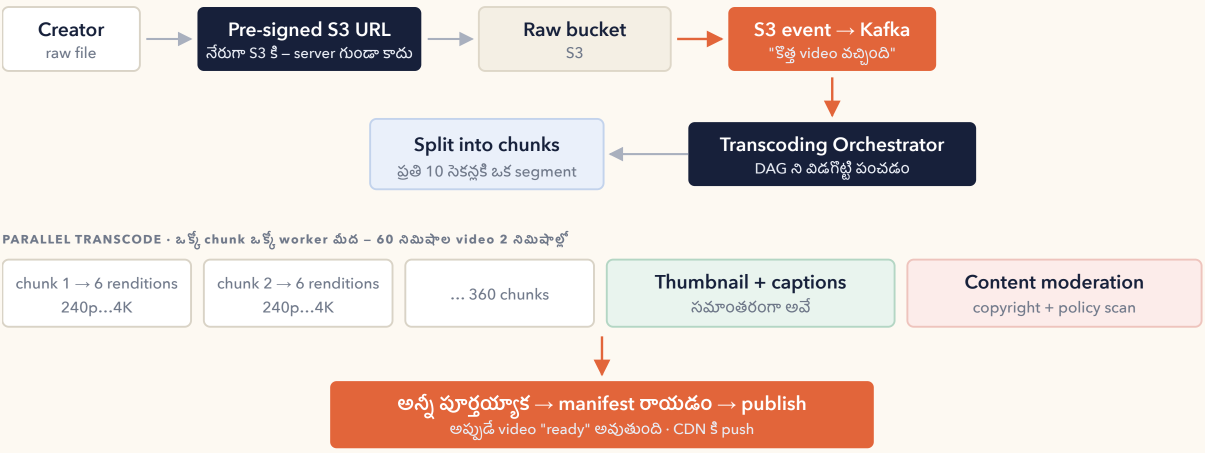
లెక్క	విలువ
DAU	500 M
Videos watched / user / day	4 (సగటు 10 నిమిషాలు)
Views / day	2 B
Uploads / day	720,000 గంటల video
Raw upload storage / day	720K గం × 2 GB ≈ **1.4 PB / day**
Transcoded (6 renditions)	raw × ~1.5 ≈ **2 PB / day**
5 ఏళ్ళ storage	≈ 3.6 EB (exabytes) – tiering తప్పనిసరి
Avg bitrate (mix of resolutions)	≈ 3 Mbps
Egress bandwidth	2B × 600s × 3 Mbps ÷ 8 ≈ **450 PB / day ≈ 40 Tbps**

ఈ ఒక్క వాక్యం మీ LEVEL ని చూపిస్తుంది

"40 Tbps egress అంటే – **ఈ business lo అతి పెద్ద ఖర్చు compute కాదు, storage కాదు, bandwidth.** అందుకే నా design lo అతి ముఖ్యమైన నిర్ణయం *CDN cache hit ratio*. 90% నుంచి 95% కి పెంచితే origin traffic సగం అవుతుంది – అది కోట్ల రూపాయల ఆదా. ఈ ఒక్క metric మిగతా అన్నిటికంటే ఎక్కువ ముఖ్యం."

4. Deep Dive – Upload & Transcoding Pipeline

UPLOAD → TRANSCODE → PUBLISH



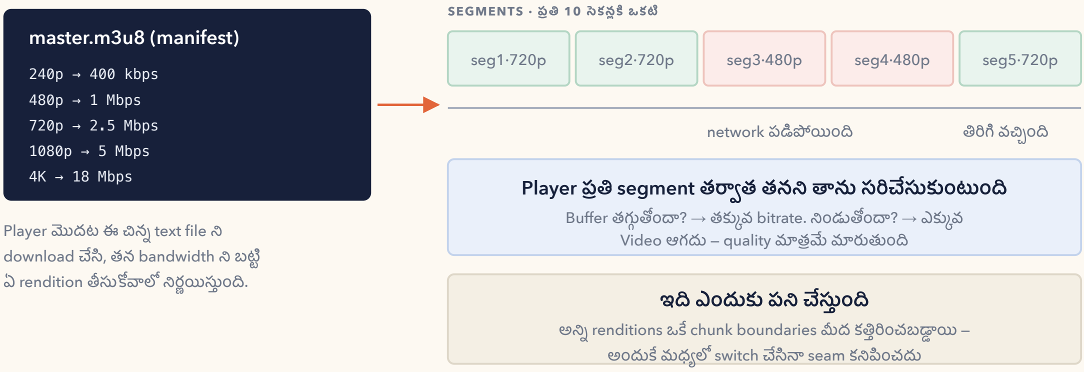
రెండు కీలక ఆలోచనలు: (1) **Client నేరుగా S3 కి upload చేస్తాడు** – pre-signed URL తో. 2 GB file ని మన API servers గుండా పంపడం అనవసరమైన ఖర్చు. (2) **Video ని chunks గా విడగొట్టి parallel transcode** – లేకపోతే 60 నిమిషాల video కి 60 నిమిషాలు పడుతుంది.

Pipeline నిర్ణయం	ఎందుకు
Chunk-level parallelism	360 chunks × 6 renditions = 2,160 స్వతంత్ర tasks. Embarrassingly parallel
Spot / preemptible instances	Transcoding batch పని, ఆలస్యం ఫర్వాలేదు → 70% చౌక. Task fail అయితే మళ్ళీ run
DAG orchestration	Thumbnail, captions, moderation – వీటికి పరస్పర ఆధారం లేదు, సమాంతరంగా. Publish మాత్రం అన్నీ పూర్తయ్యాకే
Idempotent tasks	Worker crash అయితే అదే chunk మళ్ళీ transcode. Output deterministic కాబట్టి సమస్య లేదు
Progressive publish	240p ముందు ready అయితే వెంటనే publish – 4K తర్వాత చేరుతుంది. Creator ఎదురుచూపు తగ్గుతుంది

5. Deep Dive – Adaptive Bitrate (HLS / DASH)

సమస్య: ఒకే video ని 4G lo, WiFi lo, lift lo – అన్నిచోట్లా ఆగకుండా చూపించాలి.

ADAPTIVE BITRATE · MANIFEST + SEGMENTS



ఇక్కడ చెప్పాల్సిన కీలక వాక్యం

"Adaptive bitrate యొక్క మొత్తం తెలివి **client lo** ఉంది, server lo కాదు. Server కేవలం static files ని పంచుతుంది – అందుకే దీన్ని CDN తో 100% cache చేయొచ్చు. ఇది intentional design: **intelligence ని edge కి నెట్టి, origin ని dumb గా ఉంచడం**. Server-side adaptive streaming చేస్తే CDN caching సాధ్యం కాదు, అప్పుడు 40 Tbps మొత్తం మన origin మీద పడుతుంది."

6. Deep Dive – CDN Strategy (అసలు ఖర్చు ఇక్కడే)

Video రకం	వాటా	Strategy
Viral / trending	1% videos, 80% views	CDN edge lo **ముందే push** (pre-warm). Cache hit ~99%

Video రకం	వాటా	Strategy
Popular	10% videos, 15% views	Pull-through cache – మొదటి request origin కి, తర్వాత edge lo
Long tail	89% videos, 5% views	Cache చేయకపోవడమే మేలు. Origin నుంచే. Cold storage (Glacier) lo raw ని ఉంచడం

REGIONAL PRE-WARMING

"ఒక కొత్త video upload అయినప్పుడు దాన్ని ప్రపంచంలోని అన్ని edges కి push చేయను – అది వృథా. Creator యొక్క audience ఎక్కడ ఉందో (గత videos నుంచి తెలుసు) ఆ regions కి మాత్రమే pre-warm చేస్తాను. మిగతాచోట్ల మొదటి viewer origin నుంచి తెచ్చుకుంటాడు, ఆ తర్వాత అది cache అవుతుంది."

7. Scale & Failure

సమస్య	పరిష్కారం
Storage ఖర్చు (3.6 EB)	Tiering: 30 రోజులు hot (S3 Standard), తర్వాత Infrequent Access, ఏడాది తర్వాత Glacier. 4K rendition ని unpopular videos కి delete చేసి, అవసరమైతే మళ్ళీ transcode
Transcoding backlog	Queue depth మీద autoscale. Priority: verified creators ముందు. Spot instance reclaim అయితే task ని మళ్ళీ schedule
Upload విఫలం (పెద్ద file)	**Resumable upload** – file ని 5 MB parts గా, S3 multipart. Network పోతే మిగిలిన parts నుంచి కొనసాగడం
CDN origin overload	Cache miss storm వస్తే origin చస్తుంది. **Request coalescing** – ఒకే segment కి 1000 concurrent misses వస్తే origin కి ఒకే fetch
Region outage	Videos multi-region replicated (కనీసం popular వాటిని). Manifest lo multiple CDN URLs
Thumbnail hot path	Thumbnails చాలా చిన్నవి కానీ చాలా ఎక్కువ requests – వాటిని వేరే CDN path, aggressive caching, WebP

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"First, is this video-on-demand or live? They're genuinely different systems – live means transcoding has to keep up with real time, which changes everything. I'll design VOD and treat live as an extension. I'll also assume six renditions from 240p to 4K, and put recommendations and comments out of scope."

"I want to split this into two systems immediately, because they have opposite shapes. The **upload path** is low traffic but each unit of work is enormous – minutes of CPU per video. The **playback path** is astronomical traffic where each unit of work is trivial – serve a file. Trying to design both with one architecture is where people go wrong."

"On numbers: two billion views a day at roughly three megabits average works out to about **forty terabits per second of egress**. That tells me the dominant cost in this business isn't compute or storage – it's bandwidth. So the single most important metric in my design is CDN cache hit ratio. Going from ninety to ninety-five percent halves origin traffic."

"For upload, the client gets a pre-signed URL and uploads **directly to object storage** – a two-gigabyte file should never pass through my API servers. Uploads are multipart and resumable, because a phone on mobile data will drop mid-upload."

"Object storage fires an event, and an orchestrator splits the video into ten-second chunks. Each chunk transcodes into six renditions independently – a sixty-minute video becomes about two thousand parallel tasks, so wall-clock time drops from an hour to a couple of minutes. Because these tasks are idempotent and latency-tolerant, I'd run them on spot instances at roughly a third of the cost and just re-run anything that gets preempted."

"Thumbnails, captions and content moderation run as parallel branches of the same DAG. The video only publishes when everything completes – though I'd consider progressive publish, releasing 240p as soon as it's ready so the creator isn't waiting on 4K."

"For playback I'd use HLS or DASH. The player downloads a small manifest listing the available bitrates, then requests ten-second segments, adjusting quality between segments based on how its buffer is doing. The important architectural consequence is that **all the intelligence lives in the client**. The server only serves static files, which is exactly what makes them a hundred percent CDN-cacheable. If I did the adaptation server-side, nothing would be cacheable and all forty terabits would hit my origin."

"On CDN strategy, view distribution is extremely skewed – roughly one percent of videos get eighty percent of views. I'd pre-warm trending content to edges, let popular content populate through normal pull-through caching, and deliberately **not** cache the long tail, since caching something watched twice a month is pure cost. And I'd pre-warm regionally based on where the creator's audience actually is, rather than globally."

"The failure case I'd protect hardest is a cache-miss storm on a newly viral video – a thousand simultaneous misses for the same segment. Request coalescing at the edge turns that into a single origin fetch."

8. Follow-ups

అడిగేది	జవాబు
"Live streaming ఎలా మారుతుంది?"	Transcode real-time గా జరగాలి (chunk-level parallelism సాధ్యం కాదు – future chunks ఇంకా లేవు). Latency budget 3-10 సెకన్లు. Protocol: RTMP ingest → HLS/LL-HLS out. Rewind కోసం DVR window
"Video ని ఎలా search చేస్తారు?"	Metadata (title, description, tags) → Elasticsearch. Content search కావాలంటే captions ని index చేయడం (అవి transcoding pipeline lo generate అవుతున్నాయి)
"View count ఎలా లెక్కిస్తారు?"	Real-time exact అవసరం లేదు. Client events → Kafka → stream aggregation → approximate count కి Redis, exact కి batch job. (Problem 17 lo దీన్ని లోతుగా చూస్తాం)
"DRM ఎలా?"	Segments ని encrypt చేసి, key ని license server నుంచి. Widevine/FairPlay. CDN encrypted bytes ని cache చేస్తుంది – caching చెడిపోదు
"Netflix vs YouTube తేడా?"	Netflix catalogue చిన్నది, స్థిరమైనది → అన్నీ ముందే transcode + edges lo ముందే push (Open Connect appliances). YouTube catalogue అనంతం → long tail ని cache చేయలేం. ఈ ఒక్క తేడా రెండు వేరే architectures కి దారితీస్తుంది
"Copyright detection?"	Audio/video fingerprint (Content ID) – upload సమయంలో known works database తో match. ఇది transcoding DAG lo ఒక parallel branch

PROBLEM 07 · GEOSPATIAL SYSTEMS

07

Design Ride Sharing (Uber / Ola)

Difficulty **Hard** · Frequency **ఎక్కువ** · నేర్చే concepts: geohash, quadtree, write amplification, matching, trip state machine

1. The Ask

> "Design Uber. A rider opens the app, sees nearby cars, requests a ride, and gets matched to a driver. Both sides see each other move in real time."

ఇది మొదటి **geospatial** problem. ఇక్కడ కొత్త ప్రశ్న ఒక్కటే కానీ చాలా లోతైనది: **"నా చుట్టూ 2 కిలోమీటర్లలో ఎవరున్నారు?"** అనే query ని సెకనుకి లక్షల సార్లు ఎలా జవాబివ్వాలి.

ఈ PROBLEM యొక్క దాచిన కష్టం

అందరూ matching గురించి మాట్లాడతారు. కానీ అసలు scale problem **location updates** lo ఉంది. 1 million drivers, ప్రతి 4 సెకన్లకి ఒక location – అంటే **సెకనుకి 250,000 writes**. Rides మాత్రం సెకనుకి 230. అంటే location traffic, ride traffic కంటే **వెయ్యి రెట్లు** ఎక్కువ. దీన్ని మీరే గుర్తించి చెప్తే interviewer ఆగిపోతాడు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఒక city నా ప్రపంచమా?	Sharding strategy ఇక్కడి నుంచే – cities స్వతంత్రం	ప్రపంచం, కానీ trips city లోపలే
Driver location ఎంత తరచుగా?	ఇదే మొత్తం write load ని నిర్ణయిస్తుంది	ప్రతి 4 సెకన్లకి
Matching దేని ఆధారంగా?	దూరమా, ETA నా, rating నా – ranking logic	ETA ప్రధానం
Pooling (షేర్డ్ rides) ఉందా?	ఉంటే matching అనేది ఒక optimisation problem అవుతుంది	ఇప్పుడు వద్దు
Surge pricing?	వేరే service, real-time demand/supply	ప్రస్తావిస్తాం, deep dive వద్దు
Payment scope lo ఉందా?	వేరే problem (09 చూడండి)	బయట

3. Estimation

లెక్క	విలువ
మొత్తం drivers	10 M
ఏ క్షణంలోనైనా online	1 M
Location update frequency	ప్రతి 4 సెకన్లకి
Location write QPS	1M ÷ 4 = **250,000 / sec**
Rides / day	20 M
Ride request QPS	20M ÷ 86,400 ≈ **230 / sec** (peak ≈ 1,000)
Location record size	driverId + lat + lng + ts ≈ 40 B
Location writes / day	21.6 B × 40 B ≈ 900 GB/day (ఇది store చేస్తే)

ఇక్కడ మొదటి పెద్ద నిర్ణయం

"250,000 writes/sec ని **ఏ disk-based database కీ పంపను**. Driver location అనేది **ephemeral state** – 4 సెకన్ల తర్వాత అది చెత్త. దాన్ని durable గా రాయడం అనవసరం. కాబట్టి current location అంతా **Redis lo మాత్రమే** ఉంటుంది. Trip జరుగుతున్నప్పటి route ని మాత్రం (billing, disputes కోసం) async గా Cassandra lo archive చేస్తాను. ఈ ఒక్క నిర్ణయం 250K QPS ని 1K QPS కి తగ్గిస్తుంది."

4. Deep Dive – "నా చుట్టూ ఎవరున్నారు?" (Geospatial Index)

GEOSPATIAL INDEXING · నాలుగు మార్గాలు

A · SQL లో lat/lng
WHERE lat BETWEEN ...
AND lng BETWEEN ...
B-tree ఒక column మీదే పని చేస్తుంది. రెండో దాన్ని scan చేయాలి. ❌ నెమ్మది

B · GEOHASH
2D → 1D string
`tdr1y8 (6 chars ≈ 1 km)`
Prefix ఒకటే అంటే దగ్గరగా ఉన్నట్టు.
✅ సులభం, Redis native

C · QUADTREE
Dense ప్రాంతాన్ని మాత్రమే చిన్న grids గా విడగొట్టడం
✅ Uniform load
❌ In-memory tree ని maintain చేయాలి

D · H3 / S2 ✓
Hexagon grid (Uber H3)
పొరుగు cells అన్నీ సమాన దూరంలో – geohash lo ఇది తప్పు
✅ Production choice

GEOHASH యొక్క BOUNDARY సమస్య · దీన్ని తప్పకుండా ప్రస్తావించండి

50 మీటర్ల దూరంలో ఉన్నారు
కానీ వేరే cells lo. ఒక్క cell వెతికితే driver కనిపించడు!

పరిష్కారం
ఎప్పుడూ నా cell + 8 పొరుగు cells – మొత్తం 9 cells వెతకడం

Redis తో ఎలా:

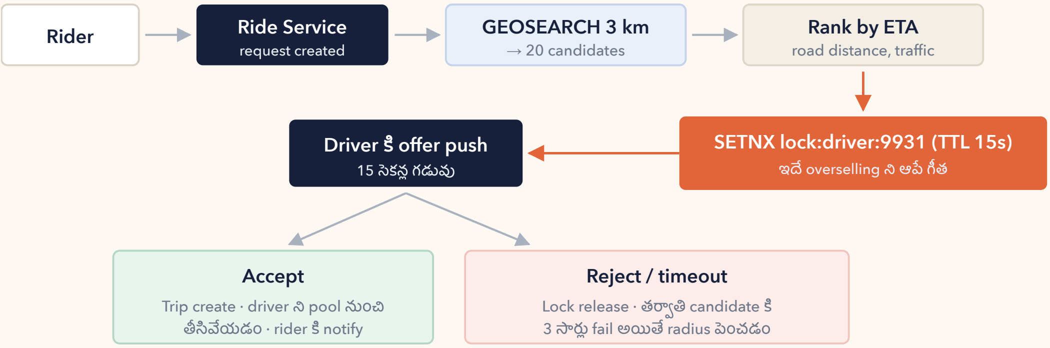
```
# Driver location update — ప్రతి 4 సెకన్లకి, ప్రతి driver
GEOADD drivers:city:hyd 78.4867 17.3850 "driver:9931"
```

```
# Rider "నా చుట్టూ ఎవరున్నారు?" — ఒక్క command
GEOSEARCH drivers:city:hyd FROMLONLAT 78.4772 17.4123 BYRADIUS 3 km ASC COUNT 20
```

> **REDIS GEO లోపల ఏముంది**
>
> Redis GEO commands అనేవి నిజానికి **Sorted Set + geohash** మాత్రమే. Lat/lng ని 52-bit geohash integer గా మార్చి score గా వాడుతుంది. అంటే `ZRANGEBYSCORE` తో range query. ఈ లోపలి విషయం చెప్తే – మీరు tool ని వాడటమే కాదు, అది ఎలా పని చేస్తుందో తెలుసని అర్థమవుతుంది.

5. Deep Dive – Matching (ఇక్కడ correctness చాలా ముఖ్యం)

RIDE REQUEST → MATCHED DRIVER



Lock TTL **15 సెకన్లు** – driver offer ని accept చేయడానికి ఇచ్చిన సమయమే. Driver app crash అయినా lock తనంతట తానే విడిపోతుంది. TTL లేని lock అంటే – ఆ driver శాశ్వతంగా ఎవరికీ దొరకడు.

Matching నిర్ణయం	ఎందుకు
సరళ దూరం కాదు, ETA	నదికి అవతల 500 మీటర్ల దూరంలో ఉన్న driver కంటే, రోడ్డు మీద 2 కిమీ దూరంలో ఉన్నవాడు ముందు వస్తాడు
ఒక్కొక్కరికే offer (sequential)	అందరికీ ఒకేసారి పంపితే ఇద్దరు accept చేస్తారు – అప్పుడు ఒకరిని తిరస్కరించాలి, అది చెడ్డ అనుభవం
Distributed lock తప్పనిసరి	ఇద్దరు riders ఒకే driver కి ఒకేసారి request చేయొచ్చు. `SETNX` + TTL ఇక్కడ సరిపోతుంది
Radius ని దశలవారీగా పెంచడం	3 కిమీ lo ఎవరూ లేకపోతే 5, తర్వాత 10. మొదటే 10 కిమీ వెతికితే అనవసరంగా దూరపు drivers వస్తారు

6. Trip State Machine

```
REQUESTED → MATCHED → DRIVER_ARRIVING → IN_PROGRESS → COMPLETED
     ↓          ↓            ↓
 NO_DRIVER  CANCELLED    CANCELLED (fee వర్తించొచ్చు)
```

STATE MACHINE ని ఎందుకు EXPLICIT గా చెప్పాలి

Trip అనేది **ఒక long-running transaction** – ఇది నిమిషాలు, కొన్నిసార్లు గంటలు నడుస్తుంది. దాన్ని DB transaction తో చేయలేం. అందుకే state machine + **ప్రతి transition ని idempotent** గా ఉంచడం. Driver "arrived" button రెండుసార్లు నొక్కితే రెండో సారి ఏమీ జరగకూడదు. ఈ ఒక్క మాట చెప్తే – distributed systems lo long-lived workflows గురించి మీకు తెలుసని అర్థం.

7. Scale & Failure

సమస్య	పరిష్కారం
250K location writes/sec	Redis lo మాత్రమే, disk DB కి కాదు. `city_id` ప్రకారం Redis cluster ని shard చేయడం – cities స్వతంత్రం కాబట్టి ఇది సహజమైన shard key
City-level hot shard	ముంబై shard, గోవా shard కంటే 100 రెట్లు busy. City ని geohash prefix తో sub-shard చేయడం
Driver app offline	Location updates ఆగిపోతాయి → Redis GEO entry కి TTL పెట్టడం (30 సెకన్లు). Stale drivers తమంతట తామే pool నుంచి పోతారు
Redis cluster down	Matching ఆగిపోతుంది. Multi-AZ replicas + AOF. కానీ location data ephemeral కాబట్టి పోయినా 4 సెకన్లలో తిరిగి నిండుతుంది – ఇది recovery చాలా సులభం చేస్తుంది
Live tracking (rider driver ని చూడటం)	WebSocket (Problem 04). Driver location → gateway → ఆ trip ని చూస్తున్న rider కి push. ప్రతి location ని అందరికీ broadcast చేయకూడదు – ఆ trip lo ఉన్న ఇద్దరికే
Surge pricing	Geohash cell స్థాయిలో demand/supply ratio ని 1 నిమిషం window lo లెక్కించి multiplier. ఇది వేరే stream job
Payment / fare	Trip route ని archive చేసి, ముగిసిన తర్వాత fare లెక్కించడం. Live గా కాదు – trip మధ్యలో network పోవచ్చు

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me scope this: are trips always within one city, how often do driver apps report location, and is matching optimised for distance or ETA? I'll assume city-scoped trips, location every four seconds, and ETA-based matching. Pooling and payments I'll treat as separate problems."

"Here's the number that shapes everything. With a million drivers online reporting every four seconds, that's **two hundred and fifty thousand location writes per second**. Ride requests are only about two hundred and thirty a second. So location traffic outweighs ride traffic by a factor of a thousand – the interesting engineering is on the location path, not the booking path."

"My first decision follows from that: driver locations never touch a disk-backed database. A location is **ephemeral** – four seconds later it's garbage. It lives in Redis only. I do archive the route asynchronously to Cassandra during an active trip, for billing and disputes, but that's a thousand writes a second, not a quarter million."

"The core query is 'who is near me'. Doing that in SQL with latitude and longitude ranges is slow, because a B-tree only indexes one dimension well – you end up scanning the other. The standard answer is to project two dimensions onto one: geohashing turns a coordinate into a string where a shared prefix means physical proximity. Uber's H3 uses hexagons instead of rectangles, which is better because every neighbouring cell is equidistant – with square geohash cells, diagonal neighbours are further away than edge neighbours."

"One trap I'd flag explicitly: geohash cells have a **boundary problem**. A rider and driver fifty metres apart can land in different cells, so searching only your own cell misses them. You always search your cell plus its eight neighbours."

"For matching, I search a three-kilometre radius, take about twenty candidates, and rank them by **ETA rather than straight-line distance** – a driver five hundred metres away across a river is further in practice than one two kilometres down the road."

"Then correctness matters. Two riders can request the same driver simultaneously. I take a distributed lock on the driver – a Redis SET NX with a fifteen-second TTL, matching the offer window. The TTL is important: if the driver's app crashes mid-offer, the lock releases itself. A lock without a TTL means that driver is invisible forever."

"I also offer to drivers **one at a time** rather than broadcasting. Broadcasting gets two acceptances and forces me to reject someone who already committed, which is a bad experience. If three drivers decline or time out, I widen the radius."

"Finally, a trip is a long-running transaction – minutes to hours – so it can't be a database transaction. I model it as an explicit state machine with idempotent transitions, so a driver double-tapping 'arrived' doesn't corrupt anything."

8. Follow-ups

అడిగేది	జవాబు
"ETA ఎలా లెక్కిస్తారు?"	Road graph + historical traffic + live traffic. ఇది ఒక ప్రత్యేక service (OSRM/Valhalla లాంటిది). Matching service దాన్ని batch గా అడుగుతుంది – 20 candidates కి ఒకే call
"Pooling ఎలా మారుస్తుంది?"	Matching ఒక **optimisation problem** అవుతుంది – ఇప్పటికే ఉన్న route lo ఈ కొత్త rider ని కలిపితే ఎంత detour? Threshold దాటకుండా. దీనికి greedy + periodic re-optimisation
"Drivers ని ఎలా reposition చేస్తారు?"	Demand prediction (ML) → drivers కి "ఈ ప్రాంతానికి వెళ్ళండి" అని సూచన + incentive. ఇది matching కంటే వేరే loop
"Geohash precision ఎంత?"	Level ని density ని బట్టి – dense city lo 6 chars (~1.2 km), rural lo 5 (~5 km). H3 lo ఇది resolution parameter
"Trip మధ్యలో driver app crash అయితే?"	Trip state server side lo ఉంది కాబట్టి పోదు. Driver reconnect అయ్యాక server current state ని push చేస్తుంది. Location gap ని route archive lo interpolate చేయడం
"Fraud (fake GPS) ఎలా పట్టుకుంటారు?"	Speed anomaly (ఒక్క update lo 5 కిమీ దూకడం), route ని road graph తో snap చేసి match కాకపోవడం, device signals. ఇది వేరే detection pipeline

PROBLEM 08 · OFFLINE / ONLINE SPLIT

08

Design Typeahead (Search Autocomplete)

Difficulty **Medium-Hard** · Frequency **ఎక్కువ** · నేర్చే concepts: trie, precomputed top-k, batch index build, latency budget

1. The Ask

> "Design Google's search autocomplete. As the user types, show the top 5 suggestions – and it has to feel instant."

ఈ problem యొక్క ప్రత్యేకత: **latency budget చాలా కఠినం.** User అక్షరం టైప్ చేసిన 100 ms లోపు suggestions కనిపించాలి. అంతకంటే ఆలస్యమైతే user తర్వాతి అక్షరం టైప్ చేసేస్తాడు, మన జవాబు అప్పటికే పనికిరాదు.

ఈ PROBLEM నేర్పే అతి ముఖ్యమైన PATTERN

Read time lo ఏ పనీ చేయకూడదు. Query వచ్చినప్పుడు ranking లెక్కలు, aggregation, sorting – ఏమీ చేయడానికి సమయం లేదు. అంటే **జవాబు ముందే సిద్ధంగా ఉండాలి**. ఇదే *offline build + online serve* pattern – ఇది search, recommendations, leaderboards, feed ranking అన్నిచోట్లా వస్తుంది.

2. Clarifying Questions

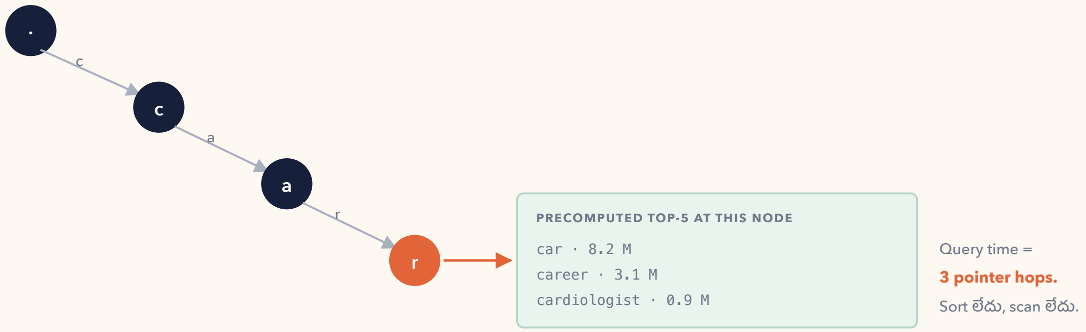
ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Prefix match మాత్రమేనా, typo tolerance ఉందా?	Fuzzy అంటే edit-distance – పూర్తిగా వేరే index	ప్రస్తుతం prefix మాత్రమే
ఎన్ని suggestions?	Top-k lo k విలువ – precompute size ని నిర్ణయిస్తుంది	5
Personalized నా global నా?	Personalized అంటే ప్రతి user కి వేరే index – భారీ	Global (personalization ని follow-up lo)
Suggestions ఎంత తాజాగా?	Real-time అంటే streaming index update	1 గంట సరిపోతుంది (trending కి 5 నిమిషాలు)
Ranking దేని ఆధారంగా?	Frequency, recency, CTR	Frequency + recency
Offensive terms filter?	తప్పనిసరి – index build lo blocklist	అవును

3. Estimation

లెక్క	విలువ
Searches / day	5 B
ఒక search lo keystrokes	~20
Debounce తర్వాత requests	~5 per search
Autocomplete requests / day	25 B
QPS	25B ÷ 86,400 ≈ **290,000 / sec** (peak ≈ 900,000)
Unique queries index lo	~100 M
సగటు query పొడవు	20 chars
Trie memory (naive)	100M × 20 × (node overhead) ≈ **చాలా GB** – sharding తప్పనిసరి
Latency budget	**p99 < 100 ms** (network తో సహా)

4. Deep Dive – Trie + Precomputed Top-K

TRIE WITH TOP-K CACHED AT EVERY NODE

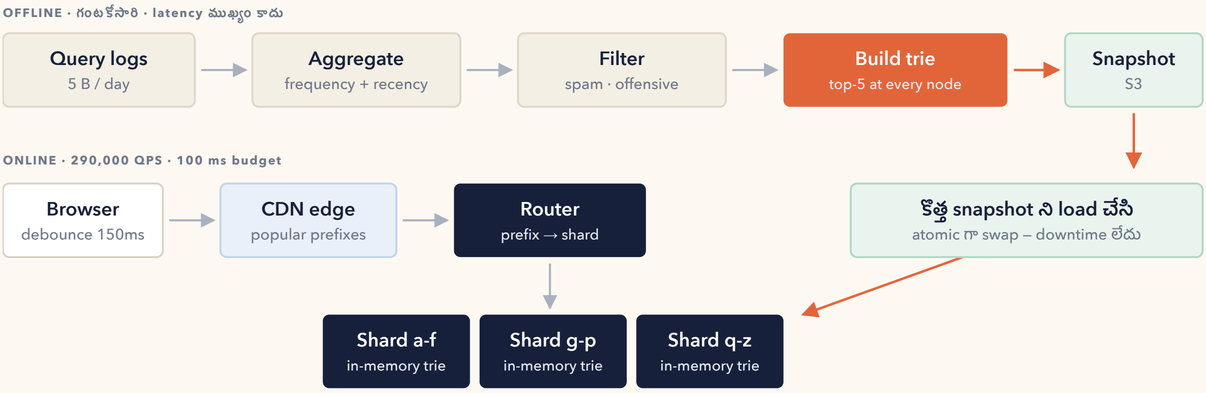


ఇది లేకపోతే: "car" prefix కింద ఉన్న అన్ని leaf nodes ని traverse చేసి, count ప్రకారం sort చేసి, top-5 తీయాలి – "a" లాంటి చిన్న prefix కి అది లక్షల nodes. 100 ms budget lo అది అసాధ్యం. అందుకే ప్రతి node lo జవాబు ముందే దాచడం.

నిర్ణయం	ఎందుకు
ప్రతి node lo top-5 cache	Query time O(prefix length) – data పరిమాణంతో సంబంధం లేదు
Memory ఖర్చు	ఒక్కో node కి 5 strings అదనం. Memory పెరుగుతుంది, కానీ latency కి అది సరైన బేరం
Trie ని immutable గా ఉంచడం	Serve చేస్తున్న trie ని ఎప్పుడూ modify చేయకూడదు. కొత్తది build చేసి **atomic swap** – lock లేదు, consistency సమస్య లేదు

5. Deep Dive – Offline Build + Online Serve

TWO INDEPENDENT PIPELINES · ఇదే ఈ DESIGN యొక్క గుండె



SERVING PATH లో DATABASE లేదు · DISK లేదు · అంతా RAM లో. అందుకే 5 ms lo జవాబు వస్తుంది.

ఈ విభజనే జవాబు. భారీ పని (5 billion logs ని aggregate చేయడం) offline lo, గంటకోసారి. Serving path lo కేవలం memory lookup. రెండూ కలిపేస్తే – query time lo aggregate చేయడానికి ప్రయత్నిస్తే – latency budget ఎప్పటికీ కుదరదు.

6. Sharding & Client-side Optimisations

Sharding option	Verdict
First letter ప్రకారం (a-z)	❌ Uneven – 'a', 's' shards భారీ; 'x', 'z' ఖాళీ
First 2-3 letters ప్రకారం, load-balanced grouping	✅ Historical traffic ని బట్టి prefixes ని groups గా కట్టడం
Hash of prefix	❌ పని చేయదు – prefix search కి ఒకే shard lo related prefixes ఉండాలి

CLIENT-SIDE తో సగం సమస్య పోతుంది

1. Debounce (150 ms) – ప్రతి keystroke కి request పంపొద్దు. ఇది traffic ని 20 → 5 కి తగ్గిస్తుంది.
2. Local cache – "car" కి జవాబు వచ్చాక, user "card" టైప్ చేస్తే – "car" జవాబులో "card" ఉంటే ఆ suggestions ని local గా filter చేయొచ్చు, request అవసరం లేదు.
3. CDN caching – "fac" లాంటి popular prefixes కి జవాబు అందరికీ ఒకటే. దాన్ని edge lo 5 నిమిషాలు cache చేస్తే backend traffic భారీగా తగ్గుతుంది.

ఈ మూడూ కలిపి 290,000 QPS ని backend కి చేరే ముందే **80%+ తగ్గిస్తాయి**. Client ని design lo భాగంగా చూడటం – ఇది చాలా మంది మర్చిపోతారు.

7. Freshness – Trending terms

గంటకోసారి build సరిపోతుంది... **సాధారణంగా**. కానీ ఏదైనా పెద్ద సంఘటన జరిగినప్పుడు కొత్త term ని జనం వెతకడం మొదలుపెడతారు, అది గంట ఆలస్యం అయితే పనికిరాదు.

TWO-LAYER APPROACH

"Base trie గంటకోసారి batch build. దాని పైన ఒక చిన్న **real-time layer** – గత 5 నిమిషాల query stream నుంచి trending terms ని Redis lo. Query వచ్చినప్పుడు రెండింటినీ చూసి merge చేస్తాను. Real-time layer చిన్నది (కొన్ని వేల terms) కాబట్టి ఇది latency కి పెద్దగా ఖర్చు కాదు. ఇది **lambda architecture** – batch layer ఖచ్చితత్వానికి, speed layer తాజాదనానికి."

8. Scale & Failure

సమస్య	పరిష్కారం
Shard down	ప్రతి shard కి 3+ replicas. Autocomplete optional feature – worst case suggestions కనిపించవు, search మాత్రం పని చేస్తుంది. **Graceful degradation**
Snapshot load సమయం	Trie build చేసిన serialized form ని load చేయడం (mmap). Load పూర్తయ్యాకే traffic ని కొత్త దానికి మార్చడం
Memory పరిమితి	Trie ని compress: common prefixes ని merge (radix tree), strings ని interned pool lo, top-5 lo ids మాత్రమే ఉంచి strings ని వేరే array lo
Bad suggestion (offensive)	Build pipeline lo blocklist + ఒక manual override list, అది real-time layer lo apply అవుతుంది (గంట ఎదురుచూడకుండా)
Traffic spike	Serving stateless కాబట్టి autoscale. CDN అప్పటికే 60%+ absorb చేస్తుంది

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me pin the requirements: prefix matching only or typo tolerance, how many suggestions, global or personalised, and how fresh do they need to be? I'll assume prefix-only, top five, global, and roughly hourly freshness – with a carve-out for trending terms."

"The defining constraint here is the latency budget. Suggestions have to appear within about a hundred milliseconds, otherwise the user has already typed the next character and my answer is stale. At around three hundred thousand queries a second, that means **I cannot do any real work at query time** – no aggregation, no sorting, no database."

"So the whole design is an offline/online split. Offline, a batch job reads query logs, aggregates by frequency and recency, filters spam and offensive terms, and builds a trie. Online, servers hold that trie in memory and do nothing but walk it."

"The key trick is that I **precompute the top five at every node**, not just at leaves. Without that, answering a short prefix like 'a' would mean traversing millions of descendant nodes and sorting them. With it, a query is just three or four pointer hops – the cost is independent of how much data sits below."

"The served trie is immutable. A new build produces a snapshot, servers load it, and swap atomically. No locking, no partial-update consistency problems."

"For sharding, I wouldn't shard by first letter – 'a' and 's' would be enormous while 'x' and 'z' sit idle. I'd group prefixes into balanced buckets based on historical traffic. Hashing doesn't work here at all, because prefix search needs related prefixes co-located."

"A lot of the load never needs to reach me. Debouncing at a hundred and fifty milliseconds cuts requests per search from twenty to about five. The client can filter locally – if I already have results for 'car', typing 'card' often doesn't need a round trip. And popular prefixes are identical for every user, so a CDN can cache them for a few minutes. Together those remove most of the traffic before it hits a server."

"Hourly freshness fails for breaking news, so I'd add a small real-time layer: trending terms from the last five minutes in Redis, merged with the batch result at query time. That's a lambda architecture – the batch layer for accuracy, a thin speed layer for recency."

"On failure, I'd note that autocomplete is a convenience, not the product. If a shard is down, the right behaviour is to return nothing and let the user press enter – degrade gracefully rather than fail the search box."

9. Follow-ups

అడిగేది	జవాబు
"Typo tolerance ఎలా?"	Prefix trie సరిపోదు. **BK-tree** లేదా edit-distance-1 variants ని ముందే generate చేసి index lo పెట్టడం. లేదా n-gram index + fuzzy scoring. ఖరీదు గణనీయంగా పెరుగుతుంది

అడిగేది	జవాబు
"Personalization ఎలా కలపాలి?"	Global top-20 ని తీసుకుని, user history తో re-rank చేయడం. User-specific trie కాదు – అది scale కాదు. Personalization ఎప్పుడూ **re-ranking layer**, index layer కాదు
"బహుళ భాషలు?"	భాష ప్రకారం వేరే tries. Language detection ని client locale + typed characters నుంచి. Unicode prefix handling జాగ్రత్త (ఒక "అక్షరం" బహుళ code points కావొచ్చు)
"Index build ఎంత సమయం?"	100M queries → MapReduce/Spark job, ~30 నిమిషాలు. గంటకోసారి build చేయడానికి ఇది సరిపోతుంది
"సరిగ్గా Google లాగా చేయాలంటే?"	Google personalization + location + session context + ML ranking వాడుతుంది. కానీ **అంతర్గత structure ఇదే** – precomputed candidates + re-ranking
"Search results కూడా design చేయాలంటే?"	అది పూర్తిగా వేరే problem – inverted index, TF-IDF/BM25, distributed scoring. Typeahead అనేది దాని ముందు ఉన్న చిన్న, ప్రత్యేకమైన system

PROBLEM 09 · CORRECTNESS OVER SCALE

09

Design a Payment System & Ledger

Difficulty **Hard** · Frequency **ఎక్కువ (fintech lo తప్పనిసరి)** · నేర్పే concepts: idempotency, double-entry, saga, reconciliation

1. The Ask

> "Design the payment system for an e-commerce site. A user pays, we charge their card through a payment provider, and our books must always be correct."

ఈ PROBLEM LO SCALE ఒక ఎర

QPS చిన్నది – సెకనుకి కొన్ని వేలు మాత్రమే. కానీ ఇక్కడ **ఒక్క తప్పు = నిజమైన డబ్బు నష్టం**. Feed lo ఒక post మిస్ అయితే ఎవరూ పట్టించుకోరు. ఇక్కడ ఒక్క double-charge అంటే customer కోపం, refund, chargeback, regulatory problem. అందుకే ఈ interview lo మీరు **scale గురించి కాదు, correctness గురించి** మాట్లాడాలి. "How do you make sure money is never lost or duplicated?" – ఇదే అసలు ప్రశ్న.

2. Clarifying Questions

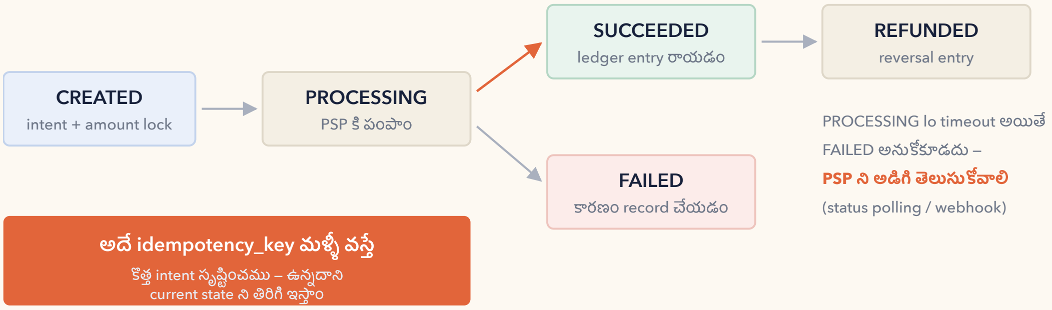
ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
మనం card data ని handle చేస్తామా?	చేస్తే PCI-DSS compliance – భారీ బాధ్యత	లేదు, PSP (Stripe/Razorpay) tokenization
ఏఏ payment methods?	ఒక్కో దానికీ వేరే flow (UPI async, card sync)	Card, UPI, wallet
Refunds, partial refunds?	Ledger design ని మారుస్తుంది	అవును
బహుళ currencies?	FX rate ని ఎప్పుడు lock చేయాలి అనే ప్రశ్న	ఒక్క currency (ఇప్పటికి)
Marketplace నా, single seller నా?	Marketplace అంటే split payments, payouts	Single seller
Reconciliation ఎవరిది?	Finance team కి రోజువారీ report కావాలి	మనదే

3. Deep Dive – Idempotency (ఇదే మొదటి రక్షణ గోడ)

దృశ్యం: User "Pay" నొక్కాడు. Request వెళ్ళింది, PSP charge చేసింది, కానీ జవాబు తిరిగి రాకముందే network timeout అయింది. App "Failed" చూపించింది. User మళ్ళీ "Pay" నొక్కాడు.

> **IDEMPOTENCY లేకపోతే**
>
> User కి **రెండుసార్లు charge** అయింది. అతను చూసేది ఒకే order, కానీ bank statement lo రెండు entries. ఇది అతి సాధారణమైన payment bug – మరియు అతి ఖరీదైనది.

PAYMENT INTENT · STATE MACHINE



PROCESSING నుంచి FAILED కి తొందరపడి వెళ్ళకూడదు. Timeout అంటే "జరగలేదు" అని అర్థం కాదు – "ఏం జరిగిందో తెలియదు" అని అర్థం. ఈ తేడాయే payment systems lo అతి ముఖ్యమైన మానసిక మార్పు.

```
async function pay(req) {
  const key = req.idempotencyKey;              // client generate చేసినది (UUID)

  // 1. ఈ key ఇప్పటికే ఉందా? ఉంటే ఉన్న ఫలితమే తిరిగి ఇవ్వు – మళ్ళీ charge చేయొద్దు.
  const existing = await db.paymentIntents.findByKey(key);
  if (existing) return existing;                // అదే జవాబు, side effect లేదు

  // 2. లేకపోతే intent ని CREATED గా రాయడం (ఇక్కడ unique constraint మీద ఆధారపడుతున్నాం)
  const intent = await db.paymentIntents.insert({
    key, orderId: req.orderId, amount: req.amount, status: 'CREATED',
  });                                            // duplicate key error → step 1 కి
retry

  // 3. PSP కి పంపేటప్పుడు మన key నే forward చేయడం –
  //    PSP కూడా idempotent, అంటే మనకి రెండు రక్షణ పొరలు.
  await db.paymentIntents.setStatus(intent.id, 'PROCESSING');
  const result = await psp.charge({ ...req, idempotencyKey: key });

  return finalise(intent.id, result);            // SUCCEEDED / FAILED + ledger entry
}
```

4. Deep Dive – Double-Entry Ledger

"Balance ఒక column lo ఉంచి, `UPDATE balance = balance - 100` చేస్తే సరిపోదా?" – సరిపోదు. అలా చేస్తే డబ్బు ఎక్కడి నుంచి ఎక్కడికి వెళ్ళిందో record లేదు. Audit సాధ్యం కాదు, bug వస్తే ఎక్కడ తప్పిందో తెలియదు.

DOUBLE-ENTRY · ప్రతి లావాదేవీకి రెండు వరుసలు, మొత్తం ఎప్పుడూ సున్నా

TRANSACTION t-8891 · CUSTOMER PAYS ₹1,000

entry_id	txn_id	account	debit	credit	
e-1	t-8891	`customer:4471:wallet`	1,000.00	—	
e-2	t-8891	`merchant:88:receivable`	—	970.00	
e-3	t-8891	`platform:fee_revenue`	—	30.00	
		SUM	**1,000.00**	**1,000.00**	✓ balanced

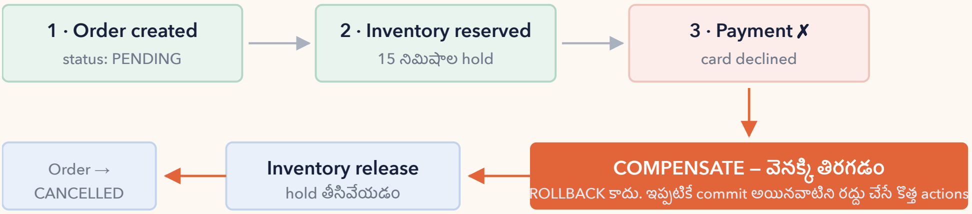
ప్రతి transaction lo debits sum = credits sum. ఇది invariant.
ఏ entry ని ఎప్పుడూ update/delete చేయము – తప్పు జరిగితే reversal entry రాస్తాం

నియమం	ఎందుకు
Append-only, immutable	Ledger row ని ఎప్పుడూ update చేయకూడదు. తప్పు జరిగితే opposite entry రాయాలి. దీంతో పూర్తి audit trail
Debits = Credits invariant	ఇది ఒక **automated check**. ప్రతి రాత్రి `SUM(debit) - SUM(credit) = 0` కాకపోతే alert. Bug వెంటనే బయటపడుతుంది
Balance ఒక derived value	`SELECT SUM(credit) - SUM(debit) WHERE account = ?`. వేగం కోసం snapshot cache, కానీ **source of truth entries మాత్రమే**
Integers, floats కాదు	`0.1 + 0.2 !== 0.3`. Money ని ఎప్పుడూ paise/cents lo integer గా store చేయాలి. ఇది ప్రస్తావిస్తే గట్టి point

5. Deep Dive – Order + Payment + Inventory (Saga)

Order create, payment charge, inventory reserve – ఇవి మూడు వేర్వేరు services. ఒక్క transaction lo చేయలేం.

SAGA · FORWARD STEPS + COMPENSATIONS



Compensation ఎప్పుడూ rollback కాదు – payment already succeed అయి తర్వాతి step fail అయితే, "charge ని undo" చేయలేం. బదులుగా REFUND అనే కొత్త transaction రాస్తాం.
అందుకే డబ్బు తీసే step ని ఎప్పుడూ saga lo చివరన పెట్టాలి.

2PC కాదు, ఎందుకు?

"Two-phase commit ఇక్కడ వాడను. కారణం – coordinator crash అయితే participants అందరూ **locks పట్టుకుని blocked** గా ఉండిపోతారు. External PSP మీద అసలు 2PC చేయలేం, వాళ్ళు మన coordinator ని పాటించరు. అందుకే **saga** – ప్రతి step ని commit చేసి, fail అయితే compensating actions నడపడం. Availability కోసం atomicity ని వదులుకోవడం, ఆ మధ్యంతర inconsistency ని explicit states తో బహిర్గతం చేయడం."

6. Deep Dive – PSP Webhooks & Reconciliation

సమస్య	పరిష్కారం
Webhook ఆలస్యంగా వస్తుంది	Webhook మీద మాత్రమే ఆధారపడొద్దు. PROCESSING lo ఉన్న intents కి **polling job** కూడా – 30s, 2min, 10min తర్వాత PSP ని అడగడం
Webhook తప్పు order lo వస్తుంది	`succeeded` కంటే ముందు `refunded` రావొచ్చు. ప్రతి webhook lo ఉన్న **event timestamp/version** ని చూసి, పాతది అయితే వదిలేయడం
Webhook duplicate	Event id ని store చేసి dedupe – idempotency మళ్ళీ ఇక్కడ
Webhook నకిలీది	Signature verification తప్పనిసరి (HMAC). లేకపోతే ఎవరైనా "payment succeeded" అని పంపొచ్చు
మన record vs PSP record తేడా	**రోజువారీ reconciliation** – PSP settlement file ని download చేసి మన ledger తో line-by-line match. తేడాలని exception queue lo పెట్టి manual review

ఈ వాక్యం చెప్తే మీరు FINTECH తెలిసినవారు

"**Source of truth మనం కాదు – bank/PSP.** మన ledger ఒక record మాత్రమే. అందుకే reconciliation ఐచ్ఛికం కాదు, తప్పనిసరి. ప్రతి రాత్రి PSP settlement report తో మన entries ని match చేసి, తేడాలని ఒక exception queue lo పెడతాను. ఏ payment system అయినా ఈ reconciliation job లేకపోతే – ఎప్పుడో ఒకప్పుడు నిశ్శబ్దంగా డబ్బు తప్పిపోతుంది, ఎవరికీ తెలియదు."

7. Scale & Failure

సమస్య	పరిష్కారం
PSP down	Circuit breaker + secondary PSP కి failover. Intent CREATED lo ఆగి, provider తిరిగి వచ్చాక retry. User కి "processing" చూపించడం, "failed" కాదు
DB write fail (ledger రాసేటప్పుడు)	Payment మరియు ledger entry **ఒకే DB transaction lo** ఉండాలి. వేరైతే – charge అయి ledger రాయకపోతే – డబ్బు కనిపించకుండా పోతుంది
Event publish + DB write (dual write)	**Outbox pattern** – event ని అదే transaction lo `outbox` table lo రాసి, ఒక relay దాన్ని Kafka కి పంపడం. దీంతో "DB rollback అయినా event వెళ్ళిపోయింది" అనే సమస్య పోతుంది

సమస్య	పరిష్కారం
Ledger పరిమాణం	Append-only కాబట్టి ఎప్పటికీ పెరుగుతుంది. నెలవారీ partitioning + పాత partitions ని cold storage కి. Account balance snapshots (month-end) తో query వేగం
Hot account (platform fee account)	ప్రతి transaction కీ ఈ ఒక్క account కి entry – contention. **Sharded sub-accounts** (`fee_revenue:shard_0..9`) పెట్టి, balance చదివేటప్పుడు కలపడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"First, a scoping question that matters a lot: do we touch raw card data? I'll assume no – we use a provider like Stripe and only ever see tokens, which keeps us out of full PCI scope."

"I want to name the shape of this problem up front. The throughput here is modest – a few thousand a second. This is not a scale problem, it's a **correctness** problem. Losing a post in a feed costs nothing; double-charging a customer costs money, trust, and possibly a regulator's attention. So I'll spend my time on the guarantees, not the box count."

"The first line of defence is idempotency. The classic failure is: the user taps Pay, we charge the provider, the response times out, the app shows 'failed', and the user taps again. Without protection that's two charges. So every payment carries a client-generated idempotency key. We insert a payment intent keyed on it with a unique constraint; if the key already exists we return the existing intent instead of charging again. And we forward that same key to the provider, so there are two independent layers of protection."

"A related subtlety: on timeout I do **not** mark the payment failed. A timeout means 'I don't know what happened', not 'it didn't happen'. The intent stays in PROCESSING and we resolve it via webhook or by polling the provider. Confusing 'unknown' with 'failed' is how systems lose money."

"For the books I'd use a **double-entry ledger** rather than a balance column. Every transaction writes at least two rows that sum to zero – debit the customer, credit the merchant, credit the platform fee. The ledger is append-only and immutable: corrections are reversal entries, never updates. That gives a complete audit trail, and it gives me an automated invariant – total debits must equal total credits – so I can detect a bug the same night instead of during an audit. And money is stored as integers in the smallest unit, because floating point cannot represent currency correctly."

"Order, inventory and payment span three services, so I'd use a **saga** rather than two-phase commit. 2PC leaves participants holding locks if the coordinator dies, and it's simply not available to me across an external payment provider. In a saga each step commits independently and failures trigger compensating actions. Compensation isn't rollback – once a charge succeeds you can't un-charge it, you issue a refund, which is a new transaction. That's why I order the saga to take money **last**."

"On provider integration: I never rely on webhooks alone. Webhooks arrive late, out of order, duplicated, and can be forged. So I verify signatures, dedupe on event ID, ignore events older than my current state, and run a polling job as a backstop for anything stuck in PROCESSING."

"Finally, the thing that distinguishes a real payment system: **daily reconciliation**. We are not the source of truth – the provider and the bank are. Every night I pull the settlement file and match it line by line against our ledger, and anything that doesn't match goes to an exception queue for a human. Without that job, discrepancies accumulate silently and nobody finds out for months."

8. Follow-ups

అడిగేది	జవాబు
"Partial refund ఎలా?"	కొత్త transaction, తక్కువ మొత్తానికి reversal entries. మొత్తం refunded amount ≤ original amount అనే check. Original entry ని ఎప్పుడూ ముట్టుకోకూడదు
"Multi-currency?"	ప్రతి account కి ఒక currency. Cross-currency అంటే FX conversion account గుండా – మూడు entries (debit USD, credit FX, debit FX, credit INR). Rate ని transaction సమయంలో lock చేసి store చేయడం
"Marketplace / split payment?"	Merchant కి ఒక receivable account, payout ఒక వేరే transaction. Escrow period lo డబ్బు platform account lo ఉంటుంది – ledger lo ఇది సహజంగా representable
"Chargeback?"	ఒక కొత్త transaction (reversal + fee). ఒక `disputes` state machine. Chargeback వచ్చినప్పుడు order ని కూడా flag చేయాలి
"Fraud detection?"	Payment path lo ఒక synchronous risk check (velocity, device, amount anomaly) + async deeper scoring. Risk score ఎక్కువ అయితే manual review queue
"Exactly-once ఎలా guarantee చేస్తారు?"	చేయము – **at-least-once + idempotency = exactly-once effect**. ఇదే నిజాయితీ గల జవాబు. "Exactly-once delivery" అని చెప్పే candidate ని interviewer వెంటనే గుర్తుపడతాడు

PROBLEM 10 · TIME & COORDINATION

10

Design a Distributed Job Scheduler

Difficulty **Hard** · Frequency **మధ్యస్థం (కానీ SSE lo ఎక్కువ)** · నేర్చే concepts: time bucketing, leases, zombie detection, jitter

1. The Ask

> "Design a system that runs scheduled jobs — cron at scale. A hundred million jobs, each with its own schedule, executed reliably across a fleet of workers."

కష్టం 1 · Discovery

"ఇప్పుడు ఏ jobs run అవ్వాలి?" — 100 million rows lo ప్రతి సెకనుకి ఈ query వేయలేం.

కష్టం 2 · Coordination

50 workers ఉన్నారు. ఒకే job ని ఇద్దరు తీసుకోకుండా ఎలా ఆపాలి? Worker crash అయితే?

కష్టం 3 · Time

అందరూ "ప్రతి గంట మొదట్లో" అని schedule చేస్తారు. :00 కి లక్షల jobs ఒకేసారి.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
One-time నా recurring నా?	Recurring అంటే run తర్వాత తర్వాతి run ని schedule చేయాలి	రెండూ
Precision ఎంత?	సెకను ఖచ్చితత్వం vs నిమిషం — architecture మారుతుంది	±1 సెకను
Job ఎంతసేపు నడుస్తుంది?	గంటల job అంటే heartbeat + lease renewal కావాలి	సెకన్ల నుంచి గంటల దాకా
At-least-once నా at-most-once నా?	ఇదే అతి ముఖ్యమైన ప్రశ్న	At-least-once (idempotent jobs)
Jobs మధ్య dependency ఉందా?	ఉంటే ఇది workflow engine (Airflow) అవుతుంది	లేదు, స్వతంత్రం
Failure అయితే retry?	Backoff policy	3 retries, exponential

AT-LEAST-ONCE VS AT-MOST-ONCE — ఇక్కడ నిజం చెప్పండి

"రెండూ ఒకేసారి ఇవ్వలేను. Worker job ని run చేసి, 'పూర్తయింది' అని రాయకముందే crash అయితే — నేను దాన్ని మళ్ళీ run చేయాలా (at-least-once, duplicate risk) లేక వదిలేయాలా (at-most-once, miss risk)? నేను **at-least-once** ఎంచుకుంటాను, ఎందుకంటే job ని miss చేయడం కంటే రెండుసార్లు run చేయడం మేలు — మరియు jobs ని idempotent గా రాయమని contract పెడతాను. ఇది system limitation ని user కి బదిలీ చేయడం — కానీ అదే నిజాయితీ గల design."

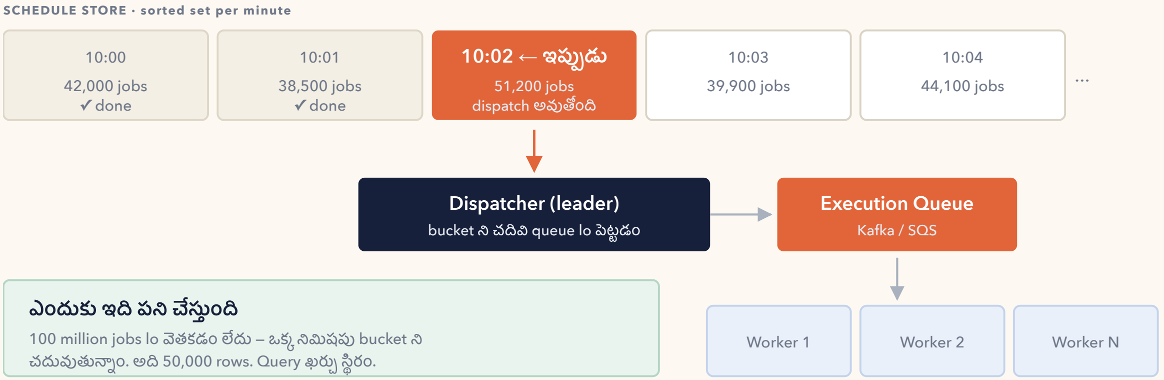
3. Estimation

లెక్క	విలువ
మొత్తం scheduled jobs	100 M
సగటు execution rate	≈ 10,000 / sec
Peak (:00 spike)	≈ **200,000 / sec** — సగటు కంటే 20 రెట్లు
Job metadata size	≈ 1 KB
Metadata storage	100 GB
Execution history (30 రోజులు)	10K/s × 86400 × 30 × 200 B ≈ 5 TB

4. Deep Dive — "ఇప్పుడు ఏ jobs due?" (Discovery)

Approach	సమస్య
`SELECT * FROM jobs WHERE next_run <= NOW()` ప్రతి సెకనుకి	❌ 100M rows మీద index scan, ప్రతి సెకనుకి, 50 workers నుంచి. DB చస్తుంది
Job కి ఒక OS timer	❌ 100 million timers. Memory పేలుతుంది, restart అయితే అన్నీ పోతాయి
Time-bucketed partitions ✅	ప్రతి నిమిషానికి ఒక bucket. ఆ నిమిషం వచ్చినప్పుడు ఆ bucket ని మాత్రమే చదవడం
Hierarchical timing wheel	✅ In-memory కి అద్భుతం (Kafka ఇదే వాడుతుంది), కానీ persistence కోసం bucketing తోడు కావాలి

TIME-BUCKETED SCHEDULING · నిమిషానికి ఒక BUCKET

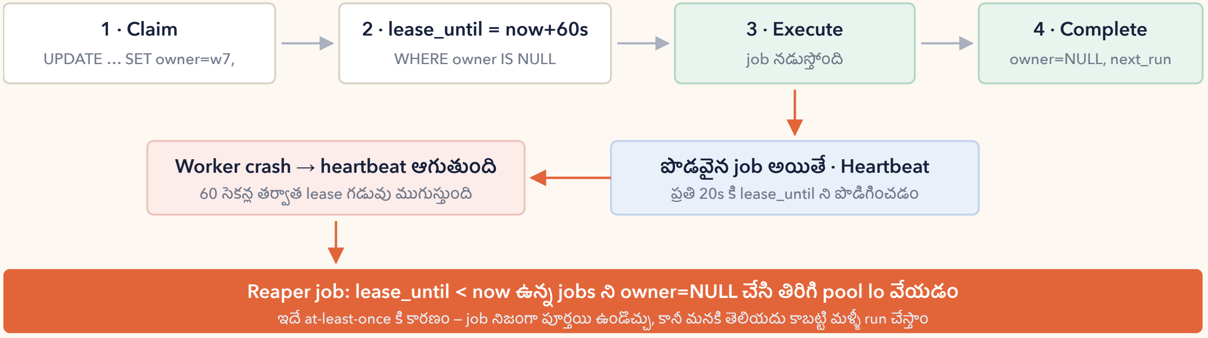


DISPATCHER ముందుగానే పని చేస్తుంది – 10:02 bucket ని 10:01:55 కే చదివి queue lo పెడుతుంది

Dispatcher మరియు worker ని విడదీయడం ముఖ్యం. Dispatcher కేవలం "ఏవి due" అని చూసి queue lo పెడుతుంది – ఇది వేగవంతమైన, తేలికైన పని. Workers అసలు execution చేస్తారు – అది నెమ్మది, భారీ పని. వీటిని కలిపేస్తే ఒక నెమ్మది job మిగతా అన్నిటినీ ఆలస్యం చేస్తుంది.

5. Deep Dive – ఒకే job ని ఇద్దరు తీసుకోకుండా (Leases)

LEASE LIFECYCLE · WORKER CRASH ని ఎలా తట్టుకోవడం



నియమం	ఎందుకు
Lease, lock కాదు	Lock శాశ్వతం – worker చస్తే job శాశ్వతంగా ఇరుక్కుంటుంది. Lease కి గడువు ఉంది, తనంతట తానే విడుదల అవుతుంది
Conditional update	`WHERE owner IS NULL` – ఇద్దరు ఒకేసారి claim చేస్తే ఒకరికే 1 row affected వస్తుంది. Distributed lock service అవసరం లేదు
Heartbeat renewal	గంట నడిచే job కి 60 సెకన్ల lease సరిపోదు. Worker నడుస్తున్నంతసేపు lease ని పొడిగించాలి
Reaper తప్పనిసరి	లేకపోతే crash అయిన worker యొక్క jobs శాశ్వతంగా stuck

6. Deep Dive – :00 Thundering Herd

నిజం: ప్రతి ఒక్కరూ "ప్రతి గంటకి", "ప్రతి రోజు అర్ధరాత్రి" అని schedule చేస్తారు. అంటే **సెకనుకి 200,000 jobs** – సగటు కంటే 20 రెట్లు.

పరిష్కారం	ఎలా
Automatic jitter	User "ప్రతి గంట" అంటే, మనం ఆ job ని `hash(job_id) % 60` సెకన్ల ఆలస్యంతో run చేయడం. అదే job ఎప్పుడూ అదే offset lo – deterministic, కానీ load నిమిషం అంతటా పరుచుకుంటుంది
Sub-minute buckets	నిమిషానికి బదులు 10-సెకన్ల buckets – spike ని మరింత సన్నగా చేస్తుంది
Queue absorption	Dispatcher అన్నిటినీ queue lo పోసేస్తుంది. Workers తమ వేగంతో తీసుకుంటారు. Queue అనేది shock absorber
Priority lanes	Critical jobs కి వేరే queue – batch reports వెనక అవి వేలాడకూడదు

JITTER గురించి ఇలా చెప్పండి

"User 'ప్రతి గంట' అని అడిగినప్పుడు అతనికి కావలసింది **గంటకోసారి**, సరిగ్గా :00:00 కి కాదు. ఆ తేడాని వాడుకుని నేను job ని hash ఆధారంగా నిమిషం అంతటా పరుస్తాను. User కి ఏ తేడా తెలియదు, నా peak load 20 రెట్లు తగ్గుతుంది. **Requirement ని కాస్త వదులు చేయడం ద్వారా భారీ engineering ఆదా** – ఇది system design lo తరచుగా అత్యుత్తమ కదలిక."

7. Scale & Failure

సమస్య	పరిష్కారం
Dispatcher SPOF	Leader election (ZooKeeper/etcd). Leader చస్తే కొత్తవాడు వస్తాడు. Buckets idempotent – ఒకే bucket ని రెండుసార్లు dispatch చేసినా job level lease duplicate ని ఆపుతుంది
Dispatcher వెనకబడటం	Bucket ని shard చేయడం – `10:02:shard_0..9`, ప్రతి shard కి వేరే dispatcher. అప్పుడు leader ఒక్కడే కాదు
Job execution చాలా నెమ్మది	Timeout పెట్టడం. Timeout దాటితే kill + retry. లేకపోతే ఒక job worker ని శాశ్వతంగా ఆక్రమిస్తుంది
Poison job (ఎప్పుడూ fail)	3 retries తర్వాత `SUSPENDED` state కి పంపి alert. మౌనంగా అనంతంగా retry చేయడం resource వృథా
Clock skew	Servers మధ్య clock తేడా. అందుకే job "due" నిర్ణయం **ఒకే చోట** (dispatcher/DB time) జరగాలి, workers తమ సొంత clock ని నమ్మకూడదు
Recurring job overlap	ప్రతి 5 నిమిషాలకి job, కానీ ఒక run 7 నిమిషాలు తీసుకుంది. Overlap allow చేయాలా? సాధారణంగా **వద్దు** – ఒక `running` flag తో skip చేయడం
Backfill	System 2 గంటలు down అయితే, 2 గంటల jobs ఏమవుతాయి? అన్నీ ఒకేసారి run చేస్తే overload. Policy కావాలి: skip, లేదా rate-limited backfill

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"A few questions: one-time jobs, recurring, or both? What precision do we need – seconds or minutes? How long can a job run? And most importantly, do we promise at-least-once or at-most-once execution?"

"On that last one I'd be explicit: I can't give both. If a worker executes a job and dies before recording completion, I either re-run it or skip it. I'd choose **at-least-once** – missing a job is usually worse than running it twice – and make idempotency part of the contract with job authors. That's pushing a constraint onto the user, but it's the honest engineering answer rather than pretending exactly-once is available."

"The first hard part is discovery: with a hundred million jobs, I can't run 'select where next_run is due' every second. And I obviously can't hold a hundred million OS timers. Instead I **bucket by time** – a sorted set per minute. To find what's due at 10:02, I read the 10:02 bucket, which is maybe fifty thousand entries. The cost of that query is constant regardless of how many jobs exist in total."

"I separate dispatch from execution. A dispatcher reads the bucket slightly ahead of time and pushes jobs onto a queue; workers consume from the queue and actually run them. If I merged those roles, one slow job would delay dispatch for everything else."

"The second hard part is making sure two workers don't run the same job. I'd use **leases rather than locks**. A worker claims a job with a conditional update – set owner and lease expiry where owner is null – so only one worker gets a row affected. The lease expires after sixty seconds, and a long-running job renews it with a heartbeat. If the worker crashes, the heartbeat stops, the lease expires, and a reaper returns the job to the pool. A plain lock would leave that job stuck forever."

"The third problem is the one people forget: everyone schedules on the hour. Average load might be ten thousand a second, but at :00 it's two hundred thousand – a twenty-times spike sizing my entire fleet for one second a minute."

"My fix is **deterministic jitter**. When a user says 'every hour', what they actually want is hourly, not exactly at zero seconds. So I offset each job by hash of its ID modulo sixty seconds. Same job always lands at the same offset, so it stays predictable, but the load spreads evenly across the minute. Relaxing a requirement slightly to save a huge amount of engineering is often the strongest move available in system design."

"One correctness detail: the decision about whether a job is due must be made in **one place** – the dispatcher or the database – never on individual workers' clocks, because clock skew across a fleet will cause both double-runs and misses."

"And I'd insist on a policy for backfill. If the scheduler is down for two hours, running two hours of accumulated jobs all at once will take down whatever they call. Usually the answer is to skip missed runs for frequent jobs and rate-limit the backfill for important ones."

8. Follow-ups

అడిగేది	జవాబు
"Cron expression ఎలా parse చేస్తారు?"	Job create/complete అయినప్పుడు `next_run` ని ముందే లెక్కించి store చేయడం. Runtime lo cron parse చేయకూడదు – ముందే లెక్కించిన timestamp మీద index వేగంగా ఉంటుంది
"Timezone ఎలా?"	`next_run` ని ఎప్పుడూ UTC lo store చేయడం, కానీ user timezone ని కూడా ఉంచడం – DST మార్పులు వచ్చినప్పుడు తర్వాతి run ని సరిగ్గా లెక్కించడానికి అది కావాలి
"Job dependencies (A తర్వాత B)?"	అప్పుడు ఇది scheduler కాదు, **workflow engine**. DAG, state per node, upstream success మీద trigger. Airflow/Temporal ఆ space
"1 సెకను కంటే మెరుగైన precision?"	Sub-second కి in-memory **hierarchical timing wheel** (Kafka's purgatory లాంటిది). Persistence కి bucketing, precision కి wheel – రెండూ కలిపి
"Job results ఎక్కడ?"	Execution history table (partitioned by date) + logs ని object storage lo. History ని 30 రోజుల తర్వాత purge
"Multi-tenant fairness?"	ఒక tenant లక్ష jobs schedule చేస్తే మిగతా అందరూ వేచి ఉండకూడదు. Tenant ప్రకారం queue quota / weighted fair queuing

PROBLEM 11 · POLITENESS & DEDUPLICATION

11

Design a Web Crawler

Difficulty **Hard** · Frequency **మధ్యస్థం** · నేర్పే concepts: URL frontier, bloom filter, simhash, politeness, traps

1. The Ask

> "Design a web crawler that downloads a billion pages a month for a search index."

ఈ PROBLEM ఎందుకు మోసపూరితం

"URL తీసుకో, download చెయ్యి, links తీసి queue lo పెట్టు" – ఇది 5 నిమిషాల్లో చెప్పేయొచ్చు. కానీ ఈ problem యొక్క నిజమైన కష్టం అంతా **అనుకోని పరిస్థితుల్లో** ఉంది: ఒకే content వేరే URLs lo, అనంతమైన calendar pages, ఒకే site ని కొట్టి కూల్చడం, robots.txt, redirect loops. Interviewer చూసేది మీరు ఈ **అసహ్యమైన నిజాలని** ఎన్ని ఊహించగలరు అన్నదే.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఎన్ని pages, ఎంత కాలంలో?	Throughput ని ఇది నిర్ణయిస్తుంది	1 B / month
HTML మాత్రమేనా, JS render కావాలా?	JS rendering అంటే headless browser – 50× ఖరీదు	HTML ప్రధానం, కొన్నిటికి JS
ఎంత తరచుగా re-crawl?	Freshness policy	Site importance ని బట్టి
robots.txt పాటించాలా?	తప్పనిసరి – లేకపోతే IP block + legal	తప్పకుండా
Content ఎక్కడ store?	Raw HTML storage భారీ	Object storage
Duplicate content ఏం చేయాలి?	Web lo 30%+ duplicate	Detect చేసి skip

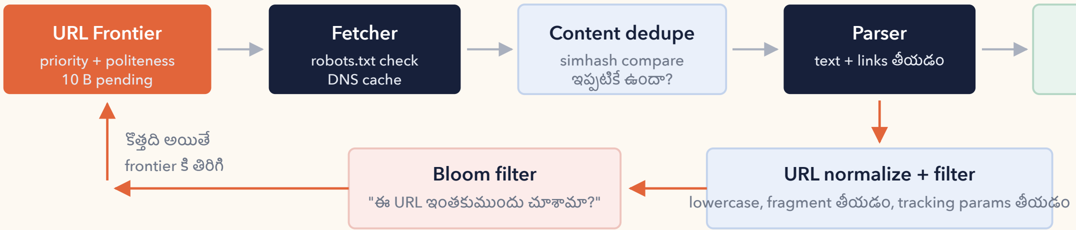
3. Estimation

లెక్క	విలువ
Pages / month	1 B
Pages / sec	1B ÷ (30 × 86,400) ≈ **400 / sec**
సగటు page size	500 KB (HTML + resources కాకుండా HTML ≈ 100 KB)
Bandwidth	400 × 100 KB ≈ **40 MB/sec ≈ 320 Mbps**

లెక్క	విలువ
Raw storage / month	1B × 100 KB = **100 TB** (compressed ≈ 20 TB)
URL frontier size	ఏ క్షణంలోనైనా pending ≈ 10 B URLs
URL seen set	10 B URLs × 8 B hash ≈ 80 GB → **bloom filter** తో ~12 GB

4. Architecture

CRAWLER PIPELINE · ప్రతి STAGE ఒక ప్రత్యేక బాధ్యత



ఇది ఒక CYCLE – crawler ఒక BFS/DFS graph traversal

Page ని fetch చేయడం → అందులోని links తీయడం → అవి కొత్తవా అని చూడటం → frontier lo చేర్చడం.
Web ఒక అనంతమైన graph కాబట్టి ఈ loop ఎప్పటికీ ఆగదు. అందుకే **frontier ఎలా ప్రాధాన్యత ఇస్తుంది** అన్నదే crawler quality ని నిర్ణయిస్తుంది – ఏ pages ముందు, ఏవి ఎప్పటికీ కాదు.

5. Deep Dive – URL Frontier (ఇదే crawler యొక్క మెదడు)

Frontier రెండు విరుద్ధమైన లక్ష్యాలని ఏకకాలంలో సాధించాలి:

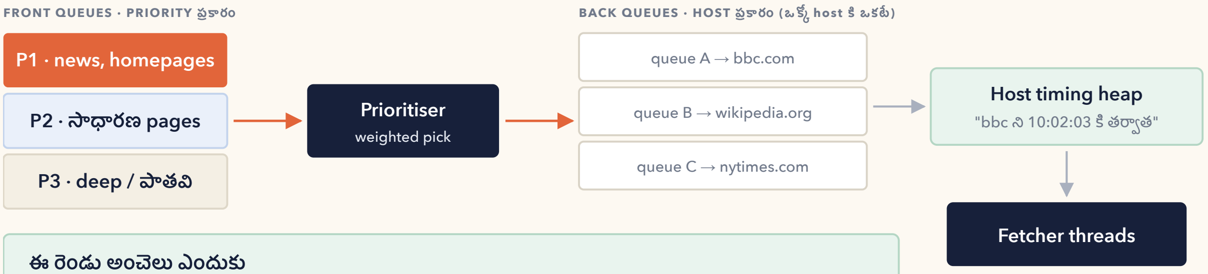
Priority

ముఖ్యమైన pages (high PageRank, తరచుగా మారేవి) ముందు crawl కావాలి.

Politeness

ఒకే host ని సెకనుకి 100 సార్లు కొట్టకూడదు – అది DDoS అవుతుంది, IP block అవుతుంది.

MERCATOR FRONTIER · రెండు అంచెల QUEUES



ఈ రెండు అంచెలు ఎందుకు

Front queues మాత్రమే ఉంటే – high priority URLs అన్నీ ఒకే site వి కావొచ్చు, ఆ site ని కూల్చేస్తాం.
Back queues మాత్రమే ఉంటే – politeness ఉంటుంది కానీ ముఖ్యమైన pages చివర్లో వస్తాయి.

రెండూ కలిపితే: priority ప్రకారం ఎంచుకోవడం, కానీ ఒక్కో host ని దాని సొంత వేగంతోనే కొట్టడం.
ఒక్కో back queue ని ఒకే fetcher thread మాత్రమే చదువుతుంది – అదే politeness guarantee.

6. Deep Dive – రెండు రకాల Duplicates

రకం	ఉదాహరణ	పరిష్కారం
URL duplicate	`site.com/page`, `site.com/page/`, `site.com/page?utm=x`, `SITE.com/Page` – అన్నీ ఒకటే	**Normalisation**: lowercase host, trailing slash, fragment తీయడం, tracking params తీయడం, query params ని sort చేయడం. తర్వాత hash ని **bloom filter** lo
Content duplicate	వేరే URLs, కానీ ఒకే వ్యాసం (mirrors, syndication, printer-friendly version)	**Simhash / MinHash** – content యొక్క 64-bit fingerprint. Hamming distance ≤ 3 అయితే near-duplicate

BLOOM FILTER ఎందుకు, ఎందుకు అది సరిపోతుంది

10 billion URLs ని hash set lo ఉంచితే 80 GB+ RAM. Bloom filter తో అదే పని ~12 GB lo. ఖరీదు: **false positive** – "ఇది ఇంతకుముందు చూశాం" అని తప్పుగా చెప్పడం, అంటే ఒక కొత్త page ని miss చేయడం. కానీ false *negative* ఎప్పుడూ రాదు.

ఇది ఎందుకు ఆమోదయోగ్యం: "1% pages ని miss చేయడం web crawler కి ఫర్వాలేదు – web అనంతం, మనం ఎలాగూ అంతా crawl చేయలేం. కానీ payment system lo ఇలాంటి approximation ఎప్పటికీ ఆమోదయోగ్యం కాదు. **ఏ problem lo ఏ తప్పు భరించగలమో తెలియడమే** సరైన tool ఎంచుకోవడానికి కీలకం."

7. Crawler Traps & Politeness

ఉచ్చు	ఏమవుతుంది	పరిష్కారం
Infinite calendar	`/calendar?month=1`, `2`, `3`... అనంతం	URL depth limit, ఒక్కో domain కి page quota, pattern detection
Session id in URL	ప్రతిసారి కొత్త URL, ఒకే content	Normalisation lo session params తీసేయడం + content simhash
Redirect loop	A → B → A	Hop limit (5)
భారీ file	2 GB PDF	Content-Length check + download cap
Slow server	Connection ని గంటలు పట్టుకోవడం	Aggressive timeouts (10s connect, 30s read)
robots.txt	Disallow ని పాటించకపోతే block + legal	Domain కి robots.txt ని cache చేసి (24h) ప్రతి fetch ముందు check
Crawl-delay	Site "10 సెకన్లకి ఒకసారి" అంటుంది	Host timing heap lo ఆ delay ని గౌరవించడం

8. Freshness – ఎప్పుడు మళ్ళీ crawl చేయాలి?

ADAPTIVE RE-CRAWL

"అన్ని pages ని ఒకే rate తో re-crawl చేయను. ప్రతి page కి **change frequency** ని track చేస్తాను – గత 10 crawls lo ఎన్నిసార్లు మారింది? News homepage ప్రతిసారీ మారుతుంది → గంటకోసారి. 2011 నాటి blog post ఎప్పుడూ మారలేదు → నెలకోసారి. ఇది **Poisson process estimate** – గత మార్పుల rate నుంచి తర్వాతి మార్పు ఎప్పుడు అని అంచనా. దీంతో అదే bandwidth తో చాలా ఎక్కువ freshness."

9. Scale & Failure

సమస్య	పరిష్కారం
DNS lookup bottleneck	ప్రతి fetch కి DNS query అంటే భారీ ఆలస్యం. Local DNS cache + ముందే resolve చేయడం (prefetch)
Frontier persistence	10 B URLs memory lo పట్టవు. Disk-backed queues (Kafka/RocksDB), memory lo hot భాగం మాత్రమే
Fetcher node crash	URL ని frontier నుంచి తీసేయకుండా lease తీసుకోవడం (Problem 10 లాగానే). Crash అయితే lease expire అయి URL తిరిగి వస్తుంది
Bloom filter ని shard చేయడం	`hash(url) % N` ప్రకారం – ఒక్కో node ఒక భాగాన్ని పట్టుకుంటుంది. అదే shard key తో frontier ని కూడా shard చేస్తే network hop తగ్గుతుంది
ఒకే site నుంచి కోట్ల URLs	Domain quota – ఒక్కో domain నుంచి గరిష్ఠంగా N pages. లేకపోతే ఒక spam site మొత్తం crawler ని మింగేస్తుంది
JS-heavy sites	Headless browser pool – ఇది 50× ఖరీదు. కాబట్టి ముందు plain HTTP fetch, content ఖాళీగా ఉంటే మాత్రమే render పంపడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me scope it: how many pages and over what period, do we need JavaScript rendering, how fresh do results need to be, and do we honour robots.txt? I'll assume a billion pages a month, mostly plain HTML, adaptive freshness, and strict robots compliance."

"A billion a month is about four hundred pages a second – roughly forty megabytes a second of HTML. That's not a lot of throughput. The difficulty in this problem isn't scale, it's everything the web does to you: duplicate content, infinite URL spaces, hostile servers, and the fact that hammering one site is indistinguishable from an attack."

"The heart of the design is the URL frontier, and it has to satisfy two goals that pull against each other. I want to crawl **important** pages first, and I must not hit any single host too fast."

"The standard structure – Mercator's – is two tiers of queues. Front queues hold URLs by priority. Back queues hold URLs by **host**, one queue per host, and each back queue is drained by exactly one fetcher thread. So the prioritiser picks what's important, but the per-host queues guarantee politeness. If I only had priority queues, all my high-priority URLs might belong to the same site and I'd take it down. If I only had host queues, I'd be polite but crawl garbage first."

"I'd pair that with a host timing heap that records the earliest time each host may be contacted again, honouring any crawl-delay in robots.txt."

"There are two completely different deduplication problems here. URL duplicates – trailing slashes, uppercase hosts, tracking parameters – I handle by normalising aggressively and then checking a **bloom filter** of seen URLs. Ten billion URL hashes in a hash set would be eighty gigabytes; a bloom filter does it in about twelve. The cost is false positives, meaning I occasionally skip a page I've never seen. That's acceptable here – the web is infinite and I'll never crawl all of it anyway. I want to flag that explicitly, though: the same approximation would be completely unacceptable in the payment system. Knowing which errors a problem can tolerate is what tells you which tool to reach for."

"Content duplicates are separate – different URLs serving the same article, mirrors, printer-friendly versions. That needs a content fingerprint like simhash, where near-duplicates land within a small Hamming distance."

"Then there are traps. Infinite calendars generating endless URLs, session IDs making every visit a new URL, redirect loops, two-gigabyte PDFs, servers that accept a connection and never respond. I'd defend with depth limits, per-domain page quotas, hop limits, content-length caps, and aggressive timeouts – none of which are clever, but all of which are the difference between a crawler that runs and one that stalls on day two."

"For freshness I wouldn't re-crawl uniformly. I'd track how often each page has actually changed and model it as a Poisson process – a news homepage gets revisited hourly, a decade-old blog post monthly. Same bandwidth, far better index freshness."

10. Follow-ups

అడిగేది	జవాబు
"Priority ఎలా నిర్ణయిస్తారు?"	Inbound links (PageRank-ish), domain authority, గత change frequency, URL depth, sitemap priority. ఇవన్నీ కలిపి ఒక score
"Distributed crawler lo work ని ఎలా పంచుతారు?"	`hash(domain) % N` – **domain ప్రకారం**, URL ప్రకారం కాదు. అలా అయితే ఒక domain యొక్క politeness state ఒకే node lo ఉంటుంది, coordination అవసరం లేదు
"sitemap.xml వాడతారా?"	తప్పకుండా – అది site స్వయంగా ఇచ్చే URL list + lastmod. Discovery చాలా చౌక అవుతుంది
"Crawler ని ఎలా ఆపాలి (kill switch)?"	Config-driven rate limits, domain blocklist – deploy లేకుండా వెంటనే మార్చగలిగేలా. Site owner complain చేస్తే నిమిషాల్లో ఆపగలగాలి
"Deep web / login pages?"	Crawl చేయము. Login అవసరమైతే అది crawl scope బయట (legal + ethical)
"ఇదే design ని site scraper కి వాడొచ్చా?"	అవును కానీ చాలా చిన్నది – ఒకే domain, politeness సులభం, frontier చిన్నది. Bloom filter బదులు సాధారణ set చాలు

PROBLEM 12 · CONCURRENCY & CONVERGENCE

12

Design Collaborative Editing (Google Docs)

Difficulty **చాలా కష్టం** · Frequency **మధ్యస్థం** · నేర్పే concepts: OT, CRDT, convergence, causality

1. The Ask

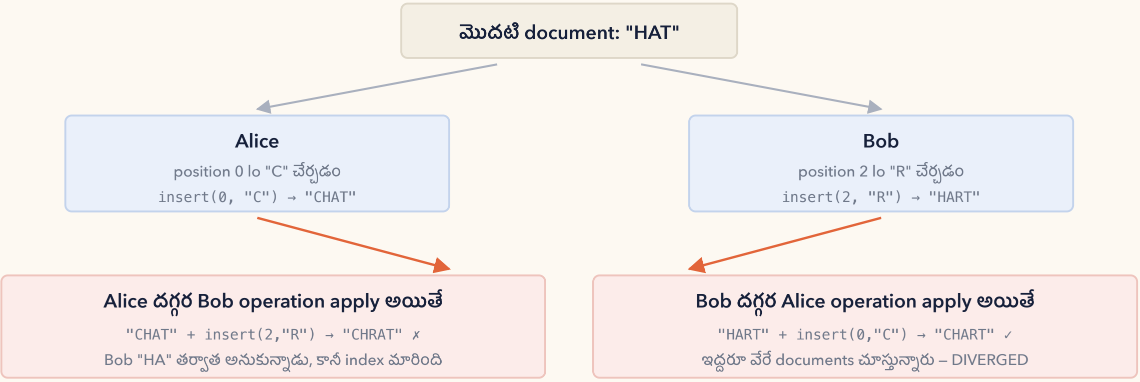
> "Design Google Docs. Ten people edit the same paragraph at the same time, and everyone must end up seeing exactly the same document."

ఈ PROBLEM ఎందుకు ప్రత్యేకం

ఇంతవరకు అన్ని problems lo మనం **scale** ని పరిష్కరించాం. ఇది వేరే – ఇది ఒక **algorithm problem**. 10 మంది users, ఒక్క paragraph – scale ఏమీ లేదు. కానీ ప్రశ్న గణితపరమైనది: *concurrent edits ని ఎలా కలిపితే అందరూ ఒకే ఫలితానికి చేరతారు?* ఇక్కడ మీరు distributed systems theory ని చూపించాలి, boxes కాదు.

2. అసలు సమస్య – ఒక ఉదాహరణ

THE DIVERGENCE PROBLEM · అమాయకమైన పరిష్కారం ఎందుకు విఫలమవుతుంది



ఇదే మొత్తం problem. Operations **ఒకే క్రమంలో** apply అయినా ఫలితం వేరుగా వస్తుంది, ఎందుకంటే **index అనేది relative** – దాని అర్థం document state మీద ఆధారపడి ఉంది. దీన్ని పరిష్కరించడానికే OT మరియు CRDT పుట్టాయి.

అమాయక పరిష్కారం	ఎందుకు విఫలం
Last-write-wins	ఒకరి edit మౌనంగా పోతుంది. Document editor lo ఇది ఆమోదయోగ్యం కాదు
Document lock	ఒకరు edit చేస్తుంటే మిగతావాళ్ళు ఆగాలి – అప్పుడు అది "collaborative" కాదు
Character lock	Lock traffic actual edits కంటే ఎక్కువ. Latency వల్ల typing ఆగిపోతుంది
Server serialises అన్నిటినీ	సరైన దిశ, కానీ ఇంకా సరిపోదు – index అర్థం మారే సమస్య అలాగే ఉంది

3. Deep Dive – Operational Transformation (OT)

కీలక ఆలోచన: Operation ని యథాతథంగా apply చేయకు. దాన్ని ఇప్పటికే apply అయిన concurrent operations కి **వ్యతిరేకంగా transform** చేసి, తర్వాత apply చెయ్యి.

OT · TRANSFORM FUNCTION ఏం చేస్తుంది

Bob యొక్క original op
`insert(2, "R")`

→

transform(op_bob, op_alice)
"Alice నా కంటే ముందు position 0 lo insert చేసింది → నా index ఒకటి పెరగాలి"

→

Transformed op
`insert(3, "R")`

↓

"CHAT" + insert(3,"R") → "CHART" ✓
ఇద్దరూ ఇప్పుడు ఒకే document చూస్తున్నారు

Server అన్ని ops కి ఒక global order ఇస్తుంది
ప్రతి client తన pending ops ని ఆ order కి వ్యతిరేకంగా transform చేసుకుంటాడు

OT కి CENTRAL SERVER తప్పనిసరి – ఆ server ఇచ్చే క్రమమే "సత్యం". Peer-to-peer OT చాలా కష్టం.
TRANSFORM FUNCTIONS ని రాయడం కుఖ్యాతమైన కష్టం – Google Wave ఇక్కడే చాలా bugs తో బాధపడింది.

4. Deep Dive – CRDT (వేరే దారి)

కీలక ఆలోచన: Index ని పూర్తిగా వదిలేయ్. ప్రతి అక్షరానికీ ఒక **శాశ్వతమైన, ప్రత్యేకమైన id** ఇవ్వు – ఆ id ఎప్పటికీ మారదు.

CRDT · POSITION కాదు, IDENTITY

ప్రతి అక్షరం ఒక UNIQUE, ORDERED ID ని పొందుతుంది · ఆ id ఎప్పటికీ మారదు

H	A	T	C – Alice చేర్చింది	R – Bob చేర్చాడు
`id 1.0 · alice`	`id 2.0 · alice`	`id 3.0 · alice`	`id 0.5 · alice`	`id 2.5 · bob`

Document = అన్ని characters ని వాటి ids ప్రకారం sort చేయడం
0.5(C) · 1.0(H) · 2.0(A) · 2.5(R) · 3.0(T) → "CHART" – ఏ క్రమంలో వచ్చినా ఇదే ఫలితం

ఎందుకు ఇది పని చేస్తుంది
Operations commutative – ఏ క్రమంలో apply చేసినా ఒకే ఫలితం. Transform అవసరం లేదు, server అవసరం లేదు.

ఖరీదు
ప్రతి అక్షరానికీ metadata (id + author + clock).
Deleted అక్షరాలని tombstones గా ఉంచాలి – memory పెరుగుతుంది.

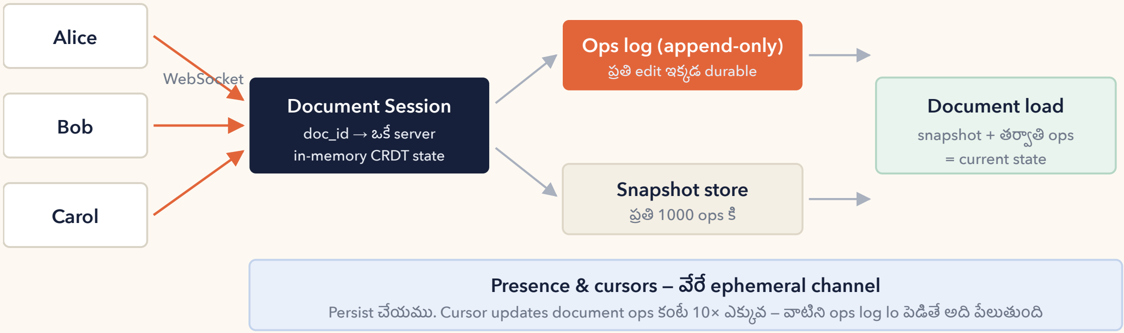
	OT	CRDT
ఆలోచన	Index ని transform చేయడం	Index బదులు unique ids
Server	**తప్పనిసరి** (global order కోసం)	అవసరం లేదు (P2P సాధ్యం)
Memory	తక్కువ – plain text	ఎక్కువ – ప్రతి char కి metadata + tombstones
Implementation	**చాలా కష్టం** – transform functions correctness prove చేయడం కష్టం	సులభం, గణితపరంగా నిరూపితం
Offline support	కష్టం	సహజం
ఎవరు వాడతారు	Google Docs, Etherpad	Figma, Notion, Apple Notes, Automerge/Yjs

నా CHOICE – ఇలా చెప్పండి

"నేను **CRDT** ఎంచుకుంటాను. కారణం correctness – CRDT convergence గణితపరంగా నిరూపితం, OT lo transform functions ప్రతి operation జతకీ సరిగ్గా రాయాలి, అది Google Wave ని ముంచిన సమస్య. CRDT memory ఎక్కువ తింటుంది, కానీ దాన్ని periodic garbage collection మరియు compaction తో నియంత్రించవచ్చు – ఇది engineering problem, correctness problem కాదు. **Google Docs OT వాడుతుంది ఎందుకంటే అది CRDTs practical అవ్వడానికి ముందు రాసినది** – ఈరోజు కొత్తగా మొదలుపెడితే పరిశ్రమ CRDT వైపే మొగ్గుతోంది (Figma, Notion అలానే చేశాయి)."

5. Architecture

DOCUMENT COLLABORATION · RUNTIME + DURABILITY



ఒక document ని **ఒకే server** handle చేయాలి (doc_id ప్రకారం routing) – అలా అయితే merge logic ఒకేచోట, servers మధ్య coordination అవసరం లేదు. Ops log ద్వారా durability, snapshots ద్వారా వేగవంతమైన load.

6. Scale & Failure

సమస్య	పరిష్కారం
Document server crash	Ops log durable కాబట్టి state పోదు. కొత్త server snapshot + తర్వాతి ops నుంచి rebuild. Clients reconnect చేసి తమ unacked ops ని మళ్ళీ పంపుతారు (ops idempotent – ఒకే op id)
CRDT memory పెరగడం	Tombstones ని GC చేయడం – అందరు clients ఆ point దాటాక (causal stability) tombstone ని తీసేయొచ్చు. పైగా consecutive characters ని ఒక "block" గా merge చేయడం (RGA optimisation)
Ops log అనంతంగా పెరగడం	Snapshot + snapshot కంటే పాత ops ని archive. Version history కావాలంటే వాటిని cold storage lo ఉంచడం
పెద్ద document (100 pages)	Document ని sections/blocks గా విడగొట్టడం – Notion ఇలానే చేస్తుంది. అప్పుడు merge unit చిన్నది, memory తక్కువ
Offline editing	CRDT యొక్క సహజ బలం – client offline lo ops పోగేసి, తిరిగి వచ్చాక పంపుతాడు. Commutative కాబట్టి merge తనంతట తానే జరుగుతుంది
చాలా మంది ఒకే doc lo (100+)	Ops broadcast $O(n^2)$ అవుతుంది. Ops ని batch చేసి 50 ms కి ఒకసారి పంపడం. చాలా మంది కేవలం చూస్తుంటారు – వాళ్ళని read-only channel కి పంపడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"I want to frame this differently from the previous problems. This isn't primarily a scale problem – ten people editing one paragraph is nothing. It's a **convergence** problem: how do concurrent edits merge so that everyone ends up with an identical document?"

"Let me show why it's hard with the smallest possible example. The document says 'HAT'. Alice inserts 'C' at position zero. Concurrently Bob inserts 'R' at position two. If Alice applies Bob's operation literally, her 'CHAT' becomes 'CHRAT' – because her insert shifted every index. Bob applying Alice's operation gets 'CHART'. They've **diverged**, and no amount of ordering fixes it, because an index is a relative reference whose meaning depends on the document state."

"Last-write-wins silently loses someone's typing. Locking, at any granularity, destroys the feel of collaboration. So we need a real algorithm, and there are two families."

"**Operational Transformation** keeps indices but transforms each operation against the concurrent operations already applied. Bob's insert-at-2 becomes insert-at-3 once we account for Alice's earlier insert. It requires a central server to impose a global order, and its weakness is that you must write a correct transform function for every pair of operation types. That is notoriously difficult – it's a large part of why Google Wave struggled."

"**CRDTs** take the opposite route: abandon indices entirely. Every character gets a permanent unique identifier that sorts, and the document is just all characters sorted by ID. Operations become commutative – apply them in any order and you converge. No transform, and in principle no server."

"I'd choose CRDTs, primarily for correctness: convergence is provable, whereas OT correctness depends on hand-written transform functions. CRDTs cost more memory – every character carries metadata, and deletions leave tombstones – but that's an engineering problem I can attack with garbage collection once changes are causally stable, and by merging runs of characters into blocks. Google Docs uses OT largely because it predates practical CRDT implementations; newer products like Figma and Notion went the CRDT route."

"Architecturally, each document is pinned to a single server by document ID, so merge logic lives in one place. Every operation appends to a durable ops log, and I snapshot every thousand operations so loading a document is snapshot-plus-tail rather than replaying everything from creation."

"One detail I'd insist on: cursors and presence go on a **separate ephemeral channel** and are never persisted. Cursor movements outnumber real edits by an order of magnitude, and putting them in the ops log would bloat it for data nobody ever needs again."

"If a document server crashes, nothing is lost – the ops log is durable, clients reconnect and resend anything unacknowledged, and operations are idempotent by ID."

7. Follow-ups

అడిగేది	జవాబు
"Version history / undo ఎలా?"	Ops log ఇప్పటికే పూర్తి చరిత్ర. Undo అంటే **inverse operation** ని కొత్త op గా apply చేయడం (log ని వెనక్కి తిప్పడం కాదు – వేరేవాళ్ళ edits పోతాయి)
"Comments, suggestions?"	అవి text కి **anchored** – CRDT ids తో anchor చేస్తే text మారినా comment సరైన చోట ఉంటుంది. Index తో anchor చేస్తే అది తప్పు చోటికి జారుతుంది
"Rich text (bold, headings)?"	CRDT ని plain text కి కాకుండా **tree/block structure** కి extend చేయడం. Formatting ని range annotations గా (start id, end id, attribute)
"Permissions?"	Document స్థాయిలో ACL, ops apply చేయడానికి ముందు check. Read-only users కి ops పంపుతాం కానీ వాళ్ళవి తీసుకోము
"OT ని ఎప్పుడు ఎంచుకుంటారు?"	Memory చాలా కీలకం అయినప్పుడు, central server ఎలాగూ ఉన్నప్పుడు, మరియు operation types చాలా తక్కువగా ఉన్నప్పుడు (plain text insert/delete మాత్రమే)
"Google Sheets కి ఇదే పని చేస్తుందా?"	కాదు – cells స్వతంత్రం కాబట్టి చాలా సులభం. Cell స్థాయిలో last-write-wins చాలు. కానీ row insert/delete అనేవి మళ్ళీ index problem – వాటికి మాత్రం OT/CRDT ఆలోచన కావాలి

PROBLEM 13 · INFRASTRUCTURE

13

Design a Distributed Message Queue (Kafka)

Difficulty **Hard** · Frequency **SSE/SDE3 lo ఎక్కువ** · నేర్పే concepts: append-only log, partitions, ISR, delivery semantics

1. The Ask

> "Design Kafka. A durable, replayable message queue that handles a million messages a second."

ఇంతవరకు మనం Kafka ని **వాడాం** (problems 03, 05, 17). ఇప్పుడు దాన్ని **నిర్మించాలి**. Senior interviews lo ఇది తరచుగా వస్తుంది, ఎందుకంటే ఇది మీరు abstraction లోపల చూడగలరా అని test చేస్తుంది.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Ordering ఎంత అవసరం?	Global ordering అంటే ఒకే partition – parallelism సున్నా	Per-key ordering చాలు
Delivery semantics?	At-most / at-least / exactly-once – పూర్తిగా వేరే complexity	At-least-once default, exactly-once option
Retention ఎంత?	Consume తర్వాత delete నా, time-based నా	Time-based (7 రోజులు) – replay కావాలి
Consumers ఎన్ని groups?	Pub-sub నా queue నా – Kafka రెండూ చేస్తుంది	రెండూ
Message size?	పెద్దవి అయితే వేరే tuning	~1 KB
Throughput?	ఇది design ని నిర్ణయిస్తుంది	1 M msg/sec

3. Deep Dive – Core Insight: Append-Only Log

సాధారణ queue (RabbitMQ): message ని consume చేశాక delete. Random deletes అంటే random disk I/O.

Kafka: ఏమీ delete చేయదు. కేవలం **append**. Consumers తమ **offset** ని గుర్తుంచుకుంటారు.

APPEND-ONLY LOG · ఒకే డేటా, పలువురు స్వతంత్ర READERS

PARTITION 0 · ఒక append-only file

ఈ ఒక్క నిర్ణయం నుంచి వచ్చే మూడు లాభాలు

1 · Sequential disk write – HDD మీద కూడా 100+ MB/s. Random write కంటే 100 రెట్లు వేగం.
2 · Replay ఉచితం – offset ని వెనక్కి పెట్టి మళ్ళీ చదవడం. Bug fix చేశాక 3 రోజుల data ని reprocess చేయొచ్చు.
3 · పలువురు consumers ఒకే data ని స్వతంత్రంగా – broker వాళ్ళ state ని maintain చేయదు, offset వాళ్ళదే.

Broker ఏ consumer ఏం చదివాడో track చేయదు. అది consumer బాధ్యత. ఈ ఒక్క మార్పు వల్ల broker దాదాపు stateless అవుతుంది – అందుకే ఇది ఇంత scale అవుతుంది.

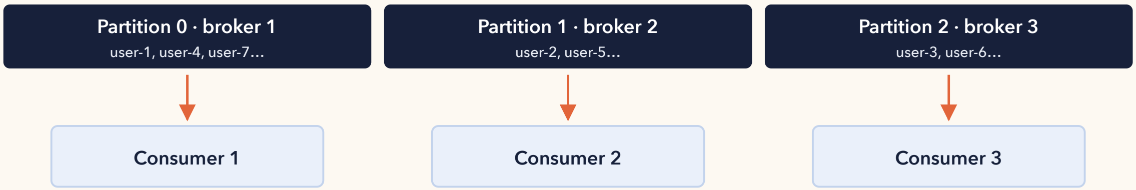
SEQUENTIAL I/O ఎందుకు ఇంత వేగం – ఈ DETAIL చెప్తే మీరు వేరే LEVEL

Disk random seek ≈ 10 ms. Sequential read/write కి seek లేదు. పైగా Kafka **OS page cache** ని నమ్ముకుంటుంది – తన సొంత cache రాయదు. కొత్తగా రాసిన messages ఇంకా RAM lo ఉంటాయి, consumers వెంటనే చదివితే disk ని ముట్టుకోరు. అలాగే **zero-copy** (`sendfile` syscall) – data ని page cache నుంచి నేరుగా network socket కి, user space గుండా కాకుండా. ఈ మూడు కలిపి 1 GB/sec ని సాధారణ hardware మీద సాధ్యం చేస్తాయి.

4. Deep Dive – Partitions (parallelism vs ordering)

PARTITIONS & CONSUMER GROUPS · ఇక్కడే TRADE-OFF ఉంది

TOPIC "orders" · 3 PARTITIONS · hash(key) % 3



ఒక consumer group lo – ఒక్కో partition ని ఒకే consumer చదువుతాడు

లాభం

Partitions పెంచితే consumers పెంచొచ్చు – throughput linear గా పెరుగుతుంది.
ఒకే key ఎప్పుడూ ఒకే partition → per-key order భద్రం.

ఖరీదు

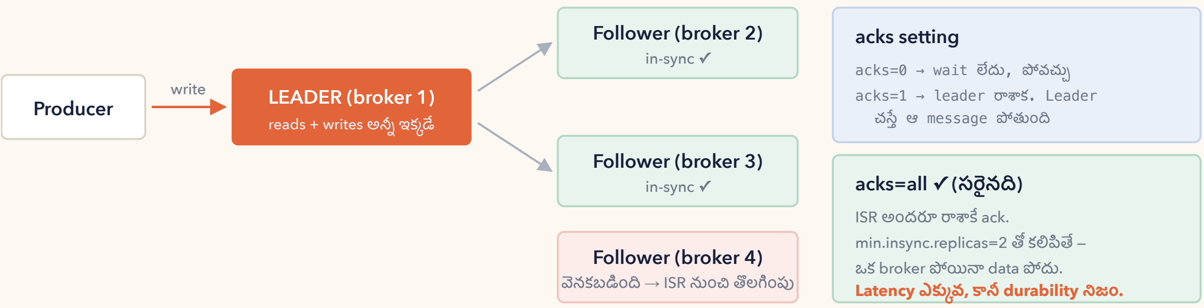
Partitions మధ్య ordering లేదు. Global order కావాలంటే ఒకే partition – అంటే ఒకే consumer, parallelism సున్నా. Consumers > partitions అయితే మిగతావాళ్ళు ఖాళీ.

ఈ TRADE-OFF ని ఇలా చెప్పండి

"Partition అనేది **parallelism యొక్క unit మరియు ordering యొక్క unit – రెండూ ఒకటే.** ఇది యాదృచ్ఛికం కాదు, ఉద్దేశపూర్వకం. అందుకే partition key ఎంచుకోవడం అతి ముఖ్యమైన నిర్ణయం: `user_id` ఇస్తే ఒక user యొక్క events క్రమంలో ఉంటాయి, వేరే users సమాంతరంగా వెళ్తాయి. అదే `order_id` ఇస్తే ఒక user యొక్క రెండు orders తప్పు క్రమంలో process అవ్వొచ్చు. **ఏ స్థాయిలో order కావాలో అదే partition key.**"

5. Deep Dive – Replication & Durability

LEADER, FOLLOWERS, ISR



ISR (In-Sync Replicas) = leader తో వేగంగా నడుస్తున్న followers. వెనకబడినవాడిని ISR నుంచి తీసేస్తారు – లేకపోతే ఒక నెమ్మది broker మొత్తం cluster ని ఆపేస్తుంది. Leader చస్తే **ISR లోపలి నుంచే** కొత్త leader – ఇది data loss ని ఆపుతుంది.

6. Deep Dive – Delivery Semantics

Semantics	ఎలా	ఖరీదు
At-most-once	Process చేయకముందే offset commit. Crash అయితే message పోతుంది	వేగం, కానీ data loss
At-least-once ✅	Process చేశాకే offset commit. Crash అయితే duplicate	Default. Consumer idempotent గా ఉండాలి
Exactly-once	Idempotent producer (sequence number per producer) + transactional writes (offset commit మరియు output write ఒకే transaction lo)	Throughput ~20-30% తగ్గుతుంది

KAFKA యొక్క "EXACTLY-ONCE" నిజంగా ఏమిటి

Kafka exactly-once **Kafka లోపల మాత్రమే** పని చేస్తుంది – "read from topic A, process, write to topic B" అనే pattern కి. ఆ మూడింటినీ ఒకే transaction lo చేస్తుంది. కానీ మీరు **బయటి system** కి రాస్తుంటే (ఒక REST API call, ఒక email) – అక్కడ exactly-once లేదు, మళ్ళీ idempotency మీద ఆధారపడాలి. ఈ సూక్ష్మ భేదాన్ని చెప్తే మీరు దీన్ని నిజంగా వాడారని అర్థమవుతుంది.

7. Scale & Failure

సమస్య	పరిష్కారం
Broker crash	Controller ఆ broker leader గా ఉన్న partitions కి ISR నుంచి కొత్త leaders ఎంచుకుంటుంది. Producers/consumers metadata refresh చేసి కొత్త leader కి వెళ్తారు
Consumer crash	Group coordinator heartbeat miss ని గుర్తించి **rebalance** – ఆ partitions మిగతా consumers కి. కొత్త consumer చివరి committed offset నుంచి మొదలు

సమస్య	పరిష్కారం
Rebalance storm	ప్రతి rebalance lo మొత్తం group ఆగుతుంది. **Cooperative/incremental rebalancing** – ప్రభావితమైన partitions ని మాత్రమే కదిలించడం
Consumer lag	Lag = latest offset − consumer offset. ఇదే ముఖ్యమైన health metric. Lag పెరిగితే consumers ని scale చేయడం (partitions కంటే ఎక్కువ కాదు)
Hot partition	ఒక key చాలా ఎక్కువ traffic. Key కి suffix కలిపి spread చేయడం – కానీ అప్పుడు ఆ key యొక్క ordering పోతుంది. Explicit trade-off
Disk నిండటం	Retention policy – time-based (7 రోజులు) లేదా size-based. **Log compaction** – key ప్రకారం చివరి value ని మాత్రం ఉంచడం (changelog topics కి)
Partition సంఖ్య తప్పు	పెంచొచ్చు కానీ తగ్గించలేం. పైగా పెంచితే `hash(key) % n` మారి existing keys వేరే partitions కి వెళ్తాయి – ordering విరుగుతుంది. అందుకే **ఉదారంగా ముందే ఎక్కువ partitions** పెట్టడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Key questions first: what ordering guarantee do we owe, what delivery semantics, do we retain after consumption, and what throughput? I'll assume per-key ordering, at-least-once by default, seven days of retention so consumers can replay, and a million messages a second."

"The central design decision is that this is an **append-only log**, not a traditional queue. A traditional broker deletes a message once it's consumed, which means random deletes and random disk I/O, and it has to track per-consumer state. I do neither. Messages are appended and retained for a fixed window; consumers track their own **offset**."

"Three things fall out of that. Writes are purely sequential, which is around a hundred times faster than random I/O. Replay is free – move an offset backwards and reprocess. And multiple independent consumer groups can read the same data without the broker tracking anything about them, which is what keeps brokers close to stateless."

"For raw speed I'd lean on the operating system rather than fight it: no application-level cache, just the OS page cache, so recently written messages are served from memory. And **zero-copy** via sendfile, so data goes from page cache straight to the network socket without being copied through user space."

"Parallelism comes from partitions. A topic is split into partitions by hashing the message key, and within a consumer group each partition is read by exactly one consumer. The thing I'd emphasise is that **the partition is simultaneously the unit of parallelism and the unit of ordering** – that's deliberate, not accidental. There's no ordering across partitions. If you want global ordering you need a single partition, which means a single consumer and no parallelism at all. So choosing the partition key is the most consequential decision: key by user ID and one user's events stay ordered while different users run in parallel."

"For durability, each partition has a leader and followers, and Kafka tracks the set of **in-sync replicas**. A follower that falls behind is ejected from that set, so one slow broker can't stall the cluster. Producers should use acks=all with min.insync.replicas of two – that costs latency, but it's the only setting where a broker failure genuinely doesn't lose data. Leader election only ever picks from the in-sync set, which is what makes that guarantee hold."

"On semantics, at-least-once is the sane default – commit the offset after processing, and accept that a crash means a duplicate, which the consumer handles idempotently. Kafka does offer exactly-once via idempotent producers and transactions, but I'd be precise about its scope: it applies to consume-transform-produce **within Kafka**. The moment you write to an external system – call an API, send an email – you're back to at-least-once plus idempotency."

"Two operational things I'd call out. Consumer lag is the health metric that matters, and rebalances are the thing that hurts – every rebalance stops the whole group, so I'd use cooperative rebalancing to move only the affected partitions. And I'd over-provision partitions from the start, because you can add them but never remove them, and adding them reshuffles the hash and breaks per-key ordering for existing keys."

8. Follow-ups

అడిగేది	జవాబు
"ZooKeeper ఎందుకు వాడేవారు, ఇప్పుడు?"	Metadata, controller election, ISR state కి. Kafka 3.x నుంచి **KRaft** – Raft consensus ని Kafka లోపలే, ZooKeeper అవసరం లేదు. ఒక తక్కువ system operate చేయాలి
"Kafka vs RabbitMQ ఎప్పుడు?"	**Kafka**: high throughput, replay కావాలి, event streaming, log. **RabbitMQ**: complex routing, per-message ack/nack, priority queues, తక్కువ latency, చిన్న scale
"Log compaction అంటే?"	ప్రతి key కి చివరి value ని మాత్రం ఉంచి మిగతావి తీసేయడం. దీంతో topic ఒక "current state snapshot" అవుతుంది – CDC, config distribution కి పనికొస్తుంది
"Consumer partitions కంటే ఎక్కువ ఉంటే?"	అదనపు consumers **ఖాళీగా** ఉంటారు. Partitions సంఖ్యే గరిష్ఠ parallelism
"Message ordering ni ఎలా guarantee చేస్తారు retry lo?"	`max.in.flight.requests=1` లేదా idempotent producer. లేకపోతే – msg1 fail అయి retry అవుతుంటే msg2 ముందే వెళ్ళిపోతుంది, order విరుగుతుంది
"Tiered storage?"	పాత segments ని S3 కి తరలించి, brokers lo ఇటీవలివి మాత్రం. దీంతో retention ని నెలలు పొడిగించొచ్చు – storage cost తక్కువ, replay ఇంకా సాధ్యం

14

Design a Distributed Key-Value Store (Dynamo)

Difficulty **చాలా కష్టం** · Frequency **SDE3 lo ఎక్కువ** · నేర్పే concepts: consistent hashing, quorum, vector clocks, LSM tree

1. The Ask

> "Design DynamoDB. A key-value store that scales horizontally, survives node failures, and never goes down."

ఇది **అన్ని problems కి తల్లి**. Cassandra, DynamoDB, Riak – అన్నీ ఇదే paper (Amazon Dynamo, 2007) నుంచి. ఇక్కడ నేర్చుకునే ఐదు ఆలోచనలు మిగతా అన్ని distributed systems lo తిరిగి కనిపిస్తాయి.

2. Clarifying Questions

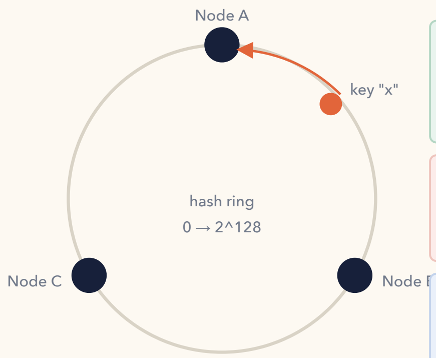
ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
API ఏమిటి?	Range queries కావాలంటే design పూర్తిగా మారుతుంది	`get(key)`, `put(key, value)` మాత్రమే
Consistency – strong నా eventual నా?	ఇదే CAP నిర్ణయం	Tunable (quorum తో)
Availability ఎంత ముఖ్యం?	Dynamo యొక్క మూల లక్ష్యం: **ఎప్పుడూ write ని తిరస్కరించకూడదు**	"Always writeable"
Value size?	పెద్దవి అయితే వేరే storage	≤ 1 MB
Transactions కావాలా?	Multi-key transactions అంటే వేరే system	లేదు, single-key మాత్రం

DYNAMO యొక్క మూల తత్వం

Amazon shopping cart కోసం ఇది రాశారు. వాళ్ళ నియమం: **"Add to cart" ఎప్పుడూ విఫలం కాకూడదు** – network partition అయినా, nodes చచ్చినా. అందుకే వాళ్ళు **CAP lo C ని వదిలేసి AP ఎంచుకున్నారు**. Conflict వస్తే? రెండు versions ని ఉంచి, application ని resolve చేయమనడం (cart కి అది సులభం – రెండు carts ని union చేయడం). ఈ context ని చెప్తే మీ మిగతా design అంతా అర్థవంతంగా మారుతుంది.

3. Deep Dive – Consistent Hashing

CONSISTENT HASHING RING · NODE చేరినా, పోయినా – 1/N KEYS మాత్రమే కదులుతాయి



నియమం ఒక్కటే

key ని hash చేసి ring మీద పెట్టు. అక్కడి నుంచి **సవ్య దిశలో** మొదట కనిపించే node ఆ key కి యజమాని. తర్వాతి N−1 nodes replicas.

సాధారణ hash తో పోలిస్తే

hash(key) % N lo N మారితే **దాదాపు అన్ని keys** కదులుతాయి – అంటే ఒక node చేర్చడానికి మొత్తం cluster ని rebalance చేయాలి.

Virtual nodes – ఎందుకు తప్పనిసరి

Physical node ని ring మీద 100+ చోట్ల పెట్టడం. లేకపోతే మూడు nodes అసమానంగా పంచుకుంటాయి, పైగా ఒకటి పోతే దాని భారం మొత్తం ఒక్క పొరుగువాడి మీద పడుతుంది.

4. Deep Dive – Quorum (tunable consistency)

N = replicas సంఖ్య · **W** = write కి ఎన్ని nodes ack ఇవ్వాలి · **R** = read కి ఎన్ని nodes నుంచి చదవాలి

R + W > N అయితే ఎందుకు తాజా DATA దొరుకుతుంది

N = 3 · W = 2 · R = 2 · R + W = 4 > 3 ✓

Replica 1	Replica 2	Replica 3
v2 (కొత్తది)	v2 (కొత్తది)	v1 (పాతది)

Write v2 · rep 1, 2 ack ఇచ్చాయి (W=2 ✓)
Replica 3 ఇంకా చేరుకోలేదు

↓

Read ఏవైనా 2 replicas నుంచి (R=2)
3 lo ఏ 2 తీసుకున్నా – కనీసం ఒకటి v2 ఉన్నదే ఉంటుంది

ఇదే pigeonhole principle
W nodes రాశాయి, R nodes చదువుతున్నాయి, R+W > N → overlap తప్పనిసరి

W=1, R=1 → చాలా వేగం, కానీ పాత data రావచ్చు (R+W=2, N=3 కంటే తక్కువ)
W=3, R=1 → reads వేగం, writes నెమ్మది + ఒక node పోతే write ఆగిపోతుంది. Read-heavy కి మంచిది

ఇదే **tunable consistency** – ఒక్కో query కీ R, W ని మార్చొచ్చు. User profile చదవడానికి R=1 (వేగం), account balance కి R=2 (correctness). ఒకే database lo రెండు రకాల guarantees.

5. Deep Dive – Conflicts (vector clocks vs LWW)

W=2 తో ఇద్దరు clients ఒకే key కి ఒకేసారి రాశారు, network partition వల్ల ఒకరికొకరు తెలియదు. ఇప్పుడు రెండు versions.

పరిష్కారం	ఎలా	ఖరీదు
Last-write-wins (timestamp)	పెద్ద timestamp గెలుస్తుంది	❌ **డేటా మౌనంగా పోతుంది.** పైగా clock skew వల్ల "పాతది" గెలవచ్చు. Cassandra default – సులభం కానీ ప్రమాదకరం
Vector clocks	ప్రతి version తో `{nodeA: 2, nodeB: 1}` లాంటి causality metadata. దీంతో "ఇది ఆ దాని తర్వాతిదా, లేక సమాంతరమా" అని ఖచ్చితంగా చెప్పొచ్చు	✅ Causality ని నిజంగా tracks చేస్తుంది. కానీ concurrent అయితే **application** resolve చేయాలి

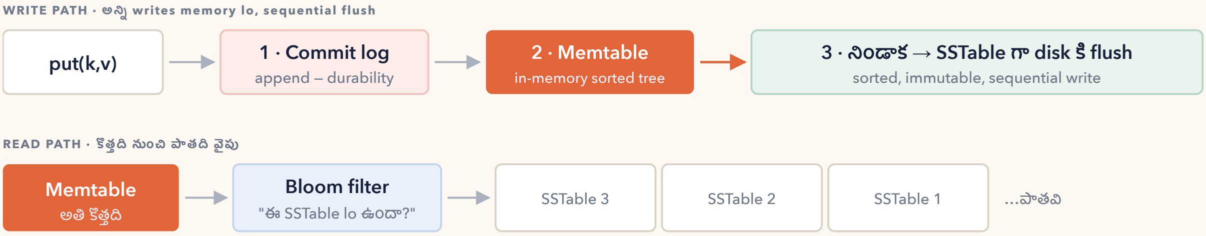
పరిష్కారం	ఎలా	ఖరీదు
CRDT values	Value నే commutative structure గా (counter, set)	✅ Automatic merge, కానీ అన్ని data types కి కుదరదు

ఇలా చెప్పండి

"Vector clock మనకి **జవాబు ఇవ్వదు** – అది ఒక ప్రశ్నకి మాత్రమే జవాబిస్తుంది: 'ఈ రెండు versions lo ఒకటి రెండోదాని తర్వాత వచ్చిందా, లేక అవి నిజంగా concurrent నా?' Concurrent అయితే database వాటిని **siblings** గా రెండింటినీ ఉంచి, తర్వాతి read lo application కి ఇస్తుంది. Shopping cart కి ఇది సులభం – రెండు carts ని union చేయడం. కానీ ప్రతి application ఈ బాధ్యతని తీసుకోలేదు, అందుకే చాలా systems LWW కే వెళ్తాయి – **data loss ని తెలిసి అంగీకరిస్తూ**."

6. Deep Dive – Storage Engine (LSM Tree)

LSM TREE · WRITE PATH మరియు READ PATH



ఎందుకు LSM (B-tree కాదు)

అన్ని writes sequential (log + flush). B-tree lo ప్రతి write ఒక random page update – **write-heavy loads కి LSM చాలా వేగం.**

ఖరీదు · read amplification

ఒక key కోసం పలు SSTables చూడాలి. అందుకే bloom filter (ఇక్కడ లేదని O(1) lo చెప్పడం) + background compaction (SSTables ని merge చేయడం).

7. Failure Handling – మూడు యంత్రాంగాలు

యంత్రాంగం	ఎప్పుడు	ఏం చేస్తుంది
Gossip	ఎప్పుడూ	ప్రతి node కొన్ని random peers తో state ని పంచుకుంటుంది. కొన్ని సెకన్లలో "ఎవరు బతికున్నారు" అనేది cluster అంతా వ్యాపిస్తుంది. Central failure detector అవసరం లేదు (అది SPOF అవుతుంది)
Hinted handoff	Node తాత్కాలికంగా down	Node C కి రాయాల్సిన write ని Node D **hint** గా పట్టుకుంటుంది. C తిరిగి రాగానే D దాన్ని అందిస్తుంది. **అందుకే write ఎప్పుడూ విఫలం కాదు** – Dynamo యొక్క మూల వాగ్దానం ఇక్కడ నెరవేరుతుంది
Merkle tree anti-entropy	Node చాలాసేపు down	రెండు replicas తమ data యొక్క hash tree ని పోల్చుకుంటాయి. మొత్తం data పంపకుండా, **తేడా ఉన్న ranges** ని మాత్రమే గుర్తించి sync చేయడం. Root hash ఒకటే అయితే పని అయిపోయింది

READ REPAIR – నాలుగోది, ఉచితమైనది

Read lo R replicas నుంచి జవాబులు వచ్చాయి, ఒకటి పాతది. ఆ క్షణంలోనే దాన్ని update చేసేయడం. అంటే **ఎక్కువగా చదివే data దానంతట అదే సరిచేసుకుంటుంది** – అదనపు ఖర్చు దాదాపు లేకుండా. తక్కువగా చదివే data కి anti-entropy ఉంది.

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me establish the philosophy first, because it drives every later decision. Dynamo was built for Amazon's shopping cart, with one non-negotiable rule: **a write must never be rejected**, even during a network partition. That means choosing availability over consistency in CAP terms, and accepting that we'll sometimes have conflicting versions to reconcile."

"Data placement uses **consistent hashing**. Keys and nodes both hash onto a ring; a key belongs to the first node clockwise from it, and the next N−1 nodes hold replicas. The reason to prefer this over hash-modulo-N is that adding or removing a node moves only about one Nth of the keys instead of nearly all of them. I'd add virtual nodes – each physical machine appears at a hundred-plus points on the ring – otherwise load is uneven, and when a node dies its entire share lands on a single neighbour."

"Consistency is **tunable** through quorums. With N replicas, W acknowledgements on write and R on read, if R plus W exceeds N then the read and write sets must overlap by the pigeonhole principle, so a read always sees the latest write. N=3, W=2, R=2 is the usual balance. But the caller can choose per query – R=1 for a profile fetch where speed matters, higher where correctness does."

"Because we allow writes during partitions, conflicts are inevitable. Last-write-wins by timestamp is simple but **silently discards data**, and clock skew means the wrong version can win. Dynamo instead uses **vector clocks**. It's worth being precise about what they do: a vector clock doesn't resolve conflicts, it only tells you whether one version causally descends from another or whether they're genuinely concurrent. If they're concurrent, both are kept as siblings and handed to the application on the next read. For a shopping cart that's easy – union the two carts. Not every application can shoulder that, which is why many systems fall back to last-write-wins and knowingly accept the data loss."

"For storage I'd use an **LSM tree** rather than a B-tree. Writes go to a commit log for durability and an in-memory sorted table, which is flushed to immutable sorted files on disk. Every write is sequential, which is what makes this fast for write-heavy workloads – a B-tree does a random page update per write. The cost is read amplification, since a key might live in any of several files. Bloom filters fix most of that by telling us in constant time that a key is definitely **not** in a given file, and background compaction merges files to keep the count down."

"Failure handling has three layers. **Gossip** spreads membership and liveness without a central detector, which would itself be a single point of failure. **Hinted handoff** is what actually delivers the 'always writeable' promise – if a replica is down, a neighbour accepts the write as a hint and forwards it when the node returns. And **Merkle trees** handle longer outages: two replicas compare hash trees to find exactly which key ranges differ, so they sync only the divergent data rather than everything. Read repair adds a fourth, nearly free layer – whenever a read reveals a stale replica, we fix it on the spot."

8. Follow-ups

అడిగేది	జవాబు
"Range queries ఎలా (`WHERE id BETWEEN`)?"	Consistent hashing తో సాధ్యం కాదు – hash order, key order ని నాశనం చేస్తుంది. **Range partitioning** కావాలి (HBase, Bigtable అలా చేస్తాయి). ఖరీదు: hotspot ప్రమాదం, ఎందుకంటే వరుస keys ఒకే node మీద
"Secondary indexes?"	Local index (ప్రతి node తన data కి – read lo అన్ని nodes ని అడగాలి) లేదా global index (వేరే table గా, కానీ అప్పుడు cross-partition consistency సమస్య)
"Cassandra vs DynamoDB తేడా?"	ఒకే Dynamo వంశం. Cassandra: self-hosted, tunable, wide-column, LWW default. DynamoDB: managed, single-digit ms SLA, strongly consistent read option కూడా ఇస్తుంది
"Strong consistency కూడా కావాలంటే?"	R=N, W=N (అప్పుడు availability పోతుంది), లేదా consensus (Raft/Paxos) per partition – అది Spanner/CockroachDB దారి. Latency ఎక్కువ, కానీ నిజమైన linearizability
"Hot partition ఎలా handle చేస్తారు?"	Adaptive capacity (DynamoDB), లేదా key కి random suffix కలిపి spread. కానీ అప్పుడు ఆ key ని చదవడానికి అన్ని suffixes ని అడగాలి
"Compaction strategy?"	**Size-tiered** – write-heavy కి మంచిది, కానీ space amplification ఎక్కువ. **Leveled** – read-heavy కి మంచిది, కానీ ఎక్కువ write amplification. Workload ని బట్టి ఎంచుకోవడం

PROBLEM 15 · CONTENT ADDRESSING

15

Design File Sync (Dropbox / Google Drive)

Difficulty **Hard** · Frequency **మధ్యస్థం** · నేర్పే concepts: chunking, content-defined boundaries, dedupe, delta sync

1. The Ask

> "Design Dropbox. A file changes on my laptop and appears on my phone within seconds — without re-uploading the whole file."

ఈ PROBLEM యొక్క ఒకే ఒక్క కీలక ప్రశ్న

"నేను ఒక 1 GB video file lo చివరన 5 సెకన్లు కత్తిరించాను. ఇప్పుడు ఎంత data upload కావాలి?" — **1 GB అని జవాబిస్తే** మీరు ఈ problem ని అర్థం చేసుకోలేదు. సరైన జవాబు: **కొన్ని MB మాత్రమే**. దీన్ని ఎలా సాధించాలో చెప్పడమే ఈ interview.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
File పరిమాణం గరిష్ఠం?	Chunking strategy	10 GB దాకా
Version history కావాలా?	ఉంటే పాత chunks ని ఉంచాలి	30 రోజులు
Sharing / collaboration?	Permission model + notification fan-out	అవును
Offline editing?	Conflict resolution తప్పనిసరి	అవును
Sync latency?	Push నా poll నా	కొన్ని సెకన్లు
ఒకే file ని ఇద్దరు edit చేస్తే?	Conflict policy	రెండు copies ఉంచడం

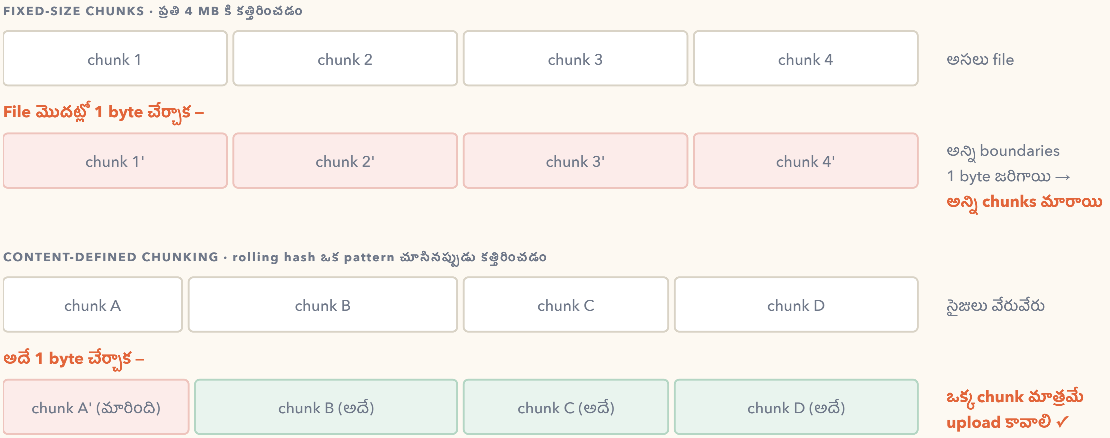
3. Estimation

లెక్క	విలువ
Users	100 M (DAU 20 M)
ఒక user సగటు storage	50 GB
Raw storage	100M × 50 GB = **5 EB** — dedupe లేకుండా అసాధ్యం

లెక్క	విలువ
Dedupe తర్వాత (సాధారణంగా 30-50% ఆదా)	≈ 2.5-3 EB
రోజుకి మారే files / user	10
Metadata QPS	20M × 10 ÷ 86,400 ≈ 2,300 / sec (peak ~10K)
Chunk size	4 MB
Upload bandwidth	Delta sync తో నిజమైన traffic చాలా తక్కువ

4. Deep Dive – Chunking (ఇదే అసలు కీలకం)

FIXED-SIZE VS CONTENT-DEFINED CHUNKING · ఎందుకు తేడా అపారమైనది



Rolling hash (Rabin fingerprint) – file మీద ఒక window ని జరుపుతూ, hash ఒక నిర్దిష్ట pattern (ఉదా. చివరి 13 bits సున్నా) చూపినప్పుడు అక్కడ కత్తిరించడం. Boundary అనేది *content* మీద ఆధారపడుతుంది, position మీద కాదు – అందుకే content జరిగినా boundaries జరగవు.

> **ఈ ఒక్క IDEA ని బాగా చెప్పండి**
>
> "Chunk boundary ని **position తో కాకుండా content తో** నిర్ణయించడం – ఇదే మొత్తం system ని సాధ్యం చేసే idea. Fixed chunks తో file మొదట్లో ఒక్క byte చేర్చితే మొత్తం file మళ్ళీ upload కావాలి. Content-defined chunking తో ఒక్క chunk మాత్రమే. ఇదే idea rsync, git, Docker layers, backup systems – అన్నిచోట్లా ఉంది."

5. Deep Dive – Content Addressing & Dedupe

```
chunk_id = SHA-256(chunk_bytes)     # content యే దాని పేరు

file "video.mp4" = [
  "a3f9…",  "7b21…",  "c04e…",  "f8d2…"     # chunk hashes క్రమంలో
]
```

స్థాయి	ఎలా	ఆదా
Per-user dedupe	ఒక user రెండు folders lo ఒకే file – ఒక్కసారే store	మధ్యస్థం
Global dedupe	ప్రపంచంలో ఎవరైనా ఆ chunk upload చేసి ఉంటే మళ్ళీ వద్దు	భారీ (30-50%) – కానీ ⚠️ చూడండి

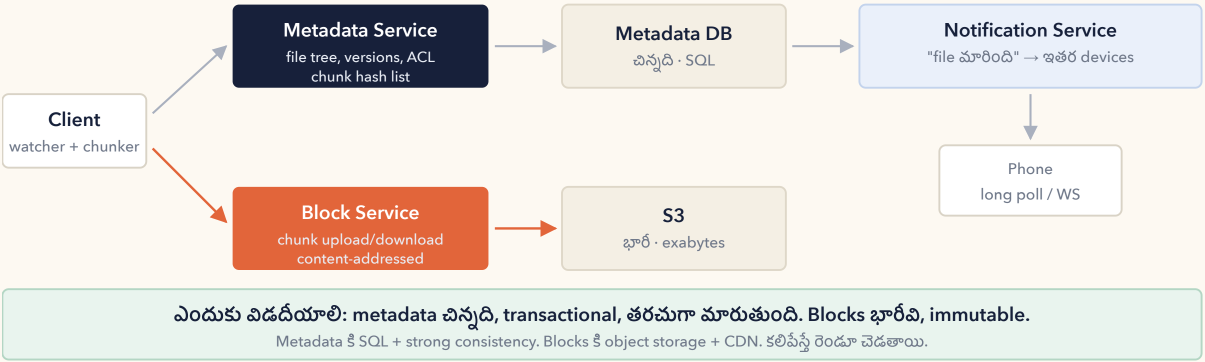
GLOBAL DEDUPE యొక్క దాచిన SECURITY BUG

Client "ఈ hash ఇప్పటికే ఉందా?" అని అడిగి, ఉంటే upload skip చేస్తే – దాడి చేసేవాడు ఒక file యొక్క hash ని ఊహించి (ఉదా. ఒక సాధారణ document) "నా దగ్గర ఇది ఉంది" అని చెప్పి, **upload చేయకుండానే** ఆ file ని పొందగలడు. దీన్ని **side-channel / confirmation attack** అంటారు.

పరిష్కారం: Dedupe ని server-side lo చేయడం (client ఎప్పుడూ upload చేస్తుంది, server duplicate అయితే reference మాత్రం పెంచుతుంది), లేదా per-user dedupe కి పరిమితం చేయడం. Dropbox ఈ కారణంగానే client-side global dedupe ని తొలగించింది.

6. Architecture – Metadata మరియు Block ని విడదీయడం

TWO INDEPENDENT SERVICES · ఇది ఎందుకు ముఖ్యం



7. Sync Flow & Conflicts

Upload: file మారింది → chunk చేయడం → hashes లెక్కించడం → **"ఈ hashes lo ఏవి server దగ్గర లేవు?"** అని అడగడం → లేనివి మాత్రం upload → metadata update.

Download: notification వచ్చింది → కొత్త chunk list తీసుకోవడం → **local lo లేని chunks** మాత్రం download → file ని reassemble.

Conflict దృశ్యం	పరిష్కారం
ఇద్దరు offline lo ఒకే file ని edit చేశారు	**రెండింటినీ ఉంచడం** – `report.docx` మరియు `report (Alice's conflicted copy).docx`. మౌనంగా ఒకటి పోగొట్టడం క్షమించరాని తప్పు

Conflict దృశ్యం	పరిష్కారం
Auto-merge చేయాలా?	**వద్దు** – binary files ని merge చేయలేం. Text కి కూడా Dropbox చేయదు. Merge అనేది application పని (Google Docs దాన్ని చేస్తుంది, file sync చేయకూడదు)
Version detection	ప్రతి file కి version vector / revision id. Client తన base revision ని పంపి, server అది తాజాదా అని check చేస్తుంది (optimistic concurrency)

8. Scale & Failure

సమస్య	పరిష్కారం
Upload మధ్యలో ఆగింది	Chunks స్వతంత్రం కాబట్టి సహజంగా resumable – ఇప్పటికే ఉన్న chunks ని మళ్ళీ పంపము
Chunk reference counting	ఒక chunk ని పలు files వాడుతున్నాయి. Delete చేసినప్పుడు వెంటనే తీసేయకూడదు. Refcount, లేదా mark-and-sweep GC (సురక్షితం, చౌక)
చాలా చిన్న files (లక్షలు)	ఒక్కో దానికి 4 MB chunk వృథా. చిన్న files ని ఒక "pack file" lo కలిపి store చేయడం (git packfiles లాంటిది)
Metadata DB scale	`user_id` ప్రకారం shard – ఒక user యొక్క file tree మొత్తం ఒకే shard lo, cross-shard queries లేవు
Notification fan-out	ఒక user కి 5 devices – చిన్న fan-out. కానీ shared folder lo 1000 మంది ఉంటే? అప్పుడు feed problem లాంటిది – active devices కి మాత్రమే push
Sync loop / storm	Client ఒక file రాస్తుంది → watcher గుర్తిస్తుంది → sync → server notify → client మళ్ళీ రాస్తుంది → అనంతం. **Client తన సొంత మార్పులని గుర్తించి ignore చేయాలి** (revision id తో)
Bandwidth ఖర్చు	LAN sync – ఒకే network lo ఉన్న devices ఒకదాని నుంచి ఒకటి chunks తీసుకోవడం, cloud నుంచి కాదు

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Let me start with the question this whole problem turns on. If I trim five seconds off the end of a one-gigabyte video, how much data should we upload? If the answer is a gigabyte, we've failed. It should be a few megabytes."

"Getting there means files are stored as **chunks**, not as whole objects. But the naive version – cut every four megabytes – breaks immediately. Insert a single byte at the start of a file and every subsequent boundary shifts by one byte, so every chunk hashes differently and you re-upload the entire file."

"The fix is **content-defined chunking**. I run a rolling hash across the file and cut wherever the hash matches a pattern – say the low thirteen bits are zero. Boundaries are then determined by content rather than position, so inserting a byte changes only the chunk containing it. Everything after it hashes identically. That single idea is what makes rsync, git, Docker layers and every deduplicating backup system work."

"Chunks are then **content-addressed** – the chunk's ID is the SHA-256 of its bytes. That gives deduplication for free: identical content is stored once. Uploading becomes: chunk the file, compute hashes, ask the server which hashes it's missing, send only those."

"I'd flag a security subtlety with global deduplication, though. If the client can skip an upload by claiming to already possess a hash, an attacker who can guess or obtain a file's hash can effectively download that file without ever having it. That's a confirmation attack, and it's why Dropbox removed client-side global dedupe. I'd either dedupe server-side – always upload, let the server collapse duplicates – or restrict dedupe to within a single user."

"Architecturally I'd split metadata from blocks, because they have opposite characteristics. Metadata – the file tree, versions, permissions, chunk lists – is small, transactional, frequently updated, and wants a relational database with strong consistency. Blocks are enormous, immutable and write-once, and want object storage fronted by a CDN. Serving both from one system makes both worse."

"For conflicts, my answer is deliberately unambitious: when two people edit offline, **keep both** as a conflicted copy. I would not auto-merge. Binary files can't be merged meaningfully, and silently discarding someone's work is the one unforgivable failure in a sync product. Merging is an application-layer concern – that's what Google Docs is for, and notably that's a completely different design."

"Two failure modes I'd design against specifically. First, chunk garbage collection: a chunk may be referenced by many files, so deletion can't be immediate – I'd use mark-and-sweep rather than refcounts, since refcounts get corrupted by partial failures. Second, sync loops: the client writes a file, its own watcher sees the change, syncs it, gets notified, and writes again. Clients must recognise and ignore their own changes by revision ID, or the system oscillates forever."

9. Follow-ups

అడిగేది	జవాబు
"Chunk size ఎంత ఉండాలి?"	చిన్నది → ఎక్కువ dedupe కానీ ఎక్కువ metadata. పెద్దది → తక్కువ metadata కానీ తక్కువ dedupe. CDC lo సగటు 4 MB, min 1 MB, max 8 MB (chunk పేలుడు ఆపడానికి bounds అవసరం)
"Encryption ఎలా?"	Client-side encryption అయితే dedupe పోతుంది (ఒకే content వేరే keys తో వేరే ciphertext). **Convergent encryption** – key = hash(content) – దీంతో dedupe ఉంటుంది కానీ confirmation attack తిరిగి వస్తుంది. ఇది నిజమైన trade-off
"Shared folder permissions?"	Metadata service lo ACL. Folder స్థాయిలో ఇచ్చి, children కి inherit. Permission మారితే ఆ folder ని చూస్తున్న అందరికీ re-sync notification
"Selective sync / smart sync?"	Metadata మొత్తం download చేసి, blocks ని on-demand. OS filesystem driver (placeholder files) తో file తెరిచినప్పుడు download
"Git కి ఇదే డిజైనా?"	చాలా దగ్గరిది – content-addressed objects, immutable, hash ids. తేడా: git కి పూర్తి history distributed గా ఉంటుంది, ఇక్కడ server అధికారికం. Git delta compression కూడా వాడుతుంది
"10 GB file ని ఎలా handle చేస్తారు?"	2,500 chunks – metadata చాలా పెద్దది. Chunk list ని కూడా chunk చేయడం (Merkle tree of chunks). అప్పుడు file యొక్క root hash ఒక్కటే metadata lo

PROBLEM 16 · CONTENTION UNDER LOAD

16

Design Ticket Booking at Scale (BookMyShow)

Difficulty **Hard** · Frequency **ఎక్కువ (Indian companies lo చాలా)** · నేర్పే concepts: inventory holds, overselling, waiting room

1. The Ask

> "Design BookMyShow. Sell seats for a show. Never sell the same seat twice. And survive a blockbuster opening where a million people arrive in the same minute for fifty thousand seats."

ఈ PROBLEM ప్రత్యేకత – EXTREME CONTENTION

ఇంతవరకు మనం చూసిన systems lo load **పరుచుకుని** ఉంటుంది – వేర్వేరు users, వేర్వేరు data. ఇక్కడ **ఒక్క show యొక్క seat map మీద ఒకేసారి 10 లక్షల మంది**. అంటే అన్ని requests ఒకే rows ని ముట్టుకుంటున్నాయి. Sharding ఇక్కడ సహాయం చేయదు – ఎందుకంటే అందరికీ కావలసినది ఒకే data. ఇదే "hot partition" యొక్క తీవ్రమైన రూపం.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Seat selection ఉందా, లేక general admission?	Numbered seats అంటే ప్రతి seat ఒక resource – contention తీవ్రం	Numbered seats
Payment పూర్తయ్యేదాకా seat ని పట్టుకోవాలా?	Hold mechanism అవసరమా	అవును, 10 నిమిషాలు
Overselling అంగీకారమా?	Airlines overbook చేస్తాయి. Cinema చేయదు	❌ ఎప్పటికీ కాదు
Peak load ఎంత?	Hot event architecture ని ఇది నిర్ణయిస్తుంది	1 M concurrent
Group booking (5 seats కలిసి)?	Atomic multi-seat reservation	అవును
Cancellation?	Inventory తిరిగి రావాలి	అవును

3. Estimation

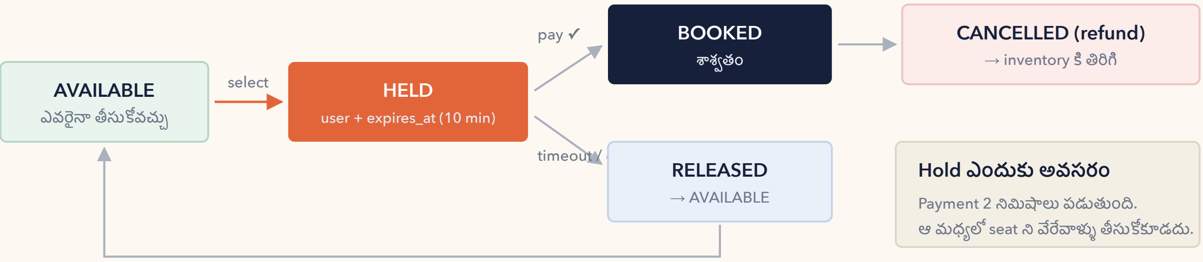
లెక్క	విలువ
సాధారణ రోజు bookings	1 M / day ≈ **12 / sec** – చాలా చిన్నది
Blockbuster opening	60 సెకన్లలో 1 M మంది → **~17,000 booking attempts / sec**
ఒక show lo seats	50,000 (పెద్ద stadium) లేదా 300 (cinema)
Seat map read QPS (hot event)	ప్రతి user ప్రతి 2 సెకన్లకి refresh → **500,000 reads / sec**
Read : Write	30 : 1 – reads వల్లే system కూలుతుంది

ఇక్కడ మొదటి INSIGHT

"సాధారణ load 12 QPS. Peak load 17,000 QPS. అంటే **1,400 రెట్లు తేడా**. ఈ system ని సగటుకి design చేస్తే opening night కూలుతుంది; peak కి design చేస్తే 364 రోజులు డబ్బు వృథా. అందుకే నా design lo **hot event ని ఒక ప్రత్యేక mode** గా చూస్తాను – waiting room, aggressive caching, pre-scaling. సాధారణ path ని సులభంగా ఉంచుతాను."

4. Deep Dive – Seat State Machine & Holds

SEAT LIFECYCLE · HOLD అనేది ఒక TTL ఉన్న STATE



Overselling ని ఆపే code

```
-- ❌ తప్పు: check-then-act. రెండు requests మధ్యలో దూరిపోతాయి.
SELECT status FROM seats WHERE seat_id = 'A12';        -- 'AVAILABLE'
UPDATE seats SET status = 'HELD' WHERE seat_id = 'A12';

-- ✓ సరైనది: ఒకే atomic conditional update.
UPDATE seats
   SET status = 'HELD', held_by = :userId, expires_at = NOW() + INTERVAL '10 minutes'
 WHERE seat_id = 'A12'
   AND (status = 'AVAILABLE'
        OR (status = 'HELD' AND expires_at < NOW()));    -- గడువు ముగిసిన hold ని
తీసుకోవచ్చు
-- rows affected = 1 → మనం గెలిచాం.  0 → వేరేవాళ్ళు ముందు తీసుకున్నారు.
```

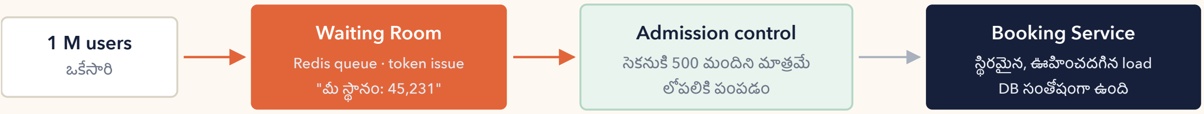
ఈ ఒక్క QUERY LO మూడు విషయాలు

1. Check మరియు act ఒకే statement lo – DB row lock దానంతట అదే race ని ఆపుతుంది.
2. `expires_at < NOW()` – గడువు ముగిసిన holds ని **read time lo** తీసుకోవడం. అంటే cleanup job ఆలస్యమైనా seats ఇరుక్కోవు (lazy expiry).
3. Group booking కి – 5 seats ని ఒకే transaction lo update చేసి, affected rows 5 కాకపోతే rollback. All-or-nothing.

5. Deep Dive – Hot Event (Virtual Waiting Room)

17,000 booking attempts/sec ని database కి పంపితే, ఎంత scale చేసినా ఒకే seat rows మీద lock contention వల్ల throughput కూలుతుంది. **పరిష్కారం: traffic ని database కి చేరనివ్వకపోవడం.**

VIRTUAL WAITING ROOM · LOAD ని సమయంలో పరచడం



ఎందుకు ఇది పని చేస్తుంది

50,000 seats ఉన్నాయి. 10 లక్షల మందిని లోపలికి పంపడం అర్థరహితం – 95% మంది ఎలాగూ ఖాళీ చేతులతో వెళ్తారు. వాళ్ళని ముందే ఆపి database ని కాపాడటం.
Load ని తగ్గించడం కాదు – సమయంలో పరచడం.

User అనుభవం కూడా మెరుగు

"Server error, మళ్ళీ ప్రయత్నించండి" కంటే "మీ స్థానం 45,231 · సుమారు 6 నిమిషాలు" చాలా మేలు. Refresh కొట్టే ఉద్ధృతి కూడా ఆగుతుంది – అది లేకపోతే users తామే load ని రెట్టింపు చేస్తారు.

Hot event lo మిగతా రక్షణలు	ఎలా
Seat map ని cache చేయడం	500K reads/sec ని DB కి పంపొద్దు. Redis lo seat map, 1-2 సెకన్ల TTL. కొంచెం stale – user ఒక seat select చేసినప్పుడు అది పోయి ఉండొచ్చు, అది ఫర్వాలేదు
Read/write ని విడదీయడం	Availability chart ఒక cached read. అసలు reservation ఒక్కటే DB write
Pre-scaling	Show ఎప్పుడు open అవుతుందో మనకి తెలుసు. ముందే servers, DB connections, cache ని warm చేయడం
Bot protection	Hot events lo 50%+ traffic bots (scalpers). CAPTCHA at waiting room entry, device fingerprint, per-account limits
Queue lo fairness	FIFO ప్రధానం, కానీ ముందుగానే login అయినవాళ్ళకి ప్రాధాన్యం ఇవ్వొచ్చు – refresh చేసేవాళ్ళు గెలవకుండా

6. Scale & Failure

సమస్య	పరిష్కారం
Payment fail అయ్యాక seat	Hold expire అయ్యే వరకు ఆగకుండా వెంటనే release. కానీ **payment status తెలియకపోతే** (timeout) – release చేయకూడదు, hold ని పొడిగించి PSP ని అడగాలి (Problem 09 logic)
Hold expiry job down	Lazy expiry (పైన query lo `expires_at < NOW()`) వల్ల system పని చేస్తూనే ఉంటుంది. Cleanup job కేవలం optimisation
DB single point	Seat inventory ని shard చేయడం – `show_id` ప్రకారం. ఒక show ఒకే shard lo (అది సహజం – ఒక show యొక్క seats ఎప్పుడూ కలిసి ఉంటాయి)
ఒకే show hot shard	ఇదే ఈ problem యొక్క పరిమితి. Waiting room తప్ప వేరే మార్గం లేదు. Seat block ప్రకారం sub-shard చేయొచ్చు (Block A, B, C వేరే rows)
Double booking bug	Unique constraint మీద ఆధారపడటం – `UNIQUE(show_id, seat_number, status='BOOKED')` (partial index). Application bug ఉన్నా DB ఆపుతుంది. **చివరి రక్షణ ఎప్పుడూ DB constraint**
Cache stale వల్ల నిరాశ	User "ఖాళీ" అని చూసిన seat తీసుకోబోతే fail. UI lo దీన్ని సున్నితంగా చూపడం + వెంటనే fresh seat map. దీన్ని పూర్తిగా ఆపలేం – honest గా చెప్పండి

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"Two questions matter most here: are seats individually numbered, and is overselling ever acceptable? Numbered seats mean each one is a contended resource. And unlike airlines, a cinema can't overbook – so 'never sell the same seat twice' is a hard constraint, not a target."

"The defining characteristic of this system is the load profile. A normal day is about twelve bookings a second. A blockbuster on-sale is seventeen thousand a second – a factor of over a thousand. And unlike every previous problem, that load isn't spread across data. A million people all want the **same fifty thousand rows**. Sharding doesn't help when everyone wants the same partition."

"The core mechanism is a seat state machine with holds. A seat is available, held, or booked. Selecting a seat puts it in **held** with an expiry, because payment takes a couple of minutes and nobody else may take it meanwhile."

"The critical detail is how the hold is taken. The obvious version – check availability, then update – has a race between the two statements. Instead it must be a **single conditional update**: set the seat to held where it is currently available, or where an existing hold has already expired. If rows-affected is one we won, if it's zero somebody beat us. The database's row lock does the concurrency control for us, with no distributed lock service."

"Including the expired-hold case in the same predicate matters: it means holds expire **lazily at read time**. If my cleanup job is delayed or down, seats still become bookable again rather than being stranded."

"For group bookings, all seats update in one transaction and I check that rows-affected equals the number requested, otherwise roll back. All or nothing."

"Now the hard part – the on-sale spike. No amount of horizontal scaling fixes contention on the same rows; past a point throughput actually drops as lock waiting grows. So the answer isn't to scale the database, it's to **not send the traffic there**. I'd put a virtual waiting room in front: everyone gets a queue token, and admission control lets a few hundred people per second into the actual booking flow."

"The reasoning is simple: there are fifty thousand seats and a million people. Letting all of them into the booking path is pointless – most will leave empty-handed regardless. The waiting room doesn't reduce the load, it **spreads it over time**. And it's a better user experience: 'you are number 45,231, about six minutes' beats a 500 error, and it stops the refresh storm where users double the load themselves."

"Alongside that I'd cache the seat map in Redis with a one-second TTL, since reads outnumber writes thirty to one and a slightly stale map is acceptable – occasionally a user picks a seat that's just gone, and the UI handles that gracefully."

"Finally, whatever the application logic does, I'd back it with a database uniqueness constraint on booked seats. Application bugs happen; that constraint is the last line that makes double-booking impossible rather than unlikely."

7. Follow-ups

అడిగేది	జవాబు
"Redis lo hold పెట్టొచ్చా, DB కాకుండా?"	వేగం ఎక్కువ (`SET NX EX`), కానీ Redis persistence బలహీనం – restart అయితే holds పోతాయి. Holds ఫర్వాలేదు (అవి ఎలాగూ ephemeral) కానీ **BOOKED state మాత్రం ఎప్పుడూ durable DB lo**. Hybrid: hold Redis lo, booking DB lo
"General admission (seat numbers లేకపోతే)?"	ఇది చాలా సులభం – ఒక counter. `DECR available` atomic. అప్పుడు contention ఒక్క key మీద, దాన్ని sharded counters తో పరచొచ్చు (`available:shard_0..9`)
"Fairness ఎలా – bots గెలవకుండా?"	Waiting room lo entry timestamp ప్రకారం FIFO, CAPTCHA, per-account seat limit, device fingerprinting, payment method verification. పూర్తిగా ఆపలేం కానీ ఖరీదు పెంచొచ్చు
"Seat map ఎలా cache చేస్తారు?"	Bitmap – ఒక్కో seat కి 1 bit. 50,000 seats = 6 KB. దీన్ని Redis lo ఉంచి 1 సెకను TTL. Bitmap కాబట్టి transfer చాలా చౌక
"Analytics – ఏ seats ముందు అమ్ముడయ్యాయి?"	Booking events ని Kafka కి. అది వేరే pipeline, booking path ని నెమ్మది చేయకూడదు
"Multi-city, multi-screen?"	`show_id` ఒక సహజ shard key. వేర్వేరు shows పూర్తిగా స్వతంత్రం – ఇది ఈ domain యొక్క పెద్ద అదృష్టం

PROBLEM 17 · STREAM PROCESSING

17

Design Ad Click Aggregation

Difficulty **Hard** · Frequency **ఎక్కువ (data-heavy companies lo)** · నేర్చే concepts: windowing, watermarks, exactly-once, lambda architecture

1. The Ask

> "Design a system that counts ad clicks. Advertisers see near-real-time dashboards, and we bill them from the same data – so the numbers must be right."

ఈ PROBLEM LO రెండు విరుద్ధమైన అవసరాలు

Dashboard కి కావలసినది వేగం – advertiser తన campaign ఎలా నడుస్తోందో ఇప్పుడే చూడాలి, ఒక నిమిషం పాతది అయినా ఫర్వాలేదు.
Billing కి కావలసినది ఖచ్చితత్వం – ఒక్క click తప్పుగా లెక్కిస్తే అది డబ్బు తప్పు.

ఒకే pipeline తో రెండింటినీ ఇవ్వలేం. ఈ ఉద్రిక్తతని మీరు గుర్తించి, **రెండు పొరలతో** పరిష్కరించడమే ఈ interview యొక్క గుండె.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Dashboard latency ఎంత?	1 సెకనా, 1 నిమిషమా – పూర్తిగా వేరే designs	~1 నిమిషం సరిపోతుంది
Billing కి approximate సరిపోతుందా?	ఇదే కీలక ప్రశ్న	❌ లేదు, ఖచ్చితం కావాలి
Aggregation granularity?	నిమిషం, గంట, రోజు × ad_id, country, device	నిమిషానికి, పలు dimensions
Late events ఎంత ఆలస్యం కావచ్చు?	Mobile offline lo ఉండి గంటల తర్వాత పంపొచ్చు	1 గంట దాకా
Duplicate clicks?	Retry, bot, double-tap	Dedupe కావాలి
Query patterns?	Ad-hoc slicing కావాలా	అవును – OLAP

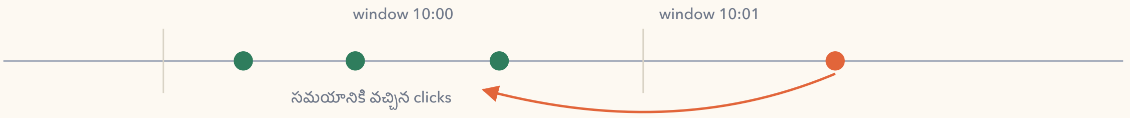
3. Estimation

లెక్క	విలువ
Ad impressions / day	100 B
Click-through rate	~1%
Clicks / day	1 B
Avg click QPS	1B ÷ 86,400 ≈ **12,000 / sec**
Peak	≈ 50,000 / sec
Event size	≈ 200 B (ad_id, user, ts, country, device, cost)
Raw events / day	1B × 200 B = **200 GB / day**
Aggregated (నిమిషానికి × 1M ad_ids)	1M × 1440 × 50 B ≈ 72 GB/day – raw కంటే చిన్నది

4. Deep Dive – Windowing & Watermarks

EVENT TIME VS PROCESSING TIME · LATE EVENTS అనే సమస్య

WINDOW 10:00 - 10:01 · నిమిషానికి aggregate



10:00:45 కి జరిగిన click,
10:01:30 కి వచ్చింది (mobile offline)
→ ఇది 10:00 window కి చెందాలి,
వచ్చిన సమయం ప్రకారం కాదు

Event time ని వాడాలి, processing time కాదు

Event lo ఉన్న timestamp ప్రకారం window కి కేటాయించడం.
లేకపోతే offline user యొక్క clicks తప్పు నిమిషానికి పడతాయి.

Watermark – "ఇక ఈ window కి ఏమీ రాదు"

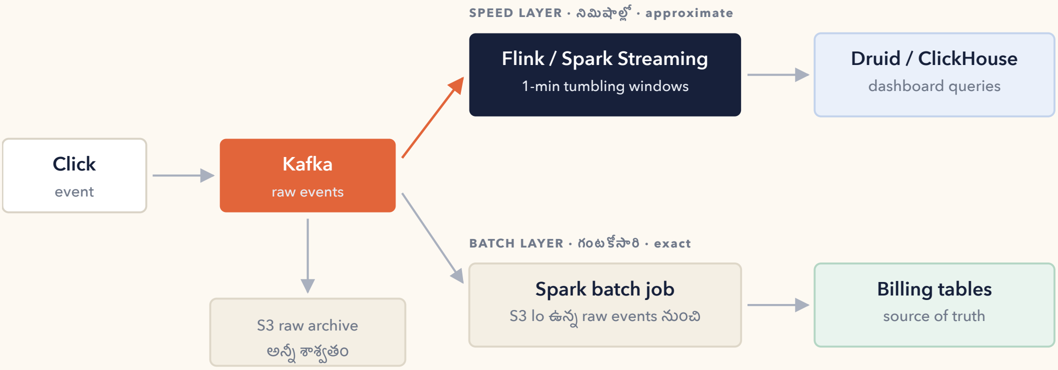
10:00 window ని 10:00:59 కి మూసేయలేం. Late events కోసం
5 నిమిషాలు ఆగి, తర్వాత మూయడం. Watermark = ఆ నిర్ణయం.

Watermark తర్వాత వచ్చినవి?

Dashboard: వదిలేయడం (1% తేడా ఫర్వాలేదు).
Billing: batch job అన్నిటినీ మళ్ళీ లెక్కిస్తుంది – ఏదీ పోదు.

5. Deep Dive – Lambda Architecture (రెండు పొరలు)

SPEED LAYER + BATCH LAYER · ఒకే DATA, రెండు హామీలు



Batch layer ఎప్పుడూ speed layer ని దిద్దుతుంది – late events, bugs, reprocessing అన్నీ ఇక్కడ సరిపోతాయి

ఇది duplication కాదు, ఉద్దేశపూర్వక redundancy. Speed layer తప్పు చేయొచ్చు – వేగం కోసం. Batch layer నెమ్మది కావొచ్చు – ఖచ్చితత్వం కోసం. Batch ఫలితం వచ్చాక అదే అధికారికం. Streaming pipeline lo bug కనిపిస్తే, code fix చేసి raw events ని మళ్ళీ process చేయొచ్చు – **raw ని ఎప్పుడూ దాచుకోవడం** అందుకే.

6. Deep Dive – Aggregation lo Exactly-Once

Stream job crash అయితే Kafka offset నుంచి మళ్ళీ చదువుతుంది. అప్పుడు కొన్ని events రెండుసార్లు లెక్కించబడతాయి – **count తప్పు అవుతుంది**.

Approach	ఎలా
❌ `UPDATE counts SET n = n + 1`	Increment idempotent కాదు. Retry lo count పెరిగిపోతుంది
✅ **Idempotent upsert by (window, key)**	Aggregate result ని `(ad_id, minute) → count` గా **overwrite** చేయడం. అదే window ని మళ్ళీ process చేసినా అదే విలువ వస్తుంది
✅ Checkpointing (Flink)	Operator state + offsets ని atomic గా checkpoint. Restore చేసినప్పుడు ఖచ్చితంగా అదే స్థానం నుంచి

ఇది ఎందుకు అందమైన పరిష్కారం

"Increment బదులు **overwrite** చేయడం – ఈ ఒక్క మార్పుతో exactly-once సమస్య పోతుంది. ఎందుకంటే 'ad-42, 10:03వ నిమిషానికి 1,847 clicks' అనేది ఒక **పూర్తి, స్వతంత్ర సత్యం**. దాన్ని ఎన్నిసార్లు రాసినా ఫలితం ఒకటే. ఇదే idempotency యొక్క సాధారణ రూపం – *delta కాకుండా final state* రాయడం. Notification system lo idempotency key, ఇక్కడ (window, key) – ఒకే ఆలోచన."

7. Scale & Failure

సమస్య	పరిష్కారం
Hot ad (viral campaign)	ఒక ad_id ఒకే partition కి వెళ్తుంది → hot. **Two-level aggregation**: ముందు local pre-aggregation (ప్రతి worker తన దగ్గర 10 సెకన్ల partial sums), తర్వాత global merge. Network traffic మరియు contention రెండూ తగ్గుతాయి
Duplicate clicks (double tap, retry)	Client ప్రతి click కి ఒక `click_id` (UUID). Dedupe window (5 నిమిషాలు) lo bloom filter / Redis set తో filter
Click fraud	వేరే pipeline – IP velocity, device fingerprint, ఒకే user పలుసార్లు, unrealistic CTR. Fraud clicks ని flag చేసి billing నుంచి తీసేయడం
Stream job crash	Flink checkpoint నుంచి restore. Kafka retention (7 రోజులు) వల్ల ఎంత వెనక్కైనా వెళ్ళొచ్చు
Kafka lag	Aggregation stateful కాబట్టి simply scale చేయలేం – partitions ని పెంచి rebalance చేయాలి. అందుకే **ముందే ఉదారంగా partitions**
Storage పెరుగుదల	Raw events 30 రోజులు hot, తర్వాత Glacier. Aggregates: నిమిషానికి 7 రోజులు → గంటకి 90 రోజులు → రోజుకి శాశ్వతం (**rollup tiers**)
Timezone / billing period	Aggregates ని ఎప్పుడూ UTC lo store చేసి, query time lo advertiser timezone కి మార్చడం

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"The question I'd ask first is whether approximate numbers are acceptable – and I expect the answer to be different for different consumers. A dashboard can tolerate being a percent off. Billing cannot, because that's money. Those two requirements pull in opposite directions, and I don't think one pipeline can serve both honestly."

"About a billion clicks a day is twelve thousand a second, peaking around fifty. That's not extreme throughput. The difficulty is **time**."

"Specifically: a click that happened at 10:00:45 might not reach us until 10:01:30, because the user's phone was offline. If I bucket by arrival time, that click lands in the wrong minute and every report is subtly wrong. So aggregation must use **event time** – the timestamp inside the event – not processing time."

"Which raises the next question: when can I close the 10:00 window? Not at 10:01, or I lose every late arrival. I use a **watermark** – wait some grace period, say five minutes, then declare the window final. Anything later than that is where the two consumers diverge: the dashboard drops it, and the batch layer picks it up."

"So the architecture has two layers over the same Kafka stream. A **speed layer** – Flink doing one-minute tumbling windows into an OLAP store like Druid – gives dashboards fresh, approximate numbers. A **batch layer** reprocesses the raw events archived in object storage on an hourly schedule and produces the authoritative billing numbers. The batch result always supersedes the streaming result."

"That's not wasteful duplication, it's deliberate. It also means that if I find a bug in my streaming logic, I can fix the code and reprocess history – which is only possible because I keep the raw events forever rather than only the aggregates."

"On correctness in the stream: if the job crashes and replays from an offset, naive counting double-counts. The fix is to stop thinking in increments. Instead of 'add one', I write 'ad-42, minute 10:03, count 1847' as an **idempotent upsert keyed on window and ad**. Reprocessing the same window produces the same value, so replay is harmless. It's the same principle as the idempotency key in the payment system – write final state, not deltas."

"The scaling problem specific to this domain is hot keys. A viral campaign sends all its clicks to one partition. I'd solve it with **two-level aggregation**: each worker pre-aggregates locally over a few seconds, and only those partial sums are shuffled for the global merge. That cuts both network volume and contention on the hot key dramatically."

"And I'd keep click deduplication and fraud detection as separate concerns – a client-generated click ID with a short dedupe window handles accidental duplicates, while systematic fraud needs its own scoring pipeline whose output subtracts from billable clicks."

8. Follow-ups

అడిగేది	జవాబు
"Kappa architecture ఎందుకు కాదు?"	Kappa = streaming ఒక్కటే, batch layer లేదు (bug వస్తే stream ని replay చేయడం). ఇది సులభం, maintain చేయడానికి ఒకే codebase. కానీ **billing కి ఒక స్వతంత్ర, verifiable batch computation** ఉండటం audit కి విలువైనది. Financial data లేకపోతే నేను Kappa కే వెళ్తాను
"Unique users ఎలా లెక్కిస్తారు?"	Exact count అంటే అన్ని user ids store చేయాలి – భారీ. **HyperLogLog** – ~2% error తో 12 KB lo కోట్ల unique values. Dashboard కి ఇది సరిపోతుంది
"OLAP store ఎందుకు, Postgres కాదు?"	Query pattern: "గత 7 రోజుల్లో country ప్రకారం clicks" – ఇది columnar scan. Druid/ClickHouse column-oriented + pre-aggregation, ఇలాంటి queries కి 100× వేగం
"Tumbling vs sliding vs session windows?"	**Tumbling** – అతివ్యాప్తి లేని fixed buckets (నిమిషానికి count). **Sliding** – అతివ్యాప్తి ఉన్నవి (గత 5 నిమిషాల moving average). **Session** – activity gap ఆధారంగా (user session analysis)
"Backfill ఎలా?"	Raw events S3 lo ఉన్నాయి కాబట్టి ఏ కాలాన్నయినా మళ్ళీ process చేయొచ్చు. Aggregate table lo (window, key) upsert కాబట్టి backfill idempotent
"Real-time budget enforcement (బడ్జెట్ అయిపోతే ad ఆపడం)?"	ఇది వేరే, కఠినమైన problem – నిమిషం ఆలస్యం అంటే overspend. Ad serving path lo ఒక Redis counter, approximate కానీ వేగం. Batch తర్వాత సరిచేయడం

PROBLEM 18 · TIME-SERIES AT SCALE

18

Design Metrics & Monitoring (Prometheus)

Difficulty **Hard** · Frequency **SSE/SDE3 lo ఎక్కువ** · నేర్పే concepts: cardinality, time-series compression, rollups, alerting

1. The Ask

> "Design a monitoring system. A hundred thousand servers report metrics, engineers query dashboards, and alerts fire when something breaks."

ఈ PROBLEM LO అసలు శత్రువు: CARDINALITY

"సెకనుకి కోటి datapoints" అనేది భయపెడుతుంది, కానీ నిజానికి అది సులభమైన భాగం – time-series data అద్భుతంగా compress అవుతుంది. **నిజమైన హంతకుడు cardinality** – ఎన్ని విభిన్న time series ఉన్నాయి అన్నది. ఒక engineer ఒక metric కి `user_id` అనే label జోడిస్తే, ఒక్క రాత్రిలో 1 series 10 million series అవుతుంది, database కూలుతుంది. ఈ ఒక్క విషయాన్ని అర్థం చేసుకున్న candidate చాలా అరుదు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Metrics మాత్రమేనా, logs/traces కూడానా?	మూడూ పూర్తిగా వేరే systems	Metrics మాత్రమే
Scrape interval?	Datapoint rate ని నిర్ణయిస్తుంది	10 సెకన్లు
Retention?	Storage + rollup strategy	Raw 15 రోజులు, rollups 2 ఏళ్ళు
Push నా pull నా?	Architecture ని మౌలికంగా మారుస్తుంది	Pull (Prometheus model)
Query patterns?	ఇటీవలివి ఎక్కువా, పాతవా	95% queries గత 24 గంటలు
Alerting scope lo ఉందా?	ఉంటే evaluation engine కావాలి	అవును

3. Estimation

లెక్క	విలువ
Hosts	100,000
Host కి metrics (labels తో సహా)	1,000 series
మొత్తం active time series	**100 M**
Scrape interval	10 సెకన్లు
Datapoints / sec	100M ÷ 10 = **10 M / sec**
Naive datapoint size	timestamp 8 B + value 8 B = **16 B**
Naive storage / day	10M × 86,400 × 16 B = **13.8 TB / day** ❌
Compression తర్వాత (~1.4 B/point)	≈ **1.2 TB / day** ✅

4. Deep Dive – Time-Series Compression

GORILLA COMPRESSION · 16 BYTES → ~1.4 BYTES PER POINT

TIMESTAMPS · DELTA-OF-DELTA

```
t: 10:00:00, 10:00:10, 10:00:20, 10:00:30…
delta: 10, 10, 10
delta-of-delta: 0, 0
```

Scrape interval స్థిరం → దాదాపు ఎప్పుడూ 0 → 1 bit lo!

VALUES · XOR

```
CPU: 42.1, 42.3, 42.2, 42.4…
```

వరుస values దాదాపు ఒకేలా ఉంటాయి →
XOR చేస్తే ఎక్కువ bits సున్నా →
సున్నా కాని bits ని మాత్రం store చేయడం

ఇది ఎందుకు పని చేస్తుంది: time-series data నిజ ప్రపంచంలో నెమ్మదిగా మారుతుంది

CPU 42% నుంచి 89% కి ఒక్క datapoint lo దూకదు. ఈ ఊహ మీద మొత్తం compression నిలబడుతుంది.

Columnar layout

Timestamps ఒక column, values ఒక column –
ఒకే రకమైన data పక్కపక్కన → compression మెరుగు

ఫలితం

13.8 TB/day → 1.2 TB/day. ఇది optimisation కాదు –
ఇది లేకపోతే ఈ system ఆర్థికంగా సాధ్యం కాదు.

5. Deep Dive – Cardinality (అసలు ప్రమాదం)

```
http_requests_total{method="GET", status="200", host="web-01"}     ← ఒక series
```

Series సంఖ్య = **అన్ని label విలువల గుణకారం**.

Labels	ప్రత్యేక విలువలు	Series
`method`	5	5
`status`	10	50

Labels	ప్రత్యేక విలువలు	Series
`host`	100,000	**5,000,000**
+ `user_id` జోడిస్తే	10,000,000	**50,000,000,000** 💀

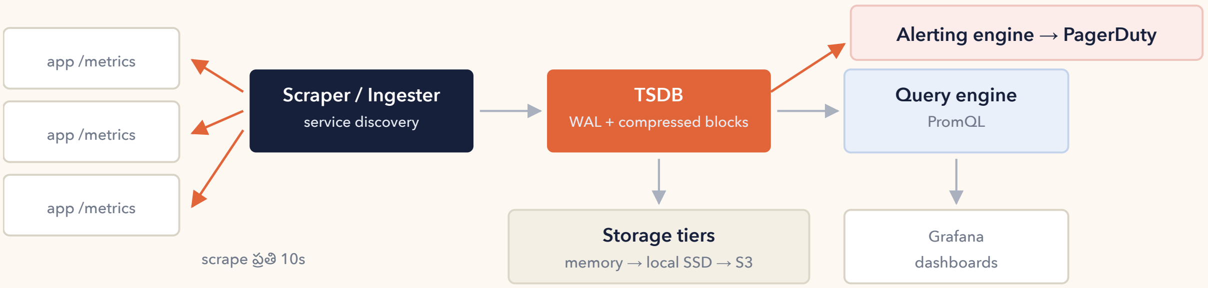
ఇది ఎందుకు SYSTEM ని చంపుతుంది

ప్రతి unique series కి **ఒక ప్రత్యేక file/chunk + index entry + memory lo ఒక active writer** ఉంటుంది. Datapoints సంఖ్య పెరగలేదు — కానీ **series సంఖ్య** పెరిగింది. Memory పేలుతుంది, index పేలుతుంది, queries నెమ్మదిస్తాయి. అందుకే నియమం: **label విలువలు తక్కువ, పరిమితమైన సమూహంలో (bounded set) ఉండాలి.** `user_id`, `request_id`, `email`, `url with params` — ఇవి ఎప్పుడూ labels కాకూడదు. అవి **logs కి చెందినవి, metrics కి కాదు.**

రక్షణ	ఎలా
Per-metric series limit	ఒక metric 10,000 series దాటితే reject + alert. System ని కాపాడటం
Label validation	High-cardinality label patterns ని ingestion lo block చేయడం
Cardinality dashboard	ఏ metrics ఎక్కువ series తింటున్నాయో teams కి చూపడం
సరైన tool	High-cardinality debugging కి logs/traces వాడాలి, metrics కాదు

6. Architecture & Pull vs Push

MONITORING ARCHITECTURE · PULL MODEL



Pull యొక్క లాభం

Target reachable కాకపోతే అదే ఒక signal ("up=0"). Rate limiting scraper చేతిలో. Config ఒకచోట.

Push ఎప్పుడు అవసరం

Short-lived batch jobs — scrape చేసేలోపు అవి చనిపోతాయి. వాటికి ఒక push gateway.

7. Rollups & Retention

Raw data ని 2 ఏళ్ళు ఉంచడం అసాధ్యం (1.2 TB/day × 730 = 876 TB). కానీ ఎవరూ ఏడాది క్రితం 10-సెకన్ల resolution అడగరు.

Tier	Resolution	Retention	Storage
Hot	10 సెకన్లు	15 రోజులు	≈ 18 TB (local SSD)

Tier	Resolution	Retention	Storage
Warm	5 నిమిషాలు	90 రోజులు	≈ 3.5 TB
Cold	1 గంట	2 ఏళ్ళు	≈ 1.2 TB (S3)

ROLLUP LO ఒక సూక్ష్మత

"5-నిమిషాల rollup lo నేను **ఒకే average** store చేయను – `min, max, sum, count` నాలుగూ store చేస్తాను. ఎందుకంటే averages ని తిరిగి aggregate చేయలేం (average of averages తప్పు), కానీ sum మరియు count ని చేయొచ్చు. పైగా **spike ని కోల్పోకూడదు** – 5 నిమిషాల్లో ఒక సెకను CPU 100% కి వెళ్తే, average lo అది కనిపించదు, max lo కనిపిస్తుంది. Incident debugging lo max యే ముఖ్యం."

8. Alerting

సమస్య	పరిష్కారం
Flapping (metric limit చుట్టూ ఊగడం)	`for: 5m` – condition 5 నిమిషాలు నిజంగా ఉంటేనే alert. ఒక్క spike కి page చేయకూడదు
Alert storm	ఒక DB down అయితే 500 services alert చేస్తాయి. **Grouping + inhibition** – root cause alert ఉంటే దాని పైన ఆధారపడిన alerts ని అణచడం
Alert fatigue	Actionable alerts మాత్రమే page చేయాలి. మిగతావి dashboard/ticket. Page అయిన ప్రతిసారీ మనిషి ఏదో చేయగలగాలి – లేకపోతే అది noise
Alerting engine SPOF	Alerting ని monitoring కంటే వేరే, సరళమైన, ఎక్కువ HA ఉన్న path lo. **Monitoring down అయితే కూడా అది తెలియాలి** – dead man's switch (ఎప్పుడూ fire అయ్యే alert; అది రాకపోతే system down)
Query load వల్ల ingestion దెబ్బతినడం	Read path మరియు write path ని వేరు చేయడం – వేరే replicas. Dashboard query storm ingestion ని ఆపకూడదు

🎤 **INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)**

"First: metrics only, or logs and traces too? They're three different systems with different data models, and I'd design metrics here. I'd also settle push versus pull early, since it changes the architecture."

"A hundred thousand hosts with a thousand series each scraped every ten seconds is a hundred million active series and ten million datapoints a second. Naively that's sixteen bytes a point and fourteen terabytes a day, which nobody would pay for."

"That's solved by compression, and it's worth explaining why it works so well. Timestamps are almost perfectly regular, so I store the **delta of the delta** – with a fixed scrape interval that's nearly always zero, which fits in a single bit. Values change slowly in the real world; CPU doesn't jump from forty-two to eighty-nine percent between adjacent samples. So I XOR consecutive values and store only the non-zero bits. Together that's the Gorilla scheme, and it takes sixteen bytes down to under one and a half. That isn't an optimisation, it's what makes the system economically possible at all."

"But I'd argue the datapoint rate is the **easy** half. The thing that actually kills monitoring systems is **cardinality** – the number of distinct series, which is the product of all label value counts. Method times status times host is already five million series. The moment somebody adds a user ID label, it becomes fifty billion."

"That's fatal because every unique series needs its own chunk, index entry and in-memory writer. The datapoint volume hasn't changed at all – the series count has. So I'd enforce hard limits: reject metrics past a series budget, block known high-cardinality label patterns at ingestion, and give teams a cardinality dashboard so they can see who's spending the budget. The underlying rule is that labels must have bounded value sets. User IDs and request IDs belong in logs and traces, not metrics."

"I'd use a **pull** model. Targets expose an endpoint and the scraper collects on a schedule. Pull is nicer operationally: failing to scrape is itself a signal that the target is down, rate limiting sits with the scraper rather than with thousands of clients, and configuration lives in one place. The exception is short-lived batch jobs that finish before anyone can scrape them, which need a push gateway."

"For retention I'd use rollup tiers – full resolution for two weeks on local SSD, five-minute rollups for ninety days, hourly for two years in object storage. One detail I'd insist on: a rollup stores min, max, sum and count, not just an average. Averages can't be re-aggregated correctly, and more importantly a one-second spike to a hundred percent disappears in a five-minute average but survives in the max – and during an incident the max is the number you actually need."

"On alerting, the failure modes are human. Alerts need a duration condition so a momentary blip doesn't page anyone, and they need grouping and inhibition, or one database outage pages five hundred times. And the alerting path should be simpler and more available than the monitoring system itself – including a dead man's switch, an alert that always fires, so that **silence** tells you monitoring is down rather than that everything is fine."

9. Follow-ups

అడిగేది	జవాబు
"Logs కి ఇదే design పని చేస్తుందా?"	లేదు. Logs high-cardinality, unstructured, పెద్దవి. అక్కడ inverted index (Elasticsearch) లేదా columnar + brute force scan (Loki). Metrics compression ఇక్కడ పని చేయదు
"Traces ఎలా వేరు?"	Trace = ఒక request యొక్క spans tree. Cardinality అపారం (ప్రతి request ప్రత్యేకం) కాబట్టి **sampling** తప్పనిసరి (ఉదా. 1%, కానీ errors ని 100%)
"Prometheus HA ఎలా?"	రెండు identical Prometheus instances ఒకే targets ని scrape చేయడం (data duplicate కానీ independent). Long-term + global view కి Thanos/Cortex
"Histogram ఎలా store చేస్తారు?"	Buckets గా – `le="0.1"`, `le="0.5"`… ప్రతి bucket ఒక series. అందుకే histograms cardinality ని వేగంగా పెంచుతాయి. p99 ని buckets నుంచి interpolate చేస్తారు – అది approximate అని గుర్తుంచుకోవాలి
"Downsampling ఎప్పుడు చేయాలి?"	Background compaction lo, ingestion path lo కాదు. Ingestion ని ఎప్పుడూ నెమ్మదిస్తే మొత్తం system దెబ్బతింటుంది
"Query timeout / expensive query?"	Series limit + time range limit + query timeout. `sum by (user_id)` లాంటి query 50 million series ని touch చేయొచ్చు – దాన్ని ముందే reject చేయాలి

ము గిం పు

18 problems తర్వాత – మీరు ఏం నేర్చుకున్నారు

ఇది revision page · interview ముందు రోజు ఇది ఒక్కటే చదివితే చాలు

తిరిగి తిరిగి వచ్చిన 8 ఆలోచనలు

ఆలోచన	ఏఏ problems lo వచ్చింది	సారాంశం
పనిని ఎప్పుడు చేయాలి – write time నా read time నా	03 feed · 08 typeahead · 17 analytics	Read-heavy అయితే write time lo పని చేయి. అదే మొత్తం system design lo అతి ముఖ్యమైన ప్రశ్న
At-least-once + idempotency	05 · 09 · 10 · 13 · 17	Exactly-once delivery సాధ్యం కాదు. Delta కాకుండా final state రాయడం, లేదా idempotency key
Lease, lock కాదు	07 uber · 10 scheduler · 16 booking	TTL ఉన్న claim. Worker చస్తే తనంతట తానే విడుదల
Conditional update (compare-and-set)	07 · 10 · 16	`UPDATE … WHERE status = expected`. Distributed lock service అవసరం లేని concurrency control
Hot key / hot partition	01 · 03 · 07 · 13 · 16 · 17	ఏ system lo అయినా load ఎప్పుడూ సమానంగా ఉండదు. Sub-sharding, two-level aggregation, waiting room
Approximate ఎప్పుడు సరిపోతుంది	02 rate limit · 11 bloom filter · 18 histogram	ఏ తప్పు భరించగలమో తెలిస్తే చాలా చౌక పరిష్కారాలు తెరుచుకుంటాయి. Payment lo మాత్రం ఎప్పటికీ కాదు
Fail-open vs fail-closed	02 · 05 · 09	Rate limiter down → allow. Preference service down → block. ఏది ప్రమాదకరమో అది నిర్ణయిస్తుంది
Requirement ని కాస్త వదులు చేయడం	04 presence · 10 jitter · 12 CRDT	"ప్రతి గంట" ≠ ":00:00". "Online status" ≠ real-time. ఈ చిన్న సడలింపులు భారీ engineering ని ఆదా చేస్తాయి

INTERVIEW ముందు 60 సెకన్ల CHECKLIST

1. Clarifying questions అడిగానా? Assumptions బోర్డ్ మీద రాశానా?
2. Numbers ఇచ్చానా? (QPS, storage, bandwidth)
3. Read:write ratio చెప్పానా? అది design ని ఎలా మార్చిందో చెప్పానా?
4. ప్రతి component కి ఎందుకు అన్నది చెప్పానా? రెండు options compare చేశానా?
5. Deep dive కి చేరానా? (ఇక్కడే seniority తెలుస్తుంది)
6. Failure గురించి మాట్లాడానా? ఏది down అయితే ఏమవుతుంది?
7. Bottleneck ఎక్కడో పేరు పెట్టానా?
8. చివర్లో ఒక వాక్యంలో summary ఇచ్చానా?

> **చివరి మాట:** ఈ 18 problems lo ఏదీ interview lo అచ్చంగా అలానే రాదు. కానీ ఇక్కడ ఉన్న **techniques** ప్రతి problem lo వస్తాయి. మీరు గుర్తుంచుకోవాల్సింది URL shortener యొక్క architecture కాదు – **read-heavy system ని ఎలా గుర్తించాలి, గుర్తించాక ఏం చేయాలి** అనేది. అదే తేడా.